OGC® DOCUMENT: 20-058

External identifier of this OGC® document: http://www.opengis.net/doc/{doc-type}/{standard}/{m.n}

Open Geospatial Consortium

OGC API - MAPS - PART 1: CORE

STANDARD
Implementation

DRAFT

Version: 1.0
Submission Date: 2024-02-15
Approval Date: 2029-03-30
Publication Date: 2029-03-30
Editor: Joan Masó, Jérôme Jacovella-St-Louis

Notice for Drafts: This document is not an OGC Standard. This document is distributed for review and comment. This document is subject to change without notice and may not be referred to as an OGC Standard.
Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

License Agreement

Use of this document is subject to the license agreement at https://www.ogc.org/license

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: http://ogc.standardstracker.org/

Copyright notice

Copyright © 2026 Open Geospatial Consortium
To obtain additional rights of use, visit https://www.ogc.org/legal

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I. ABSTRACT viii

II. KEYWORDS viii

III. PREFACE ix

IV. SECURITY CONSIDERATIONS x

V. SUBMITTING ORGANIZATIONS xi

VI. SUBMITTERS xi

VII. LEGACY xi

VIII. ACKNOWLEDGEMENTS xii

1. SCOPE 2

2. CONFORMANCE 4
   - 2.1. Requirements classes defining resources 4
   - 2.2. Requirements classes defining parameters 5
   - 2.3. Requirements classes defining origins 8
   - 2.4. Requirements classes defining representations 8
   - 2.5. Summary of conformance URIs 10

3. NORMATIVE REFERENCES 13

4. TERMS AND DEFINITIONS 16

5. CONVENTIONS 21
   - 5.1. Identifiers 21
   - 5.2. Link relations 21
   - 5.3. Use of HTTPS 22

6. OVERVIEW 25
   - 6.1. Introduction 25
   - 6.2. Map interoperability 26
   - 6.3. How to approach an implementation of an OGC API Standard 27
   - 6.4. *OGC API – Maps* within the OGC API family 29
   - 6.5. Description of the domain 31
   - 6.6. Subsetting and scaling the map 32

6.7. Available formats and map response expectations 35

7. REQUIREMENTS CLASS "CORE" 38
7.1. Overview 38
7.2. Map resource 39
7.3. Declaration of conformance classes 43

8. REQUIREMENTS CLASS "MAP TILESETS" 46
8.1. Overview 46
8.2. Link from data resource 47
8.3. Map Tileset 47

9. REQUIREMENTS CLASS "BACKGROUND" 50
9.1. Overview 50
9.2. Map operation 50

10. REQUIREMENTS CLASS "COLLECTION SELECTION" 57
10.1. Overview 57
10.2. Operation 57
10.3. Service Metadata 59

11. REQUIREMENTS CLASS "SCALING" 62
11.1. Overview 62
11.2. Map Operation 62
11.3. Scale and aspect ratio considerations (informative) 67
11.4. Service Metadata 69

12. REQUIREMENTS CLASS "DISPLAY RESOLUTION" 73
12.1. Overview 73
12.2. Map Operation 73

13. REQUIREMENTS CLASS "SPATIAL SUBSETTING" 77
13.1. Overview 77
13.2. CRS for subsetting 78
13.3. Subsetting coordinates 81
13.4. Response 86
13.5. Error conditions 87

14. REQUIREMENTS CLASS "DATE AND TIME" 90
14.1. Overview 90
14.2. Describing the temporal extent 90
14.3. `datetime` query parameter request and response 91
14.4. `subset=time` query parameter request and response 93
14.5. Actual date & time response header 95
14.6. Closest date & time permission 96
14.7. Response 96
14.8. Error conditions 97

15. REQUIREMENTS CLASS “GENERAL SUBSETTING” 99
15.1. Overview 99
15.2. Describing the extent of the additional dimensions 99
15.3. `subset` query parameter 100
15.4. Response 101
15.5. Error conditions 102

16. REQUIREMENTS CLASS “COORDINATE REFERENCE SYSTEM” 104
16.1. Overview 104
16.2. Operation 104

17. REQUIREMENTS CLASS “ORIENTATION” 108
17.1. Overview 108
17.2. Operation 108

18. REQUIREMENTS CLASS “CUSTOM PROJECTION CRS” 112
18.1. Overview 112
18.2. Operation 112
18.3. Available projections 116

19. REQUIREMENTS CLASS “COLLECTION MAP” 122
19.1. Overview 122
19.2. General 122
19.3. Geospatial data resources 123
19.4. Geospatial Data Resource Map 125

20. REQUIREMENTS CLASS “DATASET MAP” 128
20.1. Overview 128
20.2. General 128
20.3. Dataset API landing page 128
20.4. Dataset maps 131

21. REQUIREMENTS CLASS “STYLED MAPS” 134
21.1. Overview 134
21.2. Styled resources 134
21.3. Styled Resource Map 135

22. REQUIREMENTS CLASSES FOR ENCODINGS 138
22.1. Overview 138
22.2. Requirements Class “PNG” 138
22.3. Requirements Class “JPEG” 140
22.4. Requirements Class “JPEG XL” 141
22.5. Requirements Class “TIFF” 142
22.6. Requirements Class “SVG” 144
22.7. Requirements Class “HTML” 145

23. REQUIREMENTS CLASS “API OPERATIONS” 148
23.1. Overview 148
23.2. Web API description 148

24. REQUIREMENTS CLASS “CORS” 154
24.1. Overview 154
24.2. CORS for JavaScript clients 154

ANNEX A CONFORMANCE CLASS ABSTRACT TEST SUITE 157
A.1. Conformance Class “Core” 157
A.2. Conformance Class “Map Tilesets” 159
A.3. Conformance Class “Background” 160
A.4. Conformance Class “Collection Selection” 164
A.5. Conformance Class “Scaling” 165
A.6. Conformance Class “Display Resolution” 168
A.7. Conformance Class “Spatial Subsetting” 170
A.8. Conformance Class “Date and Time” 175
A.9. Conformance Class “General Subsetting” 179
A.10. Conformance Class “Coordinate Reference System” 181
A.11. Conformance Class “Orientation” 182
A.12. Conformance Class “Custom Projection CRS” 183
A.13. Conformance Class “Collection Map” 188
A.14. Conformance Class “Dataset Map” 190
A.15. Conformance Class “Styled Map” 193
A.16. Conformance Class “PNG” 194
A.17. Conformance Class “JPEG” 195
A.18. Conformance Class “JPEG XL” 196
A.19. Conformance Class “TIFF” 197
A.20. Conformance Class “SVG” 198
A.21. Conformance Class “HTML” 199
A.22. Conformance Class “API Operations” 199
A.23. Conformance Class “CORS” 201

ANNEX B EXAMPLES 204
B.1. Default map 204
B.2. Collection description 205
B.3. Scaling and subsetting the map 207
B.4. Temporal subsetting 214
B.5. Styled maps 215
B.6. Additional dimensions 219
B.7. Coordinate Reference Systems 224
B.8. Calculations to infer appropriate dimensions 226
B.9. Calculations to infer appropriate bounding boxes 228

ANNEX C EVOLUTION FROM OGC WEB MAP SERVICE 232
C.1. From OGC Web Services to OGC Web APIs 232

C.2. Comparison between *WMS* and *OGC API – Maps* 233

ANNEX D BACKGROUND COLOR CODES 240

ANNEX E REVISION HISTORY 249

BIBLIOGRAPHY 252

I ABSTRACT

The *OGC API — Maps — Part 1: Core* Standard defines a Web API for requesting maps over the Web. A *map* is a portrayal of geographic information as a digital representation suitable for display on a rendering device (adapted from OGC 06-042/ISO 19128 OpenGIS® Web Map Server (WMS) Implementation Specification). Implementations of the *OGC API — Maps* Standard are designed for a client to easily:

- Request a visual representation of one or more geospatial data layers in different styles;
- Select by area, time and resolution of interest;
- Change parameters such as the background color and coordinate reference systems.

A server that implements *OGC API — Maps* provides information about what maps are offered. *OGC API — Maps* addresses use cases similar to those addressed by the OGC 06-042/ISO 19128 OpenGIS® Web Map Server (WMS) Implementation Specification Standard.

II KEYWORDS

The following are keywords to be used by search engines and document catalogues.

ogcdoc, OGC document, API, openapi, maps

PREFACE

This document defines the *OGC API — Maps — Part 1: Core* Standard. Suggested additions, changes and comments on this standard are welcome and encouraged. Such suggestions may be submitted as an issue on the *OGC API — Maps* GitHub repository.

IV SECURITY CONSIDERATIONS

Part 1 of the OGC API — Maps Standard only defines HTTP GET operations. As such the security considerations are limited to those applicable to a "read only" service. However, implementations of OGC API — Maps have to process resource paths and query parameters in a way that cannot be used by a client to inject malicious queries that makes available unforeseen data or even force the server to perform unwanted or dangerous actions. The following paragraphs enumerate some security considerations.

Due to the flexibility in generating maps, implementations of the OGC API — Maps Standard that are not optimized can easily encounter requests that take some time to resolve. If several of these requests are processed simultaneously, the server can become slow or unresponsive. Servers should take advantage of space partitioning structures that guarantee a deterministic maximum amount of data to process, such as the 2D Tile Matrix Set data structure (used in Cloud Optimized GeoTIFF, OGC API — Tiles, GeoPackage, etc.), which results in applying cartographic generalization for regions covering a larger area. Servers should also set and apply reasonable limits (e.g., maximum width, etc.) to prevent Denial of Service attacks.

Some OGC API — Maps deployments may assign different roles to different users that may result in accessing different collections or geographical areas that can be represented as maps. The access control can be described in the OpenAPI definition as discussed in OGC API — Common (https://docs.ogc.org/is/19-072/19-072.html#rc_oas30-security). Servers should take care that all resources in all representations and ways they can be requested (e.g., adding query parameters) are managed consistently.

Another security consideration is that maps generated by an endpoint of an OGC API — Maps implementation may be positionally incorrect and potentially guide users to wrong locations. One possible way this situation could happen is the manipulation by a malicious actor of a projection library installed on a server deployment.

Using HTTPS queries is preferred to using HTTP. The obvious reason is the map response returned by the endpoint of the OGC API — Maps implementation should be only accessible to the requesting user. Another reason is related to the public accessibility of the request itself. A request to the endpoint can reveal geographical extents that might be associated with private or sensitive information. For instance, users commonly visit "home" or "work" or "target places" (such as holiday destinations or churches). These requests can be used to predict personal or private activities.

V SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- Universitat Autònoma de Barcelona (CREAF)
- US Army Geospatial Center
- Ecere Corporation
- Esri

VI SUBMITTERS

All questions regarding this submission should be directed to the editor or the submitters:

NAME	AFFILIATION
Joan Masó *(editor)*	Universitat Autònoma de Barcelona (CREAF)
Jérôme Jacovella-St-Louis *(editor)*	Ecere Corporation
Jeff Harrison	US Army Geospatial Center
Satish Sankaran	Esri

VII LEGACY

The requirements and conformance classes defined in the OGC API — Maps — Part 1: Core Standard are grounded in the work done by many OGC Members in the different versions of the OGC Web Map Service (WMS) Standard. In particular, the work of Jean-François (Jeff) de La Beaujardière was instrumental in the consolidation of WMS. He participated in the original OGC Testbeds that resulted in WMS 1.0, back in April 2000. He became the editor of all subsequent versions, including WMS 1.1, WMS 1.1.1, WMS 1.3, and ISO 19128:2005.

VIII ACKNOWLEDGEMENTS

Key work for crafting this OGC Standard was undertaken in the Open-Earth-Monitor Cyberinfrastructure (OEMC) project (https://earthmonitor.org/). The OEMC Project received funding from the European Union's Horizon Europe research and innovation program under grant agreement number 101059548. Another project that provided key research is the All Data 4 Green Deal — An Integrated, FAIR Approach for the Common European Data Space (AD4GD) project (https://www.ad4gd.eu/). This project was co-funded by the European Union's Horizon Europe research and innovation program under grant agreement number 101061001 as well as the United Kingdom and Switzerland. Another project that provided key research on applying OGC API Standards in the European digital twin of the ocean is the Integrated DigitaL Framework for Comprehensive Maritime Data and Information Services (ILIAD) project (https://ocean-twin.eu/). The ILIAD project received funding from the European Union's Horizon Europe research and innovation program under grant agreement number 101037643.

Initial implementations and an Engineering Report constituting the first draft of this Standard were also produced thanks to funding from OGC Testbed 15. The editors also thank Natural Resources Canada for progress on this Standard achieved as part of GeoConnections 2020-2021 funded project number GNS20-21IFP02 (*Modular OGC API Workflows*).

1 SCOPE

1 SCOPE

The *OGC API — Maps — Part 1: Core* Standard (hereafter termed the *Maps API*) specifies operations to distribute maps and map tiles in a manner independent of the underlying data store. The Maps API can be described and documented using the OpenAPI specification and specifies resources for discovering and retrieving maps from a Web API.

Specifically, this *OGC API — Maps* Standard supports the following:

- Discovery operations that allow an implementation instance of the Maps API Standard to be interrogated to determine capabilities and to retrieve information about this distribution of maps. This information includes the API definition (if also implementing *OGC API — Common — Part 1: Core*), as well as metadata about the data layers provided and the Coordinate Reference System(s) supported by the Web API implementation instance.
- Retrieval operations that enable client applications to retrieve a map, using a default or pre-defined style, for an arbitrary geospatial resource, a dataset representing the full content available via the Maps API endpoint, or an individual collection of geospatial data representing part of the dataset.
- Parameters for specifying the background and transparency of the map.
- Parameters for specifying the scale of the map.
- A parameter for specifying the pixel size of the device or medium on which the map is intended to be displayed.
- Parameters for retrieving only a subset of the map.
- A parameter for specifying a particular orientation for the map.
- Parameters for specifying a Coordinate Reference System (CRS) for the map by reference or using a projection operation method (as defined by OGC 18-005r4 Abstract Specification Topic 2 Referencing by Coordinates), parameters for that method and a datum.

2 CONFORMANCE

2 CONFORMANCE

Conformance with this standard shall be checked using all the relevant tests specified in Annex A (normative) of this document if the respective conformance URLs listed in Conformance class URIs are present in the conformance response. The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in the OGC Compliance Testing Policies and Procedures and the OGC Compliance Testing web site.

The standardization targets of all conformance classes are “Web APIs”.

The *OGC API — Maps — Part 1: Core* Standard defines twenty-three **(23)** conformance classes, each corresponding to one requirements class.

The core conformance class is the only mandatory class. All other conformance classes are optional and depend on the *core*.

These classes act as building blocks that should be implemented in combination with other more fundamental classes that provide support for documenting and formally describing a Web API (e.g., OpenAPI), listing supported conformance classes and listing available geospatial resources. Possible alternatives for these fundamental classes are *OGC API — Common — Part 1: Core*, *OGC API — Features Part 1: Core* or any other non-OGC classes that provide equivalent or alternative functionality.

The *OGC API — Maps — Part 1: Core* Standard is intended to define a minimal set of functions for a useful API for fine-grained access to retrieve maps. Additional classes may be specified in future parts of the *OGC API — Maps* series to add more capabilities or as vendor-specific extensions.

2.1. Requirements classes defining resources

Requirements Class “Core” (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core)

The *Maps API* Core specifies requirements that any Web API implementation instance of the Maps API Standard must implement to claim conformance with the *OGC API — Maps — Part 1: Core* Standard. This requirements class defines an operation to retrieve a map but does not define any query parameters to customize the map.

An example request is below:

```
.../map
```

Requirements Class “Map Tilesets” (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tilesets)

The *map tilesets* requirements class defines a mechanism to list available map tilesets supported by the Web API instance and a mechanism for retrieving tiles of the map representation. The terms 'tile' and 'tile set' (alias tileset) are defined in the OGC Two Dimensional Tile Matrix Set Standard (OGC 17-083r4).

Example requests are below:

```
.../map/tiles
.../map/tiles/WorldCRS84Quad
.../map/tiles/WorldCRS84Quad/0/0/0
```

NOTE 1: Despite the use of `.../map/tiles` path templates in these examples which may become common in implementations of *OGC API — Maps*, these exact paths are only examples and are not required by *OGC API — Tiles* or by this *Maps API* Standard. Other paths are possible provided they are correctly described in the API description of the Web API and the links between resources.

NOTE 2: The "Custom Projection CRS" requirements class is not included in this sub-section because its primary focus is not defining resources, but the Custom Projection requirements class also specifies a /projectionsAndDatums resource listing values available for the parameters it defines.

2.2. Requirements classes defining parameters

These requirements classes of the *Maps API* define parameters that can be used together with any map resource.

NOTE 1: In the effort to make OGC API Standards more consistent, some parts of these requirements classes are expected to be imported by a future part of OGC API — Common or be registered in the building blocks registry. A future revision of OGC API — Maps could reference them and ensure consistency with the relevant common requirements.

Requirements Class "Background" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/background)

The *background* requirements class defines how to request two particular background colors (`bgcolor` and `void-color`) and to specify whether areas of the map with no data or where the CRS / projection is invalid are transparent (using `transparent` and `void-transparent`).

Example requests are below:

```
.../map?bgcolor=0x001122&transparent=false&void-transparent=true
```

Requirements Class "Collection Selection" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collections-selection)

The *collection selection* requirements class defines how to list specific geospatial data collection resources from which to retrieve maps.

An example request is below:

```
.../map?collections=buildings,roads,...
```

Requirements Class "Scaling" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/scaling) The *scaling* requirements class defines how to retrieve a resampled map. This is done by specifying a `width` and/or `height` parameter (in pixels), or a `scale-denominator` parameter.

Example requests are below:

```
.../map?width=1024&height=512
.../map?scale-denominator=20000
```

NOTE 2: When implementing both this *scaling* requirements class and the *spatial subsetting* requirements class, the `width` and `height` can only be used for specifying the scale when a `bbox` or `subset` is also used for spatial subsetting, as otherwise they take on a subsetting role. When neither `bbox` nor `subset` is used for spatial subsetting, the `scale-denominator` parameter must be used to specify the scale.

Requirements Class "Display Resolution" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/display-resolution) The *display resolution* requirements class defines how to specify the resolution of the display (the size of a pixel in millimeters) using the `mm-per-pixel` parameter at which the map will be visualized so as to properly interpret scale denominators and apply scale-dependent symbology rules.

An example request is below:

```
.../map?mm-per-pixel=0.14
```

Requirements Class "Spatial Subsetting" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting) The *spatial subsetting* requirements class defines how to retrieve a spatial subset of a map using either a `bbox` or `subset` parameter.

Example subsetting requests using bounding box are below:

```
.../map?bbox=120,30,180,90
.../map?bbox-crs=[EPSG:3857]&bbox=500000,430000,1000000,1000000
```

Example subsetting requests using subset are below:

```
.../map?subset=Lat(30:90),Lon(120:180)
.../map?subset-crs=[EPSG:3857]&subset=X(500000:1000000),Y(430000:1000000)
```

This class also specifies how to retrieve a subset of a map using a `center` parameter, together with `width` and `height` parameters.

NOTE 3: When implementing both this *spatial subsetting* requirements class and the *scaling* requirements class, the `width` and `height` can only be used for specifying the scale when a `bbox` or `subset` is also used for spatial subsetting, as otherwise they take on a subsetting role. When neither `bbox` nor `subset` is used for spatial subsetting, the `scale-denominator` parameter must be used to specify the scale.

Example subsetting requests using center point and dimensions:

```
.../map?center=-120,60&width=1024&height=512
.../map?center-crs=[EPSG:3857]&center=750000,70000&width=1024&height=512
```

Requirements Class "Date and Time" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime) The *temporal subsetting* requirements class specifies how to request a temporal subset of the data using the `datetime` parameter, or the `subset` parameter for the `time` dimension.

Example requests are below:

```
.../map?datetime=2018-02-12T23:20:52Z
.../map?subset=time("2018-02-12T23:20:52Z")
```

Requirements Class "General Subsetting" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/general-subsetting) The *general subsetting* requirements class specifies how to request a subset of dimensions of the data besides the spatial and temporal dimensions using the `subset` parameter. This parameter also implies adopting a consistent way to describe all dimensions of the data in the collection's extent description.

An example request is below:

```
.../map?subset=atm_pressure_hpa(500)
```

Requirements Class "Coordinate Reference System" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/crs)

The *Coordinate Reference System* requirements class defines how to specify the output Coordinate Reference System (CRS) of the map by referencing a Uniform Resource Identifier (URI) or compact URI (CURIE) of a CRS definition.

An example request is below:

```
.../map?crs=[EPSG:3031]
```

NOTE 4: Every time that a URI to a CRS is required or recommended a CURIE equivalent is also valid. A CURIE {authority}[-{objectType}]:{id} would map to the following OGC URI: https://www.opengis.net/def/{objectType}/{authority}/0/{id}. If -{objectType} is missing, the default object type is `crs`.

Requirements Class "Orientation" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/orientation) The *orientation* requirements class defines how to specify an angle (expressed in degrees) for re-orienting how the map is displayed (`orientation`).

An example orientation request is below:

```
.../map?orientation=40
```

Requirements Class "Custom Projection CRS" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection) The *custom projection CRS* requirements class defines how to specify a custom CRS through a projection, including the coordinate operation method (`crs-proj-method`) and associated parameters (`crs-proj-params`), as well as a datum (`crs-datum`). This class also defines a `crs-proj-center` parameter for facilitating the selection of the most likely parameters to center the projection on an area of interest.

An example of an orthographic projection request is below:

```
.../map?
   crs-proj-method=[epsg-method:9840]&
   crs-proj-center=Lat(40),Lon(-120)
```

An example of a Lambert Conic Conformal projection with two standard parallels request is below:

```
.../map?
   crs-proj-method=[epsg-method:9802]&
   crs-proj-params=[epsg-parameter:8823](40),[epsg-parameter:8824](90)&
   crs-datum=[epsg-datum:6230]
```

NOTE 5: This "Custom Projection CRS" requirements class also defines a `/projectionsAndDatums` resource listing values available for the parameters it defines.

2.3. Requirements classes defining origins

Requirements Class "Collection Map" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collection-map)

The *collection map* requirements class specifies how to retrieve maps from a specific geospatial data resource.

An example request is below:

`/collections/buildings/map`

Requirements Class "Dataset Map" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/dataset-map)

The *dataset map* requirements class specifies how to retrieve maps for a whole dataset potentially made up of multiple geospatial data resources. Any Web API implementing this requirements class must support **dataset** maps following this *OGC API — Maps — Part 1: Core* Standard. Dataset maps may combine content from multiple geospatial resources, regardless of whether those are available separately (as maps or otherwise).

An example request is below:

`/map`

Requirements Class "Styled Maps" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/styled-map)

The *styled map* requirements class specifies how to retrieve maps for a styled resource.

An example request is below:

`.../styles/night/map`

2.4. Requirements classes defining representations

Requirements Classes for Encodings

The *Maps API* Standard does not mandate a specific encoding or format for representing maps. Requirements classes are provided for the following common map formats.

PNG (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/png)

Media type: `image/png`

JPEG (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/jpeg)

Media type: `image/jpeg`

JPEG XL (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/jpegxl)

Media type: `image/jxl`

TIFF (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tiff)

Media type: `image/tiff`

SVG (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/svg)

Media type: `image/svg+xml`

HTML (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/html)

Media type: `text/html`

The Standard remains flexible and extensible for using other formats that users and providers might need through HTTP content negotiation.

That said, this Standard includes recommendations to support, where practical, HTML.

Requirements Class "API Operations" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/api-operations)

The *API Operations* requirements class specifies requirements to fully describe the *Maps API* operations and use specific operation identifier suffixes when providing an API definition. This requirements class is intended to be used in conjunction with other conformance classes for a specific API definition language and/or version, such as the OpenAPI 3.0 requirements class defined in *OGC API — Common — Part 1: Core*, or another eventual requirements class for OpenAPI 3.1.

Requirements Class "CORS" (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/cors)

The *CORS* requirements class specifies a requirement to implement CORS to support JavaScript clients (e.g. Web Browser applications) from a domain different from the OGC API — Maps endpoint.

All requirements classes and conformance classes described in this Standard are owned by the Standard(s) identified.

2.5. Summary of conformance URIs

Table 1 — Conformance class URIs

CORRESPONDING REQUIREMENTS CLASS	CONFORMANCE CLASS URI
Core	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core
Map Tilesets	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/tilesets
Background	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/background
Collection Selection	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/collections-selection
Scaling	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/scaling
Display Resolution	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/display-resolution
Spatial Subsetting	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/spatial-subsetting
Date and Time	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/datetime
General Subsetting	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/general-subsetting
Coordinate Reference System	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/crs
Orientation	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/orientation
Custom Projection CRS	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/projection
Collection Map	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/collection-map
Dataset Map	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/dataset-map
Styled Maps	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/styled-map
PNG	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/png

CORRESPONDING REQUIREMENTS CLASS	CONFORMANCE CLASS URI
JPEG	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/jpeg
JPEG XL	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/jpegxl
TIFF	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/tiff
SVG	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/svg
HTML	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/html
API Operations	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/api-operations
CORS	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/cors

3 NORMATIVE REFERENCES

3 NORMATIVE REFERENCES

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Klyne G, Newman C, Internet Engineering Task Force: IETF RFC 3339, *Date and Time on the Internet: Timestamps*. RFC Publisher (2002). https://www.rfc-editor.org/info/rfc3339.

Charles Heazel, Open Geospatial Consortium: OGC 19-072, *OGC API — Common — Part 1: Core*. Open Geospatial Consortium (2023). http://www.opengis.net/doc/is/ogcapi-common-1/1.0.0.

Clemens Portele, Panagiotis (Peter) A. Vretanos, Charles Heazel, Open Geospatial Consortium: OGC 17-069r4, *OGC API — Features — Part 1: Core corrigendum*. Open Geospatial Consortium (2022). http://www.opengis.net/doc/IS/ogcapi-features-1/1.0.1.

Joan Masó, Jérôme Jacovella-St-Louis, Open Geospatial Consortium: OGC 20-057, *OGC API — Tiles — Part 1: Core*. Open Geospatial Consortium (2022). http://www.opengis.net/doc/IS/ogcapi-tiles-1/1.0.0.

Joan Masó, Jérôme Jacovella-St-Louis, Open Geospatial Consortium: OGC 17-083r4, *OGC Two Dimensional Tile Matrix Set and Tile Set Metadata*. Open Geospatial Consortium (2022). http://www.opengis.net/doc/IS/tms/2.0.0.

Nottingham M: IETF RFC 8288, *Web Linking*. RFC Publisher (2017). https://www.rfc-editor.org/info/rfc8288.

International Organization for Standardization (committee): ISO/IEC 18181-1:2024, *Information technology — JPEG XL image coding system — Part 1: Core coding system*. International Organization for Standardization, International Electrotechnical Commission, Geneva (2024). https://www.iso.org/standard/85066.html.

International Organization for Standardization (committee): ISO/IEC 18181-2:2024, *Information technology — JPEG XL image coding system — Part 2: File format*. International Organization for Standardization, International Electrotechnical Commission, Geneva (2024). https://www.iso.org/standard/85253.html.

International Organization for Standardization (committee): ISO/IEC 10918-1, *Information technology — Digital compression and coding of continuous-tone still images: Requirements and guidelines*. International Organization for Standardization, International Electrotechnical Commission, Geneva https://www.iso.org/standard/18902.html.

International Organization for Standardization (committee): ISO/IEC 15948, *Information technology — Computer graphics and image processing — Portable Network Graphics (PNG): Functional specification*. International Organization for Standardization,

International Electrotechnical Commission, Geneva https://www.iso.org/standard/29581.html.

Open API Initiative: OpenAPI Specification 3.0.3, https://github.com/OAI/OpenAPI-Specification/blob/master/versions/3.0.3.md

Adobe development association: TIFF Revision 6.0 Final — June 3, (1992) https://www.itu.int/itudoc/itu-t/com16/tiff-fx/docs/tiff6.pdf

W3C: Scalable Vector Graphics (SVG) v2. W3C Candidate Recommendation — October, (2018) https://www.w3.org/TR/SVG2/

WHATWG: HTML, Living Standard [online, viewed 2020-03-16]. Available at https://html.spec.whatwg.org/

NOTE: *OGC API — Features — Part 1: Core* has been included as a temporary normative reference. When *OGC API — Common — Part 2: Geospatial data* is released, the latter should replace *OGC API — Features — Part 1: Core* as a normative reference in a future version of OGC API — Maps — Part 1: Core.

4 TERMS AND DEFINITIONS

4 TERMS AND DEFINITIONS

This document uses the terms defined in OGC Policy Directive 49, which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “shall” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this document and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the ‘ModSpec’. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

For the purposes of this document, the following additional terms and definitions apply.

4.1. coordinate reference system

coordinate system that is related to the real world by a datum (source: ISO 19111)

4.2. coordinate system

set of mathematical rules for specifying how coordinates are to be assigned to points (source: ISO 19111)

4.3. data cube (or datacube)

a multi-dimensional (“n-D”) array of values. Sometimes, the term *data cube* is applied in contexts where these arrays are massively larger than the hosting computer’s main memory; examples include multi-terabyte/petabyte data warehouses and time series of image data.

4.4. geographic information

information concerning phenomena implicitly or explicitly associated with a location relative to the Earth (source: ISO 19101)

4.5. height

1. vertical elevation of an object in the real world above a reference point (height above the reference ellipsoid for geographic CRS such as [OGC:CRS84h] and [EPSG:4979]).
2. size, in pixels, of a viewport along its vertical axis

Note 1 to entry: Both the *spatial subsetting* and *scaling* requirements classes make use of the `height` query parameter. In both cases, the parameter refers to the height of the image in pixels, but the effect of specifying a fixed value for the parameter is different in each case (selecting the portion of the map to return in the image vs. resampling the map to the requested image size).

4.6. map

portrayal of geographic information as a digital representation suitable for display on a rendering device (adapted from OGC 06-042)

4.7. map background

areas of the map having no data to represent. They are commonly left transparent or filled with a background color

4.8. portrayal

presentation of information to humans (source: ISO 19117)

4.9. rendering device

medium or apparatus on which the map being retrieved will be displayed, whether temporarily (e.g., computer screen) or permanently (e.g., paper).

4.10. scale

ratio between the size of a map shown in the rendering device (commonly expressed in mm or inches) and the equivalent geographic bounding box of a map in the reality (commonly expressed in meters or miles). Commonly, the ratio is expressed as a fraction, where 1 unit in the rendering device is equivalent to `n` units in the reality (in the same units of measure). In this case, `n` is called a scale denominator.

4.11. viewport

portion of a rendering device where information is being visualized

Note 1 to entry: For the purpose of this document, the viewport refers to the portion of the map resource response, such as a digital image, where the actual geospatial content of the map is rendered. It excludes any outside margin portions of the image where layout elements such as titles or a legend may be included, as could be defined by an extension. For a two-dimensional viewport, its dimensions are named horizontal and vertical, corresponding to the width and height of the viewport, respectively.

4.12. width

size, in pixels, of a viewport along its horizontal axis

Note 1 to entry: Both the *spatial subsetting* and *scaling* requirements classes make use of the `width` query parameter. In both cases, the parameter refers to the width of the viewport in pixels, but the effect of specifying a fixed value for the parameter is different in each case (selecting the portion of the map to return in the image vs. resampling the map to the requested image size).

4.13. **Web API**

An Application Programming Interface (API) using an architectural style that is founded on the technologies of the Web (source: OGC 17-069r3)

Note 1 to entry: See Best Practice 24: Use Web Standards as the foundation of APIs (W3C Data on the Web Best Practices) for more detail.

5 CONVENTIONS

5 CONVENTIONS

This section provides details of conventions used in this document.

5.1. Identifiers

The normative provisions in this standard are denoted by the URI `https://www.opengis.net/spec/ogcapi-maps-1/1.0`.

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

5.2. Link relations

To express relationships between resources, RFC 8288 (Web Linking) is used.

Note that the CURIE based on the pattern `[ogc-rel:<relation>]` is also considered equivalent. For example, `[ogc-rel:map]` corresponds to https://www.opengis.net/def/rel/ogc/1.0/map.

The following IANA link relation types are used in this document:

- **alternate**: Refers to a substitute for this context.
- **self**: Conveys an identifier for the link's context.
- **item**: The target IRI points to a resource that is a member of the collection represented by the context IRI.
- **preview**: Refers to a resource that provides a preview of the link's context. In the context of this specification, it used to link to a low-resolution preview of the map.
- **service-desc**: Identifies service description for the context that is primarily intended for consumption by machines (Web API definitions are considered service descriptions).
- **service-doc**: Identifies service documentation for the context that is primarily intended for human consumption.
- **stylesheet**: Refers to a stylesheet. In the context of this specification, it refers to a cartographic stylesheet, as provided by *OGC API — Styles*.

In addition, the following link relation type is used for which no applicable registered link relation type could be identified:

- **https://www.opengis.net/def/rel/ogc/1.0/map**: Refers to a resource that is map representation of another geospatial resource (e.g., a collection).
- **https://www.opengis.net/def/rel/ogc/1.0/legend**: Refers to a legend for the map.

Used in combination with *OGC API — Tiles — Part 1: Core*, other link relation types will be used, including:

- **https://www.opengis.net/def/rel/ogc/1.0/tilesets-map**: The target IRI points to a resource that describes how to provide tile sets of the context resource in map format.

Used in combination with *OGC API — Styles*, other link relation types will be used, including:

- **https://www.opengis.net/def/rel/ogc/1.0/styles**: The target IRI points to a resource that describes how to provide styles of the context resource that internally can contain links to maps.

Used in combination with *OGC API — Features — Part 1: Core* or *OGC API — Common — Part 1: Core*, other link relation types will be used, including:

- **https://www.opengis.net/def/rel/ogc/1.0/conformance**: Refers to a resource that identifies the specifications that the link's context conforms to.

Used in combination with *OGC API — Features — Part 1: Core* or *OGC API — Common — Part 2: Geospatial data*, other link relation types will be used, including:

- **https://www.opengis.net/def/rel/ogc/1.0/data**: Refers to the list of collections available for a dataset.

Each resource representation includes an array of links. Implementations are free to add additional links for all resources provided by the Web API implementing Maps API requirement(s).

5.3. Use of HTTPS

For simplicity, the Maps API only refers to the HTTP protocol. This is not meant to exclude the use of HTTPS and simply is a shorthand notation for "HTTP or HTTPS." Following the recent push towards enhancing web security, public facing web servers are expected to use HTTPS rather than HTTP. In the context of Maps API Standard, use of HTTPS provides a layer of security to prevent leaking of information related to users requesting map data for a particular

geographical area, which may be private or sensitive in nature (for example, their own location, or the location of endangered species).

6 OVERVIEW

6 OVERVIEW

6.1. Introduction

The *OGC API — Maps* Standard defines building blocks which can be used in a Web API implementation to retrieve geospatial data as maps that are visual portrayals of the data created by applying a style to the data (see Requirements Class "Core"). Maps can present the whole area of the geospatial data or a subset of the content, such as by filtering the data by extent (see Requirements Class "Spatial subsetting"). Maps can be retrieved by arbitrary extent or as tiles (see Requirements Class "Map Tilesets"). The **"Core"** conformance class is the **only mandatory one**. All other conformance classes are optional and depend on the "Core".

An annex with examples of map requests and responses is included as a way to 'learn by example' how this standard can be applied. See Examples.

An example OpenAPI definition for this Standard is available. Other examples are available on the OGC API website. Details on how to adjust and bundle the modular and reusable components used in this example OpenAPI definition can be found in Requirements Class "API Operations" .

Services and clients are encouraged to support as many of the Coordinate Reference Systems (CRSs) as possible for all geospatial data resources to maximize interoperability. However, this Standard does not require support for any specific CRS.

The *OGC API — Maps* Standard does not specify any requirement for the type of *geospatial data resource* that could be delivered as maps. If the geospatial data resources can be organized into maps, they can be supported regardless of whether they are feature data, coverages, a resource that does not represent data per se (e.g., an annotation) and so forth.

NOTE: Geospatial data resources (e.g., collections) replace the concept of layer in WMS and WMTS. The main difference is that layers in WMS and WMTS were not defined by other OGC API Standards and did not support other functionalities.

These geospatial data resources can advertise one or more map portrayals in several resources, such as the dataset (see Requirements Class "Dataset Map"), a collection (see Requirements Class "Collection Map"), a dataset with a style, or a collection with a style (see Requirements Class "Styled Maps"). Implementations of other OGC API Standards can provide other origins that may be represented as maps. Accessing the *geospatial data resource* content (other than as maps) or its descriptions is possible but out of the scope of this Standard. If a description of the *geospatial data resource* is specified by another standard, and this description has a mechanism to add links to other resources, this Standard indicates the need to add a link to a map of this resource.

The *OGC API — Maps* Standard does not specify how to get a Web API definition, the conformance class list or the collections lists. However, the Maps API Standard assumes that the first two are defined by some OGC API Standard (e.g., *OGC API — Common — Part 1: Core*) and the latter by an OGC API for collections (e.g., *OGC API — Common — Part 2: Geospatial data*). A similar definition is provided directly by *OGC API — Features — Part 1: Core*.

This document is the first part of a series of *OGC API — Maps* "parts" that follows the core and extensions model. Future parts will specify other extensions, such as how to specify cartographic layout elements to be included on the map, how to specify a filter for which elements should be included on the map, and possibly how to query information for a point in a map.

6.2. Map interoperability

One of the most significant use cases for the definition of the *Maps API* is the ability to generate maps on the web from layers, provided by multiple servers, that can be overlaid into a single view. The Maps API Standard provides a clear and common way to request a map image that covers a bounding box of interest with a defined number of pixel rows and columns. When the *Maps API* is implemented by different organizations that offer different collections of information, the same bounding box, in the same CRS, and the same number of rows and columns can be requested from each organization's geospatial repository. The result is a set of images that perfectly overlap showing different variables of the same area depending on the collection of geospatial data requested. For example, a population map can be requested from one organization (e.g., SEDAC Population Density) and a weather forecast from another (e.g., NDFD Surface Wind Velocity) in the exact same way allowing to present both together in one overlaid view or side-by-side.

Table 2 — SEDAC GPWv4 Population Density, 2015 and Forecast NDFD, Surface (10m AGL) Wind Velocity (Barb, Knots)

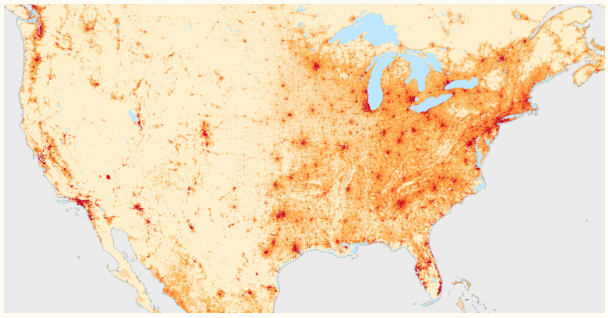
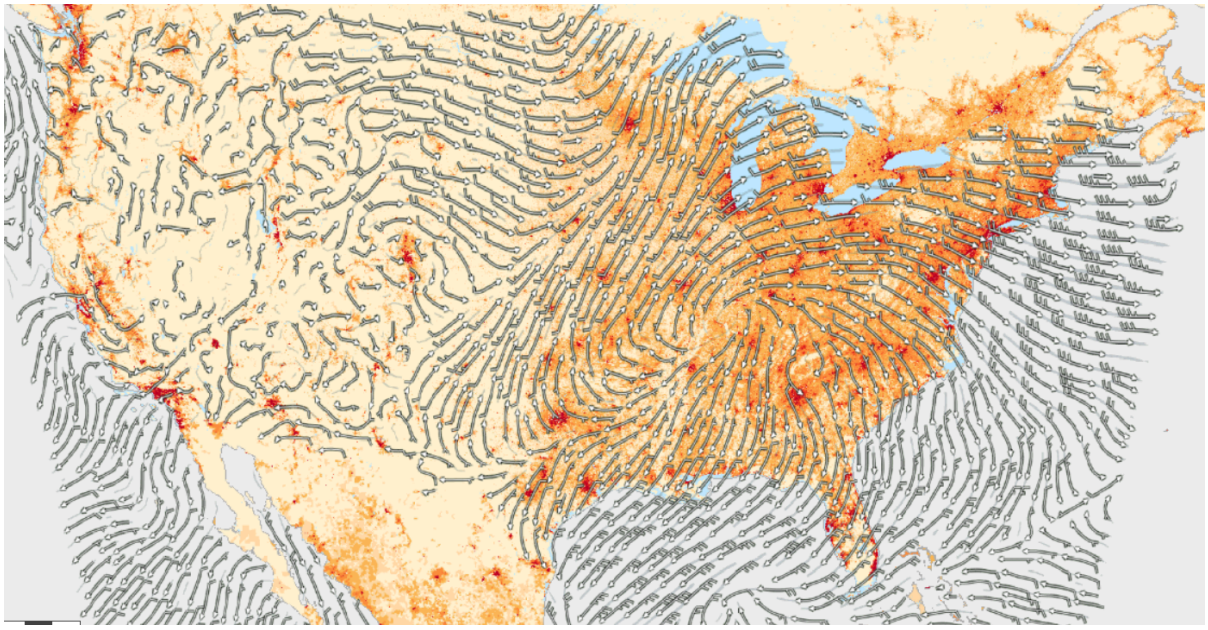


Figure 1



Figure 2



NOTE: The Requirements Class “Collection Selection” provides a way for clients to instruct a service implementing OGC API - Maps to include multiple sub-resources overlaid in the map response.

Figure 3 — Forecast NDFD wind speed on top of SEDAC Population density

6.3. How to approach an implementation of an OGC API Standard

There are at least two ways for using an implementation of one or more OGC API Standards:

- Read the landing page, look for links, follow them and discover new links until the desired resource is found
- Read a Web API definition document that specifies a list of paths and path templates to resources.

For the first approach, many resources in a deployed Web API include links with *rel* properties to document the reason and purpose for this relation. The following figure illustrates the resources as ellipses and the links as arrows with the link *rel* as a label.

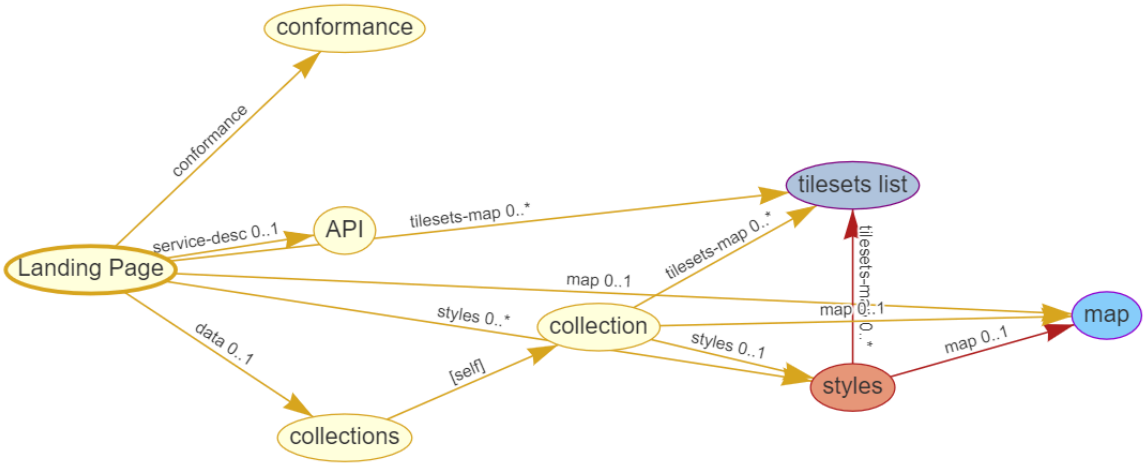


Figure 4 — Resources and relations to them via links

For the second approach, implementations should consider the Requirements Class "API Operations" which defines the use of *operationID* suffixes, providing a mechanism to associate API paths with the requirements class that they implement.

There is a third way to approach an OGC API implementation instance. This approach relies on assuming a set of predefined paths and path templates. These predefined paths are used in many examples in this document and are presented together in Overview of resources and common direct links that can be used to define an *OGC API — Maps* implementation. It is expected that many implementations of the Maps API Standard will provide a Web API definition document (e.g., OpenAPI) using this set of predefined paths and path templates to get necessary resources directly. All this could mislead the reader into getting the false impression that the predefined paths are enforced. Therefore, building a client that is assuming a predefined set of paths is risky. However, it is expected that many API implementations will follow the predefined set of paths. The clients using this assumption could be successful on many occasions. Again, be aware that these paths are not required by the Maps API Standard.

Table 3 — Overview of resources and common direct links that can be used to define an *OGC API — Maps* implementation

RESOURCE NAME	COMMON PATH
Landing page[4]	`{datasetRoot}/`
Conformance declaration[4]	`{datasetRoot}/conformance`
Dataset Maps	
Dataset maps in the default style [1]	`{datasetRoot}/map`
Dataset maps[1,2]	`{datasetRoot}/styles/{styleId}/map`

RESOURCE NAME	COMMON PATH
Dataset map tiles[1,3]	`{datasetRoot}/map/tiles/{tileMatrixSetId}/...`
Geospatial data collections[5]	
Collections[5]	`{datasetRoot}/collections`
Collection[5]	`{datasetRoot}/collections/{collectionId}`
Collection maps in the default style	`{datasetRoot}/collections/{collectionId}/map`
Collection maps[2]	`{datasetRoot}/collections/{collectionId}/styles/{styleId}/map`
Collection map tiles[3]	`{datasetRoot}/collections/{collectionId}/map/tiles/{tileMatrixSetId}/...`

[1] From the whole dataset or one or more geospatial resources or collections [2] Specified in the *OGC API — Styles* Standard [3] Specified in the *OGC API — Tiles Part 1: Core* Standard [4] Specified in the *OGC API — Common Part 1: Core* Standard [5] Specified in the *OGC API — Common Part 2: Geospatial Data* Standard

NOTE: Even though full path and full path templates in the previous table may be used in many implementations of the *OGC API — Maps* Standard, these exact paths are ONLY examples and are NOT required by this Standard. Other paths are possible if correctly described in by the Web API definition document and/or the links between resources.

6.4. *OGC API — Maps* within the OGC API family

6.4.1. What is a map?

A map is a portrayal of geographic information as a digital representation suitable for display on a rendering device (definition adapted from OGC 06-042). It can also be thought of as a portrayal of data resulting from applying a style, usually in the form of a 2D image format such as PNG or JPEG, or in presentation formats such as SVG. The way the styling rules for a style are applied to the data to create the portrayal is out of scope of this Standard (see the *OGC API — Styles* candidate Standard, as well as other OGC Standards which address this topic).

6.4.2. Implementing *OGC API — Maps* within a Web API

A map can be delivered as a single static resource (only implementing the Maps API "Core" requirements class), or as a dynamic service able to return different maps for arbitrary

extents (implementing “Subsetting” requirements class) and/or at arbitrary scales (implementing “Scaling” requirements class). In addition, a map can also be delivered as tiles by combining *OGC API — Maps* with some *OGC API — Tiles* requirements classes. This approach is defined by the “Map Tilesets” requirements class of this Standard, which also corresponds to *map tilesets* described in *OGC API — Tiles*, with a *map* being a specific type of data resource for which tiles are provided.

The Maps API Standard defines building blocks that can be combined with other APIs generating or providing access to information having a geospatial component, including the other standards in the OGC API family such as *OGC API — Tiles* and *OGC API — Processes*. The Maps API Standard can be referenced by other standards providing resources that can be offered as maps. For example:

- *OGC API — Tiles* specifies the link relation types to access map tilesets from a dataset or collection. *OGC API — Tiles* can also be used to serve the source data (e.g., vector features or coverage data)
- *OGC API — Styles*, a candidate Standard, defines paths to list available styles from which maps can also be accessed.
- *OGC API — Processes — Part 3: Workflows and Chaining*, a candidate Standard, provides a mechanism to trigger localized processing workflows as a result of retrieving maps (for a specific area and resolution of interest).

The origin resources to which the map resource can be attached, such as the dataset landing page (defined by *OGC API — Common — Part 1*) and collection (defined by *OGC API — Common — Part 2*), may also provide access to the data used to generate the maps, alongside the Maps API capability. For example:

- *OGC API — Tiles* also specifies link relation types to access tilesets of vector and coverage data from a dataset or collection.
- *OGC API — Features* defines an API to access collections of vector features at `/collections/{collectionId}/items` and individual features at `/collections/{collectionId}/items/{itemId}`, including both geometry and properties.
- *OGC API — Coverages*, a candidate Standard, defines an API to efficiently access information organized as multi-resolution and multi-dimensional datacubes at `/collections/{collectionId}/coverage`. Several common parameters in the Coverages API are shared with this Maps API. For some request formulations, it is possible to simply toggle between `/map` and `/coverage` (while keeping the same parameters) to alternate between retrieving the raw data values (a.k.a. a coverage) or a server-side visualization (a.k.a. a map).
- *OGC API — EDR* defines an API to retrieve spatiotemporal information using multiple query patterns such as cubes, trajectory, and corridors.

The possibilities are endless. For example, a generic open data API giving access to tables, some of them with columns storing latitude and longitude, could be enhanced with OGC API endpoints to provide mapping capabilities.

6.4.3. Dynamic and scalable map viewers

In the OGC, the concept of a map as an image was formulated in 1998 as part of the OGC Web Map Service standards work. At that time, the web was very young. Most HTML pages were static, and JavaScript was a rudimentary programming language capable of controlling user entries in an HTML form and not much more. In that environment, having a service capable of creating a PNG that could be embedded as an HTML page by using an IMG tag provided the first approach to static maps on the web. Replacing the source (SRC) of the IMG tag programmatically with JavaScript, as a reaction of some user actions, provided the first approach to dynamic maps.

The WMS *GetFeatureInfo* request added a limited capability for queryable maps (that is, maps that could be interactively queried). However, users are now used to moving around the map by frequently doing zoom and pan operations. If the server does not provide a very fast response, the user experience is not smooth and the map display application is perceived as not responsive enough. One possible approach to solve this problem is dividing the viewport into tiles and requesting them separately. Since tiles follow a tile matrix pattern, they can be pre-rendered on the server-side or cached in intermediate services on the Internet. For implementing fast dynamic maps, the *OGC API — Maps* requirements should be combined with *OGC API — Tiles* requirements.

6.4.4. Client-side maps versus server-side maps

The *OGC API — Maps* Standard deals with maps that are generated by the server. The client can present them with no modification. Currently, even the smallest rendering device supports hardware rendering — the transformation from geometries to pixels can be done by the GPU. Transmitting geometries from the server commonly requires less bandwidth than transmitting the rendered map from the server and offers more flexibility on the client-side to personalize the portrayal style. Because of this, it is expected that *OGC API — Maps* use cases will focus more on static maps, infrequently changing requests for dynamic maps, as well as print cartography, whereas requesting raw data values using *OGC API — Tiles* (e.g. from tiled vector data and coverage tiles) is better suited for interactive clients presenting dynamic maps.

6.5. Description of the domain

The Maps API Standard defines how to describe the domain of the maps, including spatiotemporal axes as well as additional dimensions.

With the *Collection Map* requirements class, the collection description inherited from *OGC API — Common — Part 2* contains an `extent` property that can describe both the spatial and temporal domain of the data. In addition, the *Unified Additional Dimensions* common building block, specified in the *General Subsetting* requirements class and used in the example OpenAPI definition, requires that additional dimensions be described in a similar way to the temporal

dimension. This allows providing an overall lower and upper bound (the first `interval` element), as well as optional sparse inner intervals where data is found along each dimension (additional `interval` elements). A `grid` property also supports the description of regular and irregular grids. The `resolution` (the distance between any two neighboring cells, an absolute value) and the number of cells (`cellsCount`) can be specified for each regular dimension. A list of `coordinates` where data is found can be specified for irregular dimensions. In addition, the minimum and maximum cell size (`minCellSize` and `maxCellSize`) and equivalent scale denominators (`minScaleDenominator` and `maxScaleDenominator`) can be specified in the collection resource.

The *Dataset Map* requirements class specifies the addition of an `extent` property to the landing page (root resource of the API) of *OGC API — Common — Part 1* based on the same schema as for the collection.

6.6. Subsetting and scaling the map

The core requirements class of the Maps API Standard provides a way to retrieve the map that is modified by other classes allowing for subsetting the domain, specifying a particular size for the output map image, and changing the default assumption about the physical size of a pixel on the rendering device. The combination of these parameters also defines the scale of the map, which affects how scale-dependent symbology rules should be applied. These classes (Scaling, Display resolution and Subsetting) define the following parameters interacting with each other (in a not so trivial manner):

Table 4 — Parameters for scaling and subsetting

PARAMETER	DEFINITION
`width`	Width of the viewport in pixel units
`height`	Height of the viewport in pixel units
`scale-denominator`	Number of units in the physical world that is equivalent to 1 unit on the rendering device
`mm-per-pixel`	Size of one pixel on the rendering device expressed in millimeters. The default value is 0.28 mm
`bbox` (`bbox-crs`) (and the equivalent `subset` and `subset-crs`)	Bounding box of the requested map in CRS coordinates. It defines the geographic size.
`center` (`center-crs`)	Center of the requested map in CRS coordinates. `center` and `bbox` are mutually exclusive.

All these parameters are optional. The server needs to know the geographic extent covered by the map in physical world units, and the size of the map as rendered on the viewport (in both pixel units and physical units). Some combinations completely define both sizes. Some

combinations of parameters generate impossible situations and will result in an error. Other combinations require that the server decides a default value for some parameters not provided to be able to resolve the requested sizes. The Maps API Standard only specifies the default value for `mm-per-pixel` leaving to the server freedom to decide about the other parameters. The following tables present an overview of the different combinations possible depending on whether the *Scaling*, *Subsetting* or both *Scaling* and *Subsetting* requirements classes are supported by the implementation, to clarify the relationship between these parameters and provide centralized guidance for implementers.

NOTE 1: The parameter `mm-per-pixel` is not included in these tables but is used for computing one of the `scale-denominator`, dimensions (`width` and `height`), or spatial extent (`bbox`), based on the default or provided values for the others. If not provided in the request, the default is 0.28 mm per pixel.

NOTE 2: Every time that `bbox` appears as a provided parameter in these tables, it represents either `bbox` or the equivalent `subset`.

NOTE 3: Wherever `width` and `height` appear together in these tables, it also represents either of them being specified without the other. Depending on the parameter combination, the server either computes the appropriate value of the omitted dimension so as to reflect the correct scale (when a bounding box is also provided — see relevant requirements and guidance), or uses a default value which is either fixed or tied by a default aspect ratio to the one dimension specified (see recommendation).

Table 5 — Always valid requests (no scaling or subsetting parameter)

PARAMETERS PROVIDED IN THE REQUEST	SERVER OR RESOURCE DEFAULTS USED	COMPUTED
none	`bbox`, `scale-denominator`, `center`, `width` and `height`	*None*

Table 6 — Always invalid parameter combinations

PARAMETERS PROVIDED IN THE REQUEST	EXPLANATION
`bbox`, `scale-denominator` and (`width` or `height`)	*Error (conflicts with default or provided `mm-per-pixel`)*
`bbox` and `center` (with or without additional parameters)	*Error (`bbox` and `center` are mutually exclusive)*

Table 7 — Parameter combinations for implementations supporting *Subsetting*, but not *Scaling*

PARAMETERS PROVIDED IN THE REQUEST	SERVER OR RESOURCE DEFAULTS USED	COMPUTED
`width` and `height`	`scale-denominator` and `center`	`bbox`

PARAMETERS PROVIDED IN THE REQUEST	SERVER OR RESOURCE DEFAULTS USED	COMPUTED
`bbox`	`scale-denominator`	`center`, `width` and `height`
`center`	`scale-denominator`, `width` and `height`	`bbox`
`center`, `width` and `height`	`scale-denominator`	`bbox`
`scale-denominator` [1]	`center`	`bbox`, `width` and `height`
`scale-denominator` [1] and `center`	*None*	`bbox`, `width` and `height`
`scale-denominator`, `width` and `height`	*Error (would require rescaling the map)*	
`bbox`, `width` and `height`	*Error (would require rescaling the map)*	
`bbox` and `scale-denominator`	*Error (would require rescaling the map)*	
`scale-denominator`, `center`, `width` and `height`	*Error (would require rescaling the map)*	

[1] The `scale-denominator` parameter is defined in the *Scaling* requirements class. However, an implementation supporting only *Subsetting* may (**but is not required to**) still recognize the `scale-denominator` parameter and compute `width` and `height` dimensions accordingly, along with the corresponding bounding box. In this case, a Subsetting-only implementation may not be applying scale-dependent symbolization rules correctly, since it likely would not render the map anew, but simply cut a piece from a pre-rendered map of a default scale. This is not an issue for maps without any scale-dependent symbolization, such as plain imagery.

Table 8 — Parameter combinations for implementations supporting *Scaling*, but not *Subsetting*

PARAMETERS PROVIDED IN THE REQUEST	SERVER OR RESOURCE DEFAULTS USED	COMPUTED
`width` and `height`	`bbox` and `center`	`scale-denominator`
`scale-denominator`	`bbox` and `center`	`width` and `height`
`scale-denominator`, `width` and `height`	*Error (would require subsetting the map)*	
`bbox` or `center` (with or without additional parameters)	*Error (would require subsetting the map)*	

Table 9 — Parameter combinations for implementations supporting both *Subsetting* and *Scaling*

PARAMETERS PROVIDED IN THE REQUEST	SERVER OR RESOURCE DEFAULTS USED	COMPUTED
`width` and `height`	`scale-denominator` and `center`	`bbox`
`bbox`	`width` and `height`	`scale-denominator` and `center`
`center`	`scale-denominator`, `width` and `height`	`bbox`
`center`, `width` and `height`	`scale-denominator`	`bbox`
`scale-denominator`	`center`, `width` and `height`	`bbox`
`scale-denominator` and `center`	`width` and `height`	`bbox`
`scale-denominator`, `width` and `height`	`center`	`bbox`
`bbox`, `width` and `height`	*None (fully defined combination[1])*	`scale-denominator` and `center`
`bbox` and `scale-denominator`	*None (fully defined combination[2])*	`center`, `width` and `height`
`scale-denominator`, `center`, `width` and `height`	*None (fully defined combination[2])*	`bbox`

[1] This combination corresponds to the WMS parameters and should be used for obtaining identical results from different implementations. [2] Different implementations may maintain a slightly different relationship between the dimensions (`width` and `height`), the spatial extent (`bbox`) and the `scale-denominator`, based on different considerations for calculating the scales of the map across each dimension. This may result in the `bbox`, `width` or `height` being computed differently between these implementations. Clients should always use `Content-Bbox:` header to properly georeference the output, and not expect unspecified parameters to be computed to a particular value.

NOTE: Changing the output CRS using the `crs` parameter will of course also have an impact on the mapping between pixels on the map and units in the real world, and on the calculated bounding box (in output CRS units).

See examples in an annex for computations infering dimensions and infering bounding boxes based on specified parameters.

6.7. Available formats and map response expectations

The Maps API Standard defines six requirements classes for specific encodings to encode map data. Additional encodings can be supported using HTTP content negotiation, following conventions specific to those encodings.

7

REQUIREMENTS CLASS "CORE"

7 REQUIREMENTS CLASS "CORE"

7.1. Overview

REQUIREMENTS CLASS 1: REQUIREMENTS CLASS 'CORE'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core`
NORMATIVE STATEMENTS	Requirement 1: `/req/core/map-op` Requirement 2: `/req/core/map-response` Requirement 3: `/req/core/conformance-success`

The core requirements class specifies a map resource. This is a resource that represents geospatial data as a rendered map. Requesting a map resource, as defined in the core, results in retrieving a generic map as a graphical representation of the data in the repository being accessed via the API endpoint. When using an implementation of the core requirements class, the representation is not constrained in any way by the client. Therefore, the server is free to return a total or a partial representation of the resource at an arbitrary size. Additional Maps API requirements classes add query parameters to personalize the server behavior to the client needs. These requirements classes define specific parameters that allow the client to control the map size and resolution of the map (width, height, bounding box and CRS) as well as the background of the map. An additional requirements class defines how a map resource can be retrieved as tiles by applying the OGC API — Tiles.

A map resource can be obtained from an arbitrary geospatial resource by adding `/map` to the resource URI. The core requirements class does not specify to which resources `/map` is applicable. However, a list of common paths to resources exposed by the API, where maps can commonly be obtained is presented in the following table. The Collection Maps, Dataset maps and Styled Maps requirements classes define some of these possibilities.

Table 10 — Common map resources in a geospatial API

RESOURCE PATH	DESCRIPTION
/map	A map representing datasets behind the API in the default style

RESOURCE PATH	DESCRIPTION
/styles/{styleId}/map	A map representing datasets behind the API in the `styleId` style
/collections/{collectionId}/map	A map representing `collectionId` in the default style
/collections/{collectionId}/styles/{styleId}/map	A map representing `collectionId` in the `styleId` style

NOTE: There is no mandatory dependency of the Maps API core on *OGC API — Common*. This allows various Web APIs to implement the core requirements class without the need to comply to the OGC API — Common structure (landing page, conformance, API description). However, many implementations may specify a dependency on *OGC API — Common* in their conformance page.

7.2. Map resource

A map distribution of a geospatial resource is a pictorial representation of that resource (map). To create a pictorial representation, a style is added to the data in the geospatial resource by a portrayal engine and following portrayal rules. This style can be internal (default style) or can be governed by the client.

This section defines the core resource of the *OGC API — Maps* Standard: A map representation for a geospatial resource. To keep the core of *OGC API — Maps* simple, the core only includes a mechanism to retrieve a map of the size and for the spatiotemporal extent that the server considers optimal.

7.2.1. Operation

REQUIREMENT 1	
IDENTIFIER	`/req/core/map-op`
INCLUDED IN	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
A	Every map SHALL be available as an HTTP GET request to a URI that will be composed of three parts. The first part is the URI of a resource that can be represented as a map (with or without a style path parameter). The second part follows the pattern `/map`. The third part provides the retrieval parameters as needed.

7.2.2. Response

REQUIREMENT 2	
IDENTIFIER	`/req/core/map-response`
INCLUDED IN	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
A	A successful execution of a map operation SHALL be a response with an HTTP status code 200.
B	The map response SHALL be in the storage CRS specified in the description of the geospatial resource (such as the associated collection or dataset), or https://www.opengis.net/def/crs/OGC/1.3/CRS84 if none is specified, unless overridden by a specific query parameter (see Requirements Class "Coordinate Reference System").
C	The headers of the response SHALL include the `Content-Crs:` header with the URI or the safe CURIEs of the CRS used to render the map, except if the content is in the https://www.opengis.net/def/crs/OGC/1.3/CRS84 CRS.
D	The headers of the response SHALL include a `Content-Bbox:` header with the actual geospatial boundary of the rendered map.
E	The `Content-Bbox:` coordinates SHALL be in the response CRS (indicated in the `Content-Crs:` or https://www.opengis.net/def/crs/OGC/1.3/CRS84 if it is not present) and SHALL contain four comma-separated numbers representing the lower-left and upper-right corners of the response honoring the CRS coordinates order.
F	If the geospatial resource has a temporal aspect (e.g., if a temporal extent is defined in the collection description or in the dataset landing page), the headers of the response SHALL include a `Content-Datetime:` header with the actual datetime instant or datetime interval of the rendered map.
G	In the `Content-Datetime:` response header, a datetime interval SHALL be expressed using the notation `time-instant / time-instant`.
H	In the `Content-Datetime:` response header, a time instant, whether a single value or the `time-instant` of an interval, SHALL be expressed as defined in RFC 3339, section 5.6, using either UTC time or local time offsets, with the additional options of: • only a year: `yyyy` • a year and a month: `yyyy-mm` • a year, a month and a day: `yyyy-mm-dd` • a year, a month, a day and an hour: `yyyy-mm-ddThhZ` (or e.g., `yyyy-mm-ddThh-04:00`) • a year, a month, a day, an hour and a minute: `yyyy-mm-ddThh:mmZ` (or e.g., `yyyy-mm-ddThh-04:00`)
I	The body of a response of a successful operation SHALL contain a map encoded in the negotiated format.

Below is an example of the successful HTTP response headers describing a PNG image with 1500 columns and 616 rows in the datum WGS84 and projection UTM zone 31N centered in the Barcelona area.

```
HTTP/1.1 200 OK
Content-Type: image/jpeg
Last-Modified: Fri, 05 Jul 2024 16:46:55 GMT
Content-Crs: <https://www.opengis.net/def/crs/EPSG/0/25831>
Content-Bbox: 392011.47,4563522.45,453461.47,4591522.45
Content-Datetime: 2020-01
Content-Length: 190946
```



Figure 5 — Example of a successful HTTP response body containing a JPEG image captured by LANDSAT in January 2020

Below is another example of the successful HTTP response headers describing a PNG image in the datum WGS84 and coordinates centered in the Barcelona area.

```
HTTP/1.1 200 OK
Content-Type: image/png
Last-Modified: Thu, 27 Jul 2023 14:37:46 GMT
Content-Bbox: 2.1486,41.3674,2.1886,41.4074
```

7.2.3. Recommendations

RECOMMENDATION 1	
IDENTIFIER	`/rec/core/map-op`
A	In the absence of subsetting parameters, the server SHOULD return the entire map.
B	In absence of the scaling parameter, a reasonable width and height SHOULD be selected that preserves the aspect ratio.

RECOMMENDATION 2

IDENTIFIER	`/rec/core/content-crs`
A	Even if the content is in the https://www.opengis.net/def/crs/OGC/1.3/CRS84 CRS the headers SHOULD still include a “Content-Crs” header to state the CRS.

RECOMMENDATION 3

IDENTIFIER	`/rec/core/content-attribution`
A	A `Content-Attribution:` header SHOULD be included in the response to a map request to indicate any attribution relevant to the data being returned, especially if this attribution varies based on the subset and scale of the request.

RECOMMENDATION 4

IDENTIFIER	`/rec/core/legend-thumbnail-style`
A	If a legend representing the symbols, lines and colors in the style applied to the map is available for a style of the map, or for the default map resource, the server SHOULD advertise the legend in the list of styles (for each style element) or in the originating resource for the map (e.g., the landing page or collection description), by providing an extra link with a relation type `rel=https://www.opengis.net/def/rel/ogc/1.0/legend`.
B	The server SHOULD support `minScaleDenominator` and `maxScaleDenominator` query parameters for the legend resource specifying the range of scales applicable to the style. Without these parameters the legend is applicable to all scales.
C	If a thumbnail is available for a style of the map, the server SHOULD advertise the thumbnail in the list of styles (for each style element) or in the originating resource for the map by providing an extra link with a relation type `rel=preview`.
D	If one or more style resources are available to apply the style to the map, the server SHOULD advertise them in the list of styles (for each style element) or in the originating resource for the map by providing extra links with a relation type `rel=stylesheet`.

RECOMMENDATION 5

IDENTIFIER	`/rec/core/multiple-media-types`
A	If “image/png” and “image/jpeg” are supported at an endpoint and both are present in the HTTP “Accept” request header with the same q value, and no other supported output format is present with a higher q value, the server SHOULD select between PNG and JPEG and return the format considered

RECOMMENDATION 5	
	optimal. Typically, PNG will be returned if there is transparency or for categorical maps where a limited number of colors are used, and JPEG otherwise.

Retrieving a map without subsetting or tiling has limited use. An implementation should therefore consider allowing retrieving only part of a map by supporting the *map tilesets* and/or the *spatial subsetting* requirements class.

NOTE 1: The desired encoding is selected using HTTP content negotiation. In addition to the parameters specified by the Maps API core, other parameters should be added.

NOTE 2: HTTP content negotiation addresses the same use cases as the `FORMAT=` parameter of WMS. However, for convenience, some implementations can implement an `f=` parameter for circumstances where content negotiation is not possible or available (e.g., testing the API in a web browser, or using a URL embedded in an HTML IMG SRC tag).

7.3. Declaration of conformance classes

To support "generic" clients that want to access implementations of multiple OGC API Standards and extensions — and not "just" a specific API / server, the deployed API must declare the conformance classes it implements and conforms to.

The conformance declaration response mainly consists of a list of links.

REQUIREMENT 3	
IDENTIFIER	`/req/core/conformance-success`
INCLUDED IN	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
A	If the API instance has a mechanism to advertise conformance classes, the list of conformance classes SHALL include the ones defined in this standard and listed in Conformance class URIs that are supported by this API implementation instance.

If the server declares conformity also to *OGC API — Common — Part 1: Core* or to *OGC API — Features — Part 1: Core*, then the OGC API requirements for declaring conformance, such as the use of the `/conformance` resource, must be considered. In the JSON format, the conformance resource is an array of links following the link schema defined in *OGC API — Common — Part 1: Core* or in *OGC API — Features — Part 1: Core*. Below is an example fragment of a conformance information page of an API implementation conformant to *OGC API — Common* and *OGC API — Maps*.

An example Conformance Information Page fragment is below:

```
{
  "conformsTo": [
    "https://www.opengis.net/spec/ogcapi-common-1/1.0/conf/core",
    "https://www.opengis.net/spec/ogcapi-common-2/1.0/conf/collections",
    "https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core"
  ]
}
```

8

REQUIREMENTS CLASS "MAP TILESETS"

8 REQUIREMENTS CLASS "MAP TILESETS"

8.1. Overview

REQUIREMENTS CLASS 2: REQUIREMENTS CLASS 'MAP TILESETS'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tilesets`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.2: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/tilesets`
PREREQUISITES	https://www.opengis.net/spec/ogcapi-tiles-1/1.0/req/tilesets-list Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 4: `/req/tilesets/desc-links` Requirement 5: `/req/tilesets/tiles-parameters`

The Map Tilesets requirements class describes how to provide access to a map as tilesets in combination with the *OGC API — Tiles* Standard, which is based on the *OGC 2D Tile Matrix Set and Tileset Metadata* Standard. Tiles facilitate caching of data for clients and servers by requesting predefined subsets at fixed resolutions according to multi-resolution set of grids called a *TileMatrixSet*.

NOTE 1: WMTS was focused on map tiles. A map tile is a tile that contains information in a raster form where the values of cells are colors which can be readily displayed on rendering devices (OGC 20-057). Since *OGC API — Tiles* allows for other types of tiled data (e.g., tiled vector data and tiled coverage data), the term *'map tiles'* is used in a similar way to how it was used in WMTS 1.0.

While supporting map tiles and map tilesets without conforming to this requirements class is possible, implementing this requirements class means that the tiles resources support parameters defined by *OGC API — Maps* in the following requirements classes:

- Background (`bgcolor` and `transparent` parameter),
- Display resolution (`mm-per-pixel`),

- Spatial subsetting (only if a vertical dimension is available, since the 2D Tile Matrix Sets already handle partitioning the other two dimensions, and only for `subset` and `subset-crs` parameters),
- General subsetting (`subset` for dimensions beyond spatial and temporal),
- Scaling (`scale-denomimator`, `width` and `height` which will override the usual `tileWidth` and `tileHeight` of the requested tile in pixels from the tile matrix, but those values are still used for calculating the geospatial bounding box corresponding to each tile),

NOTE 2:*OGC API — Tiles* defines its own equivalent requirements class for temporal subsetting.

8.2. Link from data resource

A map resource can be accessed as tiles if the API implementation instance exposes a link to map tilesets in the resource that is available as map tiles.

REQUIREMENT 4	
IDENTIFIER	`/req/tilesets/desc-links`
INCLUDED IN	Requirements class 2: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tilesets`
A	The geospatial data resource (e.g., the collection or landing page description's `links` property) SHALL include a link with the href pointing to a list of tilesets provided from this geospatial data resource using rel: `https://www.opengis.net/def/rel/ogc/1.0/tilesets-map`.

Example — Fragment of a collection description document with a links array and with one item of the array pointing to a list of map tilesets.

```
{
    "links": [
    ...
    {
      "href": "https://data.example.com/collections/buildings/map/tiles",
      "rel": "https://www.opengis.net/def/rel/ogc/1.0/tilesets-map",
      "type": "application/json"
    }
  ]
}
```

8.3. Map Tileset

A tileset contains the necessary metadata to enable a client application to formulate a tile request from a single geospatial data resource.

8.3.1. Operation

The operations to retrieve a list of tilesets, individual tilesets and individual tiles are described respectively in the Tilesets List, TileSet and Core requirements classes of the *OGC API — Tiles* Standard.

The Map Tileset requirements class specifies that the query parameters of the GET request to individual map tiles support additional parameters as defined in the *background*, *display resolution*, *spatial subsetting* (if a vertical dimension is available), *general subsetting*, and *scaling* requirements classes.

REQUIREMENT 5	
IDENTIFIER	`/req/tilesets/tiles-parameters`
INCLUDED IN	Requirements class 2: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tilesets`
A	Implementations of the Maps API SHALL support parameters specified in the *background*, *display resolution*, *spatial subsetting* (only for `subset` and `subset-crs` using a vertical dimension, if available), *general subsetting*, and *scaling* requirements classes, for map tiles endpoint if the implementation declares conformance to the associated conformance classes.

8.3.2. Response

Successful GET responses to requests for a list of map tilesets, an individual map tileset, and map tiles are described in the corresponding requirements class of the *OGC API — Tiles* Standard. The response to map tiles requests shall also reflect the parameters as specified in the operation requirement.

9

REQUIREMENTS CLASS "BACKGROUND"

9 REQUIREMENTS CLASS "BACKGROUND"

9.1. Overview

REQUIREMENTS CLASS 3: REQUIREMENTS CLASS 'BACKGROUND'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/background`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.3: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/background`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 6: `/req/background/bgcolor-definition` Requirement 7: `/req/background/transparent-definition` Requirement 8: `/req/background/void-color-definition` Requirement 9: `/req/background/void-transparent-definition` Requirement 10: `/req/background/map-success`

The Background requirements class describes how to define the background of the map that is presented.

9.2. Map operation

The Maps API core requirements class defines how to get a map. The Map operation requirements class specifies parameters supporting customization of the background of the map.

9.2.1. Parameters `transparent` and `bgcolor`

The `transparent` and `bgcolor` parameters indicate how the absence of data will be represented in the map, what color will show underneath any non-opaque layer, and allow for the map to be overlaid with other maps without completely obscuring the lower layers.

REQUIREMENT 6

IDENTIFIER	`/req/background/bgcolor-definition`
INCLUDED IN	Requirements class 3: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/background`

A	The map operation SHALL support a parameter `bgcolor` to define a background color with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`bgcolor:`
	`  name: bgcolor`
	`  in: query`
	`  description: |-`
	`  Hexadecimal red-green-blue color value (0-255) or case-insensitive W3C web color name for the`
	`  background color (default=0xFFFFFF). For a six digit hexadecimal value, the first and second`
	`  digits specify the intensity of red. The third and fourth digits specify the intensity of`
	`  green. The fifth and sixth digits specify the intensity of blue.`
	`  required: false`
	`  style: form`
	`  explode: false`
	`  schema:`
	`  type: string`
	`  default: 0xFFFFFF`

B	The map operation SHALL support a `bgcolor` parameter which can be a hexadecimal red-green-blue color value (from 00 to FF, FF representing 255) or a case-insensitive W3C web color name for the background color of the map (default=0xFFFFFF). For a six-digit hexadecimal value, the first and second digits specify the intensity of red. The third and fourth digits specify the intensity of green. The fifth and sixth digits specify the intensity of blue.
C	If `bgcolor` is not specified, and either `transparent` is set to `false` or the output format cannot encode transparency, the server SHALL use the background color specified by the requested style.

RECOMMENDATION 6

IDENTIFIER	`/rec/background/bgcolor-definition`
A	If `bgcolor` is not specified, and either `transparent` is set to `false` or the output format cannot encode transparency and the style applied to the map does not specify a background color, the server SHOULD use a neutral default color such as white or a light shade of gray.

The list of W3C web color names which can be specified instead of a hexadecimal value can be found in Background color codes

PERMISSION 1

IDENTIFIER	`/per/background/bgcolor-alpha`

PERMISSION 1

A	The implementation MAY support a `bgcolor` parameter with eight hexadecimal digits where the first two represent an alpha value, to be used as the opacity value for no data areas (where "FF" is completely opaque and "00" is fully transparent).

REQUIREMENT 7

IDENTIFIER	`/req/background/transparent-definition`
INCLUDED IN	Requirements class 3: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/background`

A	The map operation SHALL support an optional parameter `transparent` specifying whether to use or not a transparent background with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`name: transparent`
	`in: query`
	`description:`
	`'(defaults to `true` without a `bgcolor` specified, but to `false` when a `bgcolor` is used).'`
	`required: false`
	`style: form`
	`explode: false`
	`schema:`
	`type: boolean`
	`default: true`

B	The server SHALL interpret `transparent` as a Boolean indicating whether the background of the map should be transparent.
C	If `transparent` is not specified and a `bgcolor` is not specified, the server SHALL assume a value of `true`.
D	If `transparent` is not specified and a `bgcolor` is specified, the server SHALL assume a value of `false`.
E	If `transparent` is `true` and a `bgcolor` is specified, the server SHALL use 0 for the background's opacity.

NOTE 1: When not specified, the value of `transparent` defaults to `true` when `bgcolor` is not used, but to `false` when used together with `bgcolor`. For formats with an alpha channel, explicitly setting `transparent=true` together with a `bgcolor` allows for defining the color value to use for the RGB channels where there is no data. Zero (0) will be used for the alpha channel. For formats reserving a color to define transparency (color key), the combination supports selecting a color that does not interfere with the actual values and colors in the map.

NOTE 2: The `opaque` attribute of layer element in the GetCapabilities response in WMS is not specified by the Maps API.

9.2.2. Parameters `void-transparent` and `void-color`

The `void-transparent` and `void-color` parameters indicate how parts of the map outside of the valid area of the projection and CRS will be represented.

If not specified, these parameters default to the same value as `transparent` and `bgcolor`, respectively.

NOTE: These parameters only apply to CRSs including an invalid area, such as a CRS using an orthographic projection.

REQUIREMENT 8	
IDENTIFIER	`/req/background/void-color-definition`
INCLUDED IN	Requirements class 3: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/background`

A	The map operation SHALL support a parameter `void-color` to define a void color with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`void-color:`
	`  name: void-color`
	`  in: query`
	`  description: |-`
	`  Hexadecimal red-green-blue color value (0-255) or case-insensitive W3C web color name for the`
	`  color of parts of the map outside of the valid CRS / projection areas (default=0xFFFFFF). For a six digit hexadecimal value, the first and second`
	`  digits specify the intensity of red. The third and fourth digits specify the intensity of`
	`  green. The fifth and sixth digits specify the intensity of blue.`
	`  required: false`
	`  style: form`
	`  explode: false`
	`  schema:`
	`  type: string`
	`  default: 0xFFFFFF`

B	The map operation SHALL support a `void-color` parameter which can be a hexadecimal red-green-blue color value (from 00 to FF, FF representing 255) or a case-insensitive W3C web color name for the parts of the map outside of the valid areas of the projection / CRS (default=0xFFFFFF). For a six-digit hexadecimal value, the first and second digits specify the intensity of red. The third and fourth digits specify the intensity of green. The fifth and sixth digits specify the intensity of blue.
C	If `void-color` is not specified, the same color value as for `bgcolor` (specified or default) SHALL be used.

As for `bgcolor`, the W3C web color names can be specified instead of a hexadecimal value.

REQUIREMENT 9	
IDENTIFIER	`/req/background/void-transparent-definition`
INCLUDED IN	Requirements class 3: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/background`

A	The map operation SHALL support an optional parameter `void-transparent` specifying whether to use a different transparent setting for parts of the map outside of the valid areas of the projection / CRS with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`name: void-transparent`
	`in: query`
	`description:`
	`  '(defaults to the same value, specified or default, as `transparent`).'`
	`required: false`
	`style: form`
	`explode: false`
	`schema:`
	`  type: boolean`
	`  default: true`

B	The server SHALL interpret `void-transparent` as a Boolean indicating whether the parts of the map outside of the valid areas of the projection / CRS should be transparent.
C	If `void-transparent` is not specified, the server SHALL assume the same value as for `transparent` (specified or default).

9.2.3. Response

A successful GET response is described in the core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core)

REQUIREMENT 10	
IDENTIFIER	`/req/background/map-success`
INCLUDED IN	Requirements class 3: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/background`
A	The color of the map in the areas with no data SHALL be exactly the one specified in the `bgcolor`.
B	The color in parts of the map outside of the valid areas of the projection / CRS SHALL be the one specified by `void-color`, or otherwise default to the same as the background color (whether specified by `bgcolor` or default).
C	The transparency setting in parts of the map outside of the valid areas of the projection / CRS SHALL be the one specified by `void-transparent`, or otherwise default to the same as the background transparency setting (whether specified by `transparent` or default).
D	In case the output format allows it and in the absence of the `transparent` parameter (or if it is `false`), the opacity (alpha value) of the map in the areas with no data SHALL be exactly 100% if `transparent` is `false` or 0% if `transparent` is `true`.

NOTE: If the renderer supports anti-aliasing, at the edges between data and no-data areas, the opacity may be a value between 0% and 100%.

10

REQUIREMENTS CLASS "COLLECTION SELECTION"

10 REQUIREMENTS CLASS "COLLECTION SELECTION"

10.1. Overview

REQUIREMENTS CLASS 4: REQUIREMENTS CLASS 'COLLECTION SELECTION'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collections-selection`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.4: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/collections-selection`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 11: `/req/collections-selection/collections-parameter` Requirement 12: `/req/collections-selection/collections-response`

In a deployed Web API incorporating elements of the OGC API — Maps Standard that provides access to a complex dataset formed by several geospatial data resources, selecting specific sub-resources of interest when requesting data from this dataset can be useful. The Collection Selection requirements class defines how to include a query parameter when requesting a resource (e.g., dataset tiles) to specify which geospatial data resources (e.g., collections) should be used to generate the response.

This requirements class is particularly useful in conjunction with the Requirements Class "Dataset Map" requirements class. However, sub-components of other resource types could be selected using this requirements class. A requirements class equivalent to this one is included in *OGC API — Tiles*.

10.2. Operation

By default, the geospatial data resources included in the dataset maps responses are unspecified but should represent the dataset as a whole (but not necessarily the entire extent of the dataset content). This class adds a mechanism to select the geospatial data resources to be used to

generate the derived resources (e.g., maps or map tiles). In practice this enables the capability to generate resources involving more than one geospatial data sub-resource.

10.2.1. Parameter `collections`

REQUIREMENT 11	
IDENTIFIER	`/req/collections-selection/collections-parameter`
INCLUDED IN	Requirements class 4: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collections-selection`

A	An operation that acts on a resource consisting of multiple geospatial data sub-resources (e.g., a resource derived from a root dataset) SHALL support an optional parameter `collections` with the following characteristics (shown as an OpenAPI Specification 3.0 fragment):
	`name: collections`
	`in: query`
	`required: false`
	`style: form`
	`explode: false`
	`schema:`
	`  type: array`
	`  items:`
	`  type: string`

B	The parameter `collections` SHALL be supported by maps originating from resources consisting of multiple geospatial data sub-resources that can be addressed by identifiers (e.g. dataset map at `{datasetAPI}/maps/`).
C	Implementations SHALL support both a comma-separated list of geospatial resource identifiers (e.g., collectionId's) and full URLs to local geospatial resources.

When this parameter refers to more than one geospatial data resource, this parameter will use the comma (",") as the separator between the resource identifiers in the list. Additional white space will not be used to delimit list items. If a geospatial data resource identifier includes a space or comma, it shall be escaped using the URL encoding rules (IETF RFC 2396).

PERMISSION 2	
IDENTIFIER	`/per/collections-selection/valid-collections`
A	An implementation MAY return an error when the specified list of collections is not supported, for reasons such as an incompatible combination, or an unsupported encoding or CRS for some of the selected collections.

An example request is:

`www.example.com/map?collections=buildings,roads`

10.2.2. Response

A successful GET response is described in the core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core)

REQUIREMENT 12	
IDENTIFIER	`/req/collections-selection/collections-response`
INCLUDED IN	Requirements class 4: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collections-selection`
A	Only the collections of geospatial data enumerated in the values of the `collections` parameter SHALL be used to generate the responses for the resource (map) to which they apply.
B	If there is more than one collection name and the style applied does not specify otherwise, collections SHALL be rendered in the result in an order starting with the first (leftmost) collection and ending with the last (rightmost).

NOTE: In maps and map tiles, sub-requirement B will result in the first collection being portrayed at the "bottom" of the display stack with the others rendered on top of the previous ones, one by one (the rightmost collection will become topmost in the portrayal).

10.2.3. Error conditions

If the value of the parameter `collections` contains a resource id of URI that does not exist on the implementation instance of the Maps API or too many collections are requested, the status code of the response is 400.

If the value of the parameter `collections` has a wrong format or combines one of more geospatial data resources that are not compatible (e.g., they do not have a compatible value specified by other parameters in the request), the status code of the response is 400.

10.3. Service Metadata

The OGC API — Common — Part 1: Core Standard describes a mechanism to expose service-metadata for the API. See: https://docs.ogc.org/is/19-072/19-072.html#service-metadata-examples

RECOMMENDATION 7

IDENTIFIER	`/rec/collection-selection/maxcollections`

A	The service metadata may include a maximum number of collections that can be included to create the map. Clients should not request a map with more collections than this limit and if they do, they should expect that the server will not be able to create a map. These characteristics are defined as (shown as an OpenAPI Specification 3.0 fragment).
	`x-OGC-limits:`
	`  type: object`
	`  properties:`
	`  maps:`
	`  type: object`
	`  properties:`
	`  maxCollections:`
	`  type: integer`
	`  description:`
	`  Maximum number of collections in the collections parameter. If absent the server imposes no limit.`
	`  example: 5`

B	In the absence of maxCollections the client will assume that there is no limit in the number of collections.

NOTE: This mimics the `LayerLimit` element in the WMS 1.3 *GetCapabilities* document.

11

REQUIREMENTS CLASS "SCALING"

11 REQUIREMENTS CLASS "SCALING"

11.1. Overview

REQUIREMENTS CLASS 5: REQUIREMENTS CLASS 'SCALING'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/scaling`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.5: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/scaling`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 13: `/req/scaling/width-definition` Requirement 14: `/req/scaling/height-definition` Requirement 15: `/req/scaling/scale-denominator-definition`

The Scaling requirements class describes how to scale a map for display by specifying a set of parameters that will define its size (width and height) in pixels of a display device that indirectly defines the scale of the map.

11.2. Map Operation

The Maps API *core* requirements class defines how to retrieve a map. The Scaling requirements class specifies two alternative mechanisms for specifying the scale of the map, either implicitly by providing output dimensions using `width` and `height` parameters, or explicitly by providing a `scale-denominator` parameter.

11.2.1. Parameters `width` and `height`

The width and height parameters indicate the size of the viewport in rows and columns where the map is to be displayed. If the format of the response is a raster image, the number of

columns and rows of the image in the response will commonly match the width and height of the viewport.

REQUIREMENT 13	
IDENTIFIER	`/req/scaling/width-definition`
INCLUDED IN	Requirements class 5: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/scaling`

A	The map operation SHALL support a parameter `width` with the characteristics defined in the Open API Specification 3.0 fragment:
	`width:`
	`  name: width`
	`  in: query`
	`  description:`
	`  Width of the viewport in pixel units to present the response (the map subset).`
	`  required: false`
	`  style: form`
	`  schema:`
	`  type: number`

B	The `width` value SHALL be interpreted as the horizontal size (columns) of the viewport where the response will be presented in pixel units (number of pixels).
C	An HTTP 4xx error SHALL be returned if the `width` number is not a positive integer number.
D	An HTTP 4xx error SHALL be returned (preferably a 413) if the value of the `width` exceeds the `maxWidth` property specified in the `x-OGC-limits.maps` object included in the service metadata[1].
E	An HTTP 4xx error SHALL be returned (preferably a 413) if the value of the `width` (specified or calculated) times `height` (specified or calculated) exceeds a `maxPixels` property from an `x-OGC-limits.maps` object included in the service metadata[1].
F	An HTTP 4xx error SHALL be returned if the `width` parameter is used together with the `bbox` (or `subset` for spatial dimensions) as well as the `scale-denominator` parameter.
G	An HTTP 4xx error SHALL be returned if the `width` parameter is used together with the `scale-denominator` parameter and the implementation does not also support the “Subsetting” requirements class.
H	When the `width` parameter is omitted, the implementation SHALL use an appropriate width which accurately reflects the default or requested scale established as the ratio between the horizontal dimension of the viewport and the corresponding size of the physical world, specifically for the local subset (bounding box) of the map being returned, and taking into consideration the default (0.28 mm/pixel) or specified display resolution (`mm-per-pixel`). [1] Service metadata may be provided as an extension of the `info` section of the OpenAPI document as indicated in OGC API – Common – Part 1: Core, Annex B.4 and discussed in the last subsection of the Maps API scaling requirements class section.

REQUIREMENT 14	
IDENTIFIER	`/req/scaling/height-definition`
INCLUDED IN	Requirements class 5: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/scaling`

A	The map operation SHALL support a parameter `height` with the characteristics defined in the Open API Specification 3.0 fragment:
	`height:`
	`  name: height`
	`  in: query`
	`  description:`
	`  Height of the viewport in pixel units to present the response (the map subset).`
	`  required: false`
	`  style: form`
	`  schema:`
	`  type: number`

B	The `height` value SHALL be interpreted as the vertical size (rows) of the viewport where the response will be presented in pixel units (number of pixels).
C	An HTTP 4xx error SHALL be returned if the `height` value is not a positive integer number.
D	An HTTP 4xx error SHALL be returned (preferably a 413) if the value of the `height` exceeds the `maxHeight` property specified in the `x-OGC-limits.maps` object included in the service metadata[1].
E	An HTTP 4xx error SHALL be returned (preferably a 413) if the value of the `width` (specified or calculated) times `height` (specified or calculated) exceeds a `maxPixels` property from a `x-OGC-limits.maps` object included in the service metadata[1].
F	An HTTP 4xx error SHALL be returned if the `height` parameter is used together with the `bbox` (or `subset` for spatial dimensions) as well as the `scale-denominator` parameter.
G	An HTTP 4xx error SHALL be returned if the `height` parameter is used together with the `scale-denominator` parameter and the implementation does not also support the “Subsetting” requirements class.
H	When the `height` parameter is omitted, the implementation SHALL use an appropriate height which accurately reflects the default or requested scale established as the ratio between the vertical dimension of the viewport and the corresponding size of the physical world, specifically for the local subset (bounding box) of the map being returned. [1] Service metadata may be provided as an extension of the `info` section of the OpenAPI document as indicated in OGC API – Common – Part 1: Core, Annex B.4 and discussed in the last subsection of the Maps API scaling requirements class section

NOTE: The `width` and `height` parameters are also defined by the Maps API *spatial subsetting* requirements class. The parameter takes this resampling (scaling) role in requests not using either of the `scale-denominator` parameter (defined in this requirements class), or the `center` parameter (defined in the *spatial subsetting* requirements class), or when the *spatial subsetting* requirements class is not supported. In combination with a bounding box and display resolution

(either the default of 0.28 mm/pixel or one specified by `mm-per-pixel` parameter from the *display resolution* conformance class), the `width` and `height` parameters define the scale of the map.

RECOMMENDATION 8	
IDENTIFIER	`/rec/scaling/dimensions`
A	If the `width` or `height` parameter is not specified for a request, and no specific bounding box is requested (using the `subset` or `bbox` parameters defined in the Subsetting requirements class), the implementation SHOULD use the same value for both dimensions to return a map with a square viewport or preserve a particular width to height ratio.
B	If neither `width` nor `height` parameter is specified for a request, and no specific bounding box is requested (using the `subset` or `bbox` parameters defined in the Subsetting requirements class), the implementation SHOULD select reasonable dimensions that will produce a map that is neither too small, nor require too much bandwidth to transfer or process (for example, around 1000 x 1000 pixels for the default 0.28 mm/pixel resolution).

11.2.2. Parameter `scale-denominator`

A client can specify the desired scale of the map as a scale denominator value using a `scale-denominator` parameter. Given a selected unit of measure, the scale denominator specifies how many of these units in the real world correspond to a distance of one unit on the map as printed or displayed.

NOTE 1: The correspondence between real world units and display units depends on the display resolution for which a default 0.28 mm/pixel is assumed, unless a client specifies otherwise through the display resolution requirements class.

NOTE 2: The `scale-denominator` is the only way to specify scaling when requesting a spatial subset using a `center` point.

REQUIREMENT 15	
IDENTIFIER	`/req/scaling/scale-denominator-definition`
INCLUDED IN	Requirements class 5: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/scaling`

A	The map operation SHALL support a parameter `scale-denominator` with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`scale-denominator:`
	`  name: scale-denominator`
	`  in: query`
	`  description:`
	`  Number of units in the real-world corresponding to one such unit on the display.`
	`  required: false`
	`  style: form`

REQUIREMENT 15

```
      schema:
        type: number
```

B	The `scale-denominator` value SHALL be interpreted as the number of real-world units corresponding to one of the same unit on the map (as printed or displayed), considering only the local subset of the map being returned, based on the selected (e.g., from display resolution requirements class) or default (0.28 mm/pixel) display resolution.
C	The implementation SHALL establish the correspondence between real-world units and pixel units based on the equation: *physicalMetersPerPixel*= (`mm-per-pixel` / 1000 mm/m) * `scale-denominator`, where the *physicalMetersPerPixel* are not necessarily the same as the CRS units (even if those units are expressed in meters) for the region of that CRS consisting of the map subset being returned.
D	An HTTP 4xx error SHALL be returned if the `scale-denominator` parameter is used together with `width` and/or `height` and the implementation does not declare conformance to the *spatial subsetting* requirements class (which specifies that the `width` and `height` parameters can also take on a subsetting role).
E	An HTTP 4xx error SHALL be returned if the `scale-denominator` parameter is used together with `width` and/or `height` as well as a `bbox` (or equivalent `subset` parameter for a spatial dimension).
F	If the `scale-denominator` parameter is omitted, the implementation SHALL compute it as needed (for purposes such as applying scale-dependent symbology rules) based on the default or selected dimensions, display resolution, and the spatial subset of the map to return.
G	For implementations also supporting "Subsetting", when the spatial subset of the map is not specified in the request, the `scale-denominator` value (default or specified) SHALL be used to compute this bounding box, taking into consideration the display resolution as well as the default or specified dimensions.

RECOMMENDATION 9

IDENTIFIER `/rec/scaling/symbology`

A	An implementation serving dynamically rendered maps with styling capabilities SHOULD consider the requested or computed scale in applying scale-dependent symbology rules.

11.2.3. Response

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core)

RECOMMENDATION 10

IDENTIFIER `/rec/scaling/map-success-scale`

RECOMMENDATION 10	
A	The resolution of the content represented in the map SHOULD be consistent with the `width` and `height` of the viewport indicated and should consider the display resolution (0.28 mm/pixel, unless the `mm-per-pixel` parameter is used) when applying symbology rules. These considerations should also be applied when simplifying the geometries for a vector format, such as KML or SVG.

11.2.4. Error conditions

A general summary of the HTTP status codes can be found in OGC API — Features — Part 1: Core, version 1.0 as well as in the OGC API – Common – Part 1: Core Standard.

If the values of `width` or `height` parameters are out-of-range, the status code of the response will be `413` (Payload too big). If a map is not provided due to lack of data in the area, the status code of the response will be `204` or `404`.

11.3. Scale and aspect ratio considerations (informative)

Since all parameters for map requests are optional, implementations must fill in missing parameters by using default values or computing appropriate values based on the parameters provided by the client and the specifics of the resource for which a map is being retrieved. Implementations supporting the “Scaling” requirements class should compute these values in a way that respects the scale of the map in all dimensions, and thus the aspect ratio of physical features being represented.

For a non-global map, and for implementations supporting both “Scaling” and “Subsetting” where clients can request a subset of the map for a small local region, implementations should consider the **local** scale of the map. For example, the applicable scale at the center, or most equatorial latitude of the map. This is as opposed to the global scale (e.g., at the equator). In addition, implementations should aim for the map scale to apply evenly to all dimensions. Consider a CRS such as Plate Carrée projection (CRS84 or EPSG:4326). When filling in an unspecified `bbox` and/or `width` and `height` parameters, this is possible by applying a different linear scaling for the horizontal and vertical dimensions between the CRS units and the pixel dimensions, based on the most equatorial latitude included in the subset.

Another example is using a World Mercator projection (EPSG:3395). Although the CRS unit is “meters”, they are only true meters at the equator. When computing a bounding box extending away from a `center` and a `scale-denominator`, the ratio between the CRS unit meters and true physical meters should be considered in order to accurately reflect the requested scale.

The ideal aspect ratio to maintain is the `physical_width / physical_height` = `width / height`, whereas `physical_width` and `physical_height` refer to the length of features in the horizontal and vertical dimensions respectively, and `width` and `height` are the dimensions of the returned map in pixels.

In the case of a Plate Carrée CRS, a simple way to accurately reflect the two-dimensional scale is to use the most equatorial latitude contained in the map response as the standard parallel of the equirectangular projection. This effectively implies scaling the horizontal distances of the Plate Carrée projection (assuming a standard parallel at 0°N) units by the cosine of that most equatorial latitude in order to get a more accurate physical horizontal distance.

In the case of a World Mercator CRS, an implementation could compute the number of CRS units separating one degree of longitude at the center latitude and compare that with the actual number of physical meters, based on the `cos(lat)` scaling mentioned above, and the length of a degree in meters (WGS84 semi-major axis of 6,378,137 m * Pi / 180, or 111,319.4907932735805 m).

For slightly more accurate calculations, the WGS84 ellipsoid, can be taken into account using the following polynomial equations (from Wikipedia):

```
metersPerDegLat = 111132.92 - 559.82 * cos(2*lat) + 1.175 * cos(4*lat) - 0.0023
* cos(6*lat)
metersPerDegLon = 111412.84 * cos(lat) - 93.5 * cos(3*lat) + 0.118 * cos(5*lat)
```

If maintaining the aspect ratio between physical units proves too difficult, implementations could fall back to maintaining the CRS unit aspect ratio instead. This way they ensure that `(maxx - minx)/(maxy - miny) = width / height`, where `minx, miny` are the lower bounds of the output CRS unit space in the returned map, `maxx, maxy` the upper bounds of that output CRS unit space on the map, and `width` and `height` are the dimensions of the returned map in pixels.

When neither width nor height is provided or implied, a good practical approach is to default to a reasonable size which is neither too low resolution, nor requiring too much computation power from the server.

When only one of the two dimension parameters is provided and no specific bounding box is requested, the value of the other could be used to return a square map by default.

When a bounding box is requested, the other dimension should be computed so as to preserve the aspect ratio.

When both dimensions are provided, but no bounding box, those dimensions should be used together with the default or requested center, and the scale denominator should be used to compute the bounding box.

Refer to the guidance for implementers provided in a section of the overview on the interactions of subsetting and scaling parameters. This includes the specific combinations of parameters that can be provided based on which requirements classes are implemented, and which parameter assume default values or should be computed in each case.

When `bbox`, `width` and `height` are all provided, as in implementations of the WMS Standard, the client enforces a specific aspect ratio, and these recommendations are not relevant. However, in general circumstances (e.g., 2D representations on a screen), the client should try to provide parameters that preserve the aspect ratio.

CAUTION

Due to the inherent difficulty in implementing the computations correctly for all CRSs and some degree of variability based on how these calculations are

performed, implementations may compute missing parameters for the same request differently. Clients should therefore always consider the `Content-Bbox:` response header and the actual dimensions of the returned map when georeferencing and displaying images returned by the Maps API. For identical responses to the same request across different implementations, clients should always provide a `bbox` (or its `subset` equivalent), `width` and `height` parameters (as in classic WMS requests). This is rather than the `scale-denominator` and `center` parameters, which are primarily intended for convenience directly accessing a Maps API, such as from a Web browser. For the purpose of overlaying map responses from different implementations, a client which does not itself georeference the output based on the `Content-Bbox:` response header should always request maps using the `bbox`, `width` and `height` parameters.

See also detailed step-by-step examples in an annex of computations to infer dimensions and infer bounding boxes based on specified parameters.

11.4. Service Metadata

The OGC API — Common Standard describes a mechanism to expose service-metadata for the API. See Annex B.4 of OGC API — Common — Part 1: Core (OGC 19-072).

RECOMMENDATION 11

IDENTIFIER `/rec/scaling/max-width-height`

A

The service metadata may include a maximum width, maximum height and maximum product of both indicating what are the maximum values that the server will accept to create the map. Clients should not request a map subset with a width or a height greater than these limits and if they do, they should expect that the server will not be able to create a map. These characteristics are defined as (shown as an OpenAPI Specification 3.0 fragment):

```
  x-OGC-limits:
    type: object
    properties:
      maps:
        type: object
        properties:
          maxWidth:
            type: integer
            description:
              Maximum width parameter value. If absent the server imposes no
limit.
            example: 2048
          maxHeight:
            type: integer
            description:
              Maximum height parameter value. If absent the server imposes no
limit.
            example: 2048
```

RECOMMENDATION 11

```
            maxPixels:
              type: integer
              description:
                Maximum  product of width and height parameter values. If absent
the server imposes no limit.
              example: 4194304
```

B

A Maps API implementation instance may specify the maximum width and height by adding a `maximum` property to the `width` and `height` parameter definition in the OpenAPI definition document as in this OpenAPI Specification 3.0 fragment:

```
  width:
    name: width
    in: query
    description:
      Width of the viewport to present the response in pixel units (the map
subset).
    required: false
    style: form
    explode: false
    schema:
      type: number
      maximum: 2048
  height:
    name: height
    in: query
    description:
      Height of the viewport to present the response in pixel units (the map
subset).
    required: false
    style: form
    explode: false
    schema:
      type: number
      maximum: 2048
```

C

In the absence of the `maxWidth` or `maxPixels`, the client will assume that there is no limit in the parameter `width`. In the absence of the `maxHeight` or `maxPixels`, the client will assume that there is no limit in the parameter `height`.

NOTE 1: This mimics the `maxWidth` and `maxHeight` elements in the WMS 1.3 *GetCapabilities* document.

NOTE 2: Service metadata can be embedded in the info section of the OpenAPI instance or in an independent document linked to the landing page with the rel="service-meta"

NOTE 3: The attributes `maxWidth`, `maxHeight` and `maxPixels` are intended to limit server workload, by providing limitations in the size of the output. If the `width` and `height` limits are defined in relation to the size of a common device screen, Maps API implementations should consider that new devices are being built with more and more pixels and a reasonable limit from the current year may be too restrictive for devices emerging in the next year.

Example — Example of the OpenAPI service metadata section

```
info:
  title: My Web API
  version: 1.0.0
  description: This example shows population of an OpenAPI Info element with
identifying
    metadata for both the service and the service provider.
```

```
x-OGC-limits:
  maps:
    maxWidth: 2048
    maxHeight: 2048
    maxPixels: 10000000
```

12

REQUIREMENTS CLASS "DISPLAY RESOLUTION"

12 REQUIREMENTS CLASS "DISPLAY RESOLUTION"

12.1. Overview

REQUIREMENTS CLASS 6: REQUIREMENTS CLASS 'DISPLAY RESOLUTION'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/display-resolution`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.6: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/display-resolution`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 16: `/req/display-resolution/mm-per-pixel-definition` Requirement 17: `/req/display-resolution/map-success`

The Display Resolution requirements class describes how to specify an accurate physical size of a pixel for the target display device or medium with a `mm-per-pixel` parameter. This is instead of a default 0.28 mm/pixel and allows applying symbology rules correctly. The physical pixel size is considered for establishing the relationship between the scale of the map and its output dimensions in pixels.

12.2. Map Operation

The Display Resolution requirements class specifies the mm-per-pixel parameter to accurately interpret scale denominators and symbology rules.

12.2.1. Parameter `mm-per-pixel`

By themselves, the width and height of the image to be presented on the display device are not enough to determine the scale of a map. This is because these values are provided in pixel

units. Conversely, the `scale-denominator` does not specify the size in pixels of a map for a given spatial area. The scale, being the ratio between the size of features as represented on the map and the physical size of the same features, also depends on the size of display device pixels in physical units. Knowing the size of these pixels supports accurately establishing the physical size of features as they will be presented on the display device and therefore compute the scale correctly by dividing the actual size of the display device in physical units (e.g., millimeters) by the actual size of the extent of the map (e.g., in meters). When this parameter is not specified, a default 0.28 mm/pixel is assumed (as standardized in WMS).

REQUIREMENT 16

IDENTIFIER	`/req/display-resolution/mm-per-pixel-definition`
INCLUDED IN	Requirements class 6: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/display-resolution`

A	The map operation SHALL support a parameter `mm-per-pixel` with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`mm-per-pixel:`
	`  name: mm-per-pixel`
	`  in: query`
	`  description:`
	`  size (in millimeters) of a pixel of the rendering device that presents the response.`
	`  required: false`
	`  style: form`
	`  schema:`
	`  type: number`

B	The implementation SHALL interpret `mm-per-pixel` as the size (in millimeters) of a rendering device pixel.
C	An HTTP 4xx error SHALL be returned if the `mm-per-pixel` value is not a positive number.
D	If the parameter `mm-per-pixel` is not provided, the server SHALL assume that the pixel size is 0.28 millimeters (90.71 pixels per inch).

NOTE 1: The definition of 0.28 as a default value is the same value as used in the Web Map Service (WMS) 1.3.0 Standard (OGC 06-042) and in the Symbology Encoding (SE) Implementation Specification 1.1.0 (OGC 05-077r4) that was later adopted by WMTS 1.0 (OGC 07-057r7) and 2D-TMS OGC (17-083r4). 0.28 mm was the actual pixel size of a common display from 2005. This value is still being used as reference, even if current display devices are built with much smaller pixel sizes.

NOTE 2: Since the 1980s, the Microsoft Windows operating system has set its default standard display pixels per inch (PPI) to 96. This value results in an approximate value of 0.264 mm per pixel. The similarity of this value with the actual 0.28 mm adopted in this Standard can create some confusion.

NOTE 3: Modern display devices (screens) have pixels so small that operating systems allow for defining a presentation scale factor bigger than one (e.g. 150%). In these circumstances, the actual size of the device pixels is not the same as the size used by the operating system.

With this parameter, the server has information on the physical size of the map as presented on a display device or printed and can therefore apply styling and symbology rules accordingly. For example, if the server receives a very small value, the server could decide to represent linear elements wider than for normal values, preventing the creation of a representation of linear features that is going to result in lines that are too thin to be correctly perceived.

12.2.2. Response

A successful GET response is described in the core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core)

REQUIREMENT 17	
IDENTIFIER	`/req/display-resolution/map-success`
INCLUDED IN	Requirements class 6: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/display-resolution`
A	For an implementation supporting the Maps API *scaling* requirements class, the implementation SHALL use the `mm-per-pixel` value instead of the default 0.28 mm/pixel when establishing the relationship between the dimensions of the output image, the scale and the spatial extent of the map.
B	The `mm-per-pixel` value SHALL be used instead of the default 0.28 mm/pixel when establishing scale for the purpose of applying styling and symbology rules to the map. For example, this needs to be considered for scale-dependent rule selectors as well as for graphical units in real world units (e.g., meters) or display units (e.g., millimeters).

12.2.3. Error conditions

A general summary of the HTTP status codes can be found in OGC API — Features — Part 1: Core, version 1.0 as well as in OGC API — Common — Part 1: Core Standard.

If the values of the `mm-per-pixel` parameters are out-of-range, the status code of the response will be `400`.

13

REQUIREMENTS CLASS "SPATIAL SUBSETTING"

13 REQUIREMENTS CLASS "SPATIAL SUBSETTING"

13.1. Overview

REQUIREMENTS CLASS 7: REQUIREMENTS CLASS 'SPATIAL SUBSETTING'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/spatial-subsetting`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 18: `/req/spatial-subsetting/bbox-crs` Requirement 19: `/req/spatial-subsetting/subset-crs` Requirement 20: `/req/spatial-subsetting/center-crs` Requirement 21: `/req/spatial-subsetting/bbox-definition` Requirement 22: `/req/spatial-subsetting/subset-definition` Requirement 23: `/req/spatial-subsetting/subset-response` Requirement 24: `/req/spatial-subsetting/center-definition` Requirement 25: `/req/spatial-subsetting/width-height` Requirement 26: `/req/spatial-subsetting/map-success`

The Maps API core requirements class defines the map resource. The Maps API core also defines a minimum set of metadata that enables a client application to formulate a map request. The Subsetting requirements class adds the ability to request an arbitrary 2D or 3D spatial subset of a map using either a bounding box specified using one of two equivalent syntaxes: a `bbox` parameter similar to the one in WMS 1.3, or a `subset` parameter similar to WCS 2.1. An alternative possibility is to specify a center point as well as `width` and/or `height` parameters taking into account the scale and the display resolution.

How the third (vertical) spatial dimension is applied to filter the output is left to the implementation and may depend on the nature of the data being represented and/or the negotiated output format. For example, the server could include features located within a vertical interval as part of a 2D map being rendered. In a second example, the extent of a 3D perspective view or a 3D model output would include only the subset of the vertical dimension specified. In a third example, a vertical profile could be returned for a single point or for a path

across the other spatial dimensions. In this last example, the subset mechanism would likely only be used to subset the vertical dimension, while another mechanism would be used to select an arbitrary path in the horizontal dimensions (this requirements class does not specify how to provide a path in a map request).

13.2. CRS for subsetting

In WMS 1.3 the CRS parameters had two purposes: Indicate the CRS of the data rendered in the map and the CRS of the coordinates of the `bbox` parameter. In the Maps API Standard, these two objectives have been separated. This section defines the CRS parameters for subsetting.

In this document the CRS for subsetting can have 3 possible values:

- https://www.opengis.net/def/crs/OGC/1.3/CRS84
- `storageCrs` indicated in the description of the geospatial resource, such as the collection or dataset.
- the one indicated in the `crs` parameter (the one picked for the map response). This possibility is described in the "CRS" requirements class.

The CRS advertised in the collection's spatial extent is always in https://www.opengis.net/def/crs/OGC/1.3/CRS84 (for Earth centric data and as required by *OGC API — Features*). At least for *OGC API — Maps* and *OGC API — Coverages*, a native CRS (also called storage CRS) is defined. The `Content-Crs:` header (see Overview) specifies the CRS used in the response to avoid confusion.

NOTE 1: There is no obligation to provide the maps in https://www.opengis.net/def/crs/OGC/1.3/CRS84.

NOTE 2: The collection or dataset description will contain a `storageCrs` property indicating the native CRS. If it is not present, then https://www.opengis.net/def/crs/OGC/1.3/CRS84 is assumed. This is specified in the *Collection Map* and *Dataset Map* requirements classes.

NOTE 3: Providing the extent of the collection or dataset in its native CRS (*storage CRS*) is also required, as specified in the *Collection Map* and *Dataset Map* requirements classes.

NOTE 4: A list of supported CRSs will be provided by the collection or dataset, as specified in the *Collection Map* and *Dataset Map* requirements classes. The first CRS should be the same as the native CRS (`storageCrs`).

13.2.1. Parameter `bbox-crs`

REQUIREMENT 18	
IDENTIFIER	/req/spatial-subsetting/bbox-crs
INCLUDED IN	Requirements class 7: https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting

A	The map retrieval operation SHALL support a query parameter bbox-crs with the characteristics defined in the OpenAPI Specification 3.0 fragment
	`bbox-crs:`
	`  name: bbox-crs`
	`  in: query`
	`  description: A URI (or safe CURIE) of the coordinate reference system for the coordinates specified in the `bbox` parameter. The valid values are [OGC:CRS84], the native (storage) CRS (if different), or the output `crs` (if specified).`
	`  required: false`
	`  schema:`
	`  type: string`
	`  example: https://www.opengis.net/def/crs/OGC/1.3/CRS84`

B	For Earth centric data, the implementation SHALL support https://www.opengis.net/def/crs/OGC/1.3/CRS84 as a value.
C	If the bbox-crs is not indicated https://www.opengis.net/def/crs/OGC/1.3/CRS84 SHALL be assumed.
D	If the storage (native) CRS is known, the storage CRS as a value SHALL be supported. Other conformance classes may allow additional values (see crs parameter definition).
E	The CRS expressed as URIs or as safe CURIEs SHALL be supported.
F	If the bbox parameter is not used, the bbox-crs SHALL be ignored.

13.2.2. Parameter `subset-crs`

REQUIREMENT 19	
IDENTIFIER	/req/spatial-subsetting/subset-crs
INCLUDED IN	Requirements class 7: https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting

A	The map operation SHALL support a parameter subset-crs with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`crs:`
	`  name: subset-crs`
	`  in: query`
	`  description: A URI (or safe CURIE) of the coordinate reference system for the coordinates specified in the `subset` parameter. The valid values are [OGC:CRS84], the native (storage) CRS (if different), or the output `crs` (if specified).`
	`  required: false`
	`  schema:`

REQUIREMENT 19	
	`type: string` `example: https://www.opengis.net/def/crs/OGC/1.3/CRS84`
B	For Earth centric data, https://www.opengis.net/def/crs/OGC/1.3/CRS84 SHALL be supported as a value.
C	If the `subset-crs` is not indicated https://www.opengis.net/def/crs/OGC/1.3/CRS84 SHALL be assumed.
D	If the storage (native) CRS is known, the storage CRS SHALL be supported as a value. Other requirements classes may allow additional values (see crs parameter definition).
E	CRS expressed as URIs or as safe CURIEs SHALL be supported.
F	If no `subset` parameter referring to an axis of the CRS is used, the `subset-crs` SHALL be ignored.

13.2.3. Parameter `center-crs`

REQUIREMENT 20	
IDENTIFIER	`/req/spatial-subsetting/center-crs`
INCLUDED IN	Requirements class 7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`

A	The map retrieval operation SHALL support a parameter `center-crs` with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`crs:`
	`  name: center-crs`
	`  in: query`
	`  description: A URI (or safe CURIE) of the coordinate reference system for the coordinates specified in the `center` parameter. The valid values are [OGC:CRS84], the native (storage) CRS (if different), or the output `crs` (if specified).`
	`  required: false`
	`  schema:`
	`  type: string`
	`  example: https://www.opengis.net/def/crs/OGC/1.3/CRS84`

B	For Earth centric data, https://www.opengis.net/def/crs/OGC/1.3/CRS84 SHALL be supported as a value.
C	If the `center-crs` is not used, https://www.opengis.net/def/crs/OGC/1.3/CRS84 SHALL be assumed.
D	If the storage (native) CRS is known, the storage CRS SHALL be supported as a value. Other requirements classes may allow additional values (see `crs` parameter definition).
E	CRS expressed as URIs or as safe CURIEs SHALL be supported.

REQUIREMENT 20	
F	If no `center` parameter is used, the `center-crs` SHALL be ignored.

NOTE 1: For clarification, the default CRS of the `bbox`, `center` and `subset` spatial subsetting parameters is `https://www.opengis.net/def/crs/OGC/1.3/CRS84` and the default output CRS of the map is its native (storage) CRS.

NOTE 2: The `center-crs`, `subset-crs` and `bbox-crs` are defined identically except they are affecting `center`, `subset` and `bbox` parameters respectively.

13.2.4. CURIE permission

PERMISSION 3	
IDENTIFIER	`/per/spatial-subsetting/crs-curie`
A	For the `center-crs`, `bbox-crs` and `subset-crs` query parameters, an unsafe CURIE without square brackets MAY be supported by the implementation.

NOTE: This makes the notation compatible with WMS.

13.3. Subsetting coordinates

13.3.1. Parameter `bbox`

The `bbox` parameter defines how to subset a map using a bounding box (in `bbox-crs` coordinates). This parameter selects a spatial region consisting of any pixels of the map whose area is contained or intersects with the specified bounding box. Individual pixels (including the ones in the four corners of the map) occupy an area on the ground (unlike some coverage whose cell values represent an infinitely small point). The bounding box describing the spatial content of the map (e.g., in the collection description) will go around the "outside" of the pixels of the map, rather than through the centers of the map's border pixels. This is also true for the description of a coverage whose cells represent a value for an area, which may be available as both *Maps* as well as through *OGC API — Coverages*, allowing to offer them both from the same OGC API collection. For example, a global dataset will always be described using -180 to 180 longitude bounds.

REQUIREMENT 21

IDENTIFIER	`/req/spatial-subsetting/bbox-definition`
INCLUDED IN	Requirements class 7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`

A	The map operation SHALL support a parameter `bbox` with a comma-separated list of four or six floating point numbers.
	If the bounding box consists of six numbers, the first three numbers are the coordinates of the lower bound corner of a three-dimensional bounding box and the last three are the coordinates of the upper bound corner. The axis order is determined by the `bbox-crs` parameter value or longitude and latitude if the parameter is missing (https://www.opengis.net/def/crs/OGC/1.3/CRS84 axis order for a 2D bounding box, https://www.opengis.net/def/crs/OGC/1.3/CRS84h for a 3D bounding box). For example in https://www.opengis.net/def/crs/OGC/1.3/CRS84 the order is left_long, lower_lat, right_long, upper_lat.
	with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`bbox:`
	`  name: bbox`
	`  in: query`
	`  description:`
	`  Bounding box of the rendered map. The bounding box is provided as four or six coordinates`
	`  * Lower left corner, coordinate axis 1`
	`  * Lower left corner, coordinate axis 2`
	`  * Minimum value, coordinate axis 3 (optional)`
	`  * Upper right corner, coordinate axis 1`
	`  * Upper right corner, coordinate axis 2`
	`  * Maximum value, coordinate axis 3 (optional)`
	`  The coordinate reference system and axis order of the values are indicated in the `bbox-crs` parameter or if the parameter is missing in https://www.opengis.net/def/crs/OGC/1.3/CRS84`
	`  required: false`
	`  schema:`
	`  type: array`
	`  oneOf:`
	`  - minItems: 4`
	`  maxItems: 4`
	`  - minItems: 6`
	`  maxItems: 6`
	`  items:`
	`  type: number`
	`  format: double`
	`  style: form`
	`  explode: false`

B	If the `bbox` parameter is used together with the `center` and/or with a `subset` parameter including any of the dimensions corresponding to those of the map bounding box, the server SHALL return a 4xx client error.

NOTE: The `bbox` uses the comma (“,”) as the separator between the coordinates. Additional white space will not be used to delimit list items. The comma character should not be escaped (IETF RFC 2396).

Example: `bbox=-180,-90,180,90&bbox-crs=[OGC:CRS84]`

Example of a 3D bounding box: `bbox=-180,-90,10000,180,90,12000&bbox-crs=[OGC:CRS84h]`

13.3.2. Parameter `subset`

The `subset` parameter is a common building block used to select a subset of a geospatial resource shared with other OGC API Standards. This can be specified as an interval between lower and higher bound values or as a single value. An example of an interval is "between 10.0 and 20.0 degrees of latitude" `subset=Lat(10:20)`. An example of single value is "150 meters above mean sea level" `subset=h(150)`. The dimensions that can be used within the value specified for this parameter, in the context of this particular requirements class, are the abbreviations for the spatial axes specified in the CRS definition.

The `subset` parameter is defined as follows:

REQUIREMENT 22

IDENTIFIER	`/req/spatial-subsetting/subset-definition`
INCLUDED IN	Requirements class 7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`

A	The implementation SHALL support a `subset` parameter conforming to the following Augmented Backus Naur Form (ABNF) fragment:
	`SubsetSpec = "subset" "=" axisName "(" intervalOrSingle ")"`
	`intervalOrSingle = interval / single`
	`interval = low ":" high`
	`low = single`
	`high = single`
	`single = number`
	
	`Where:`
	`` `axisName` is an abbreviation identifying an axis starting by a lowercase or uppercase letter A-Z, optionally followed by any number of alphanumeric characters ``
	`` `number` is an integer or floating-point number. ``

B	Axis names `Lat` and `Lon` SHALL be supported for geographic CRS and `E` and `N` for projected CRS, which are to be interpreted as the best matching spatial axis in the CRS definition.
C	If a third spatial dimension is supported (if the resource's spatial extent bounding box is three dimensional), the implementation SHALL also support a `h` dimension. This is the elevation above the ellipsoid in EPSG:4979 or CRS84h for geographic CRS and `z` for projected CRS, which are to be interpreted as the vertical axis in the CRS definition.
D	A 4xx error status code SHALL be returned if an axis name in the subset parameter value does not correspond to one of the axes of the subsetting CRS [2].
E	If the *interval* values fall entirely outside the range of valid values defined for the identified axis, a 204 or 404 status code SHALL be returned.
F	For a CRS where an axis can wrap around, such as subsetting across the dateline (anti-meridian) in a geographic CRS, a *low* value greater than *high* SHALL be supported to indicate an extent crossing that wrapping point.

REQUIREMENT 22	
G	Coordinates SHALL be interpreted as values for the named axis of the CRS specified in the `subset-crs` parameter value or in https://www.opengis.net/def/crs/OGC/1.3/CRS84 (https://www.opengis.net/def/crs/OGC/1.3/CRS84h for vertical dimension) if the `subset-crs` parameter is missing.
H	If the `subset` parameter including any of the dimensions corresponding to those of the map bounding box is used with a `bbox` and/or `center` parameter, the server SHALL return a 4xx error.
I	Multiple `subset` parameters SHALL be interpreted, as if all dimension subsetting values were provided in a single `subset` parameter (comma separated)[1]. [1] Example: `subset=Lat(-90:90)&subset=Lon(-180:180)` is equivalent to `subset=Lat(-90:90),Lon(-180:180)` [2] Note that this is only valid of the spatial dimensions. The 'additional' dimensions rely on the names of the extent of the collection.

NOTE 1: A subset parameter for https://www.opengis.net/def/crs/OGC/1.3/CRS84 will read as subset=Lon(left_lon:right_lon),Lat(lower_lat:upper_lat).

NOTE 2: When the *interval* values fall partially outside of the range of valid values defined by the CRS for the identified axis, the service is expected to return the non-empty portion of the resource resulting from the subset.

NOTE 3: For the operation of returning a map, there normally is no value in preserving dimensionality, therefore a *slicing* operation (using the *point* notation) is usually equivalent to a *trimming* operation (using the *interval* notation) when the low and high bounds of an interval are the same. Therefore, use of the point notation is encouraged in these cases.

NOTE 4: For typical 2D map requests, a trimming operation (interval) is always used (never a single slicing value) for the dimensions corresponding to the latitude (or northing) and longitude (or easting) dimensions, while a slicing or trimming operation for a third vertical dimension may be provided, where that third dimension is flattened on the map. For a response format where the content can actually be in three dimensions, an interval for the third dimension would not need to be flattened. In the case of more specialized maps, such as a vertical profile, or a temporal graph for a vegetation index at a particular spatial location, a slicing operation with single values might be appropriate for all spatial dimensions.

While the processing of the `subset` parameter is specific to the resource and operation for which it is applied, there is a general set of requirements which all implementations must address.

REQUIREMENT 23	
IDENTIFIER	`/req/spatial-subsetting/subset-response`
INCLUDED IN	Requirements class 7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`

REQUIREMENT 23	
A	Only the part of the resource that falls within the bounds of the subset interval and/or corresponds to the single point value SHALL be returned.

RECOMMENDATION 12	
IDENTIFIER	`/rec/spatial-subsetting/subset-crs-axis-names`
A	The names of the axis SHOULD be the abbreviated names of the axis in the CRS definition (e.g. the ones defined in the EPSG database).
B	'e' (in lowercase), 'X' (lowercase/uppercase) or 'Easting' (lowercase/uppercase) SHOULD be interpreted as synonymous of 'E'.
C	'n' (in lowercase) or 'Y' (lowercase/uppercase) or 'Northing' (lowercase/uppercase) SHOULD be interpreted as synonymous of 'N'.
D	'Long' (lowercase/uppercase) or 'Longitude' SHOULD be interpreted as synonymous of 'Lon'.
E	'Latitude' SHOULD be interpreted as synonymous of 'Lat'.

13.3.3. Parameter `center`, `width` and `height`

The `center` parameter defines the center point of the map in the `center-crs` coordinates from which the `width` and `height` parameters will define a subset, taking into consideration the scale and display resolution of the map.

REQUIREMENT 24	
IDENTIFIER	`/req/spatial-subsetting/center-definition`
INCLUDED IN	Requirements class 7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`
A	A `center` parameter SHALL be supported to specify the center of the subset of the map to include, with coordinates in the CRS specified in the `center-crs` parameter value or in https://www.opengis.net/def/crs/OGC/1.3/CRS84 if the `center-crs` parameter is missing.
B	If the `center` parameter is used together with the `bbox` and/or with a `subset` parameter including any of the dimensions corresponding to those of the map bounding box, the server SHALL return a 4xx client error.

NOTE 1: This parameter uses the comma (",") as the separator between the coordinates. Additional white space will not be used to delimit list items. The comma character should not be escaped (IETF RFC 2396).

REQUIREMENT 25	
IDENTIFIER	`/req/spatial-subsetting/width-height`
INCLUDED IN	Requirements class 7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`
A	When the `center` parameter and/or the `scale-denominator` parameter is used, or if the *scaling* conformance class is not supported, a `width` and `height` parameter specifying the subset of the map to return around the specified or default center of the map SHALL be supported.
B	The scale of the map SHALL be considered whether returning the map at a native scale or resampled (e.g., using the *scaling* conformance class `scale-denominator` parameter), as well as the display resolution (either the default 0.28 mm/pixel, or the one specified by the `mm-per-pixel` parameter of the *display resolution* conformance class).

NOTE 2: Although the `scale-denominator` parameter is defined in the "Scaling" requirements class and not in "Spatial Subsetting", implementations supporting only Spatial Subsetting are allowed (but not required) to recognize the parameter to compute `width` and `height` dimensions accordingly, together with a corresponding bounding box.

13.3.4. Scale and aspect ratio considerations when subsetting

For implementations not implementing "Scaling", the map is assumed to always be returned with the same fixed aspect ratio in both its native CRS unit space and pixel space, and the same CRS unit to pixel unit scale regardless of the subset being requested.

For implementations implementing both "Scaling" and "Spatial Subsetting" requirements classes, the scaling and subsetting parameters interact somewhat intricately. Guidance for implementers is provided in a section of the overview on these interactions, including the specific combinations of parameters that can be provided based on which requirements classes are implemented, and which parameters assume default values or should be computed in each case.

The "Scaling" requirements class also includes specific guidance for scale and aspect ratio considerations.

See also examples in an annex of computations to infer dimensions and infer bounding boxes based on specified parameters.

13.4. Response

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core).

REQUIREMENT 26	
IDENTIFIER	`/req/spatial-subsetting/map-success`
INCLUDED IN	Requirements class 7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`
A	The content of the response SHALL represent elements inside or intersecting with the spatial extent of the geographical area of the map identified with the subsetting coordinates.

Normally, the content partially outside the map bounding box will be clipped at the extent of the bounding box. This can be done efficiently when map subsets are in raster format (e.g., map tiles). However, maps containing features in vector format may not clip features that are partially outside to ensure continuity of features or for performance.

RECOMMENDATION 13	
IDENTIFIER	`/rec/spatial-subsetting/map-outside-bounds`
A	The server should try to honor the subsetting coordinates requested. For example, if the server has no data in the requested area, a map filled with the background color or transparent values based on the relevant query parameters specified should be returned. For example, if the subsetting coordinates requested are partially outside the server data, the area with no data SHOULD be filled with the background color or transparent values based on the relevant query parameters specified.

PERMISSION 4	
IDENTIFIER	`/per/spatial-subsetting/map-outside-bounds`
A	When the requested subset is outside the bounds of the requested CRS, the server MAY limit the Content-Bbox to the valid values in the CRS.

NOTE: As defined in the Maps API core requirements class, the `Content-Bbox:` and the `Content-Crs:` response headers report the actual bounding box of the data returned.

13.5. Error conditions

A general summary of the HTTP status codes can be found in OGC API — Features — Part 1: Core, version 1.0 as well as in OGC API — Common.

If the CRS in the parameter value `bbox-crs`, `subset-crs` or `center-crs` is not supported by the server for this resource, or the parameter value is out-of-range, the status code of the response will be `400`. If the map is not provided due to lack of data in the area, the status code of the response will be `204` or `404`.

14

REQUIREMENTS CLASS "DATE AND TIME"

14 REQUIREMENTS CLASS "DATE AND TIME"

14.1. Overview

REQUIREMENTS CLASS 8: REQUIREMENTS CLASS 'DATE AND TIME'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.8: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/datetime`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 27: `/req/datetime/datetime-definition` Requirement 28: `/req/datetime/datetime-response` Requirement 29: `/req/datetime/subset-definition` Requirement 30: `/req/datetime/subset-response` Requirement 31: `/req/datetime/axis` Requirement 32: `/req/datetime/map-success`

The Date and Time Subsetting requirements class defines the way date and time can be used as a parameter to filter the content in the map resource.

14.2. Describing the temporal extent

Depending on the type of resource, the way the temporal extent and the resolution of the datetime values that are available for the client to request are described may be different:

- For Collection maps, the collection description should specify the temporal extent of the resources. Maps can be requested inside this extent. If the extent is specified in a way that instant values are provided (e.g., by listing them or by including a resolution) then it should be possible to request maps for these instants.

- For Dataset maps, the landing page should specify the temporal extent of the dataset. Maps can be requested inside this extent.

14.3. datetime query parameter request and response

The `datetime` parameter is a common building block that is shared with other OGC API Standards.

In the context of this requirements class, the `datetime` parameter selects a time instant or interval of the resource for which a map is requested.

The `datetime` parameter is defined as follows:

REQUIREMENT 27	
IDENTIFIER	`/req/datetime/datetime-definition`
INCLUDED IN	Requirements class 8: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime`

A	The `datetime` parameter SHALL have the following characteristics (using an OpenAPI Specification 3.0 fragment):
	`name: datetime`
	`in: query`
	`required: false`
	`schema:`
	`  type: string`
	`style: form`
	`explode: false`

B	Temporal geometries are either a date-time value or a time interval. The parameter value SHALL conform to the following syntax (using ABNF):
	`interval-bounded = instant "/" instant`
	`interval-bounded-start = [".."] "/" instant`
	`interval-bounded-end = instant "/" [".."]`
	`interval-unbounded = [".."] "/" [".."]`
	`interval = interval-bounded / interval-bounded-start / interval-bounded-end / interval-unbounded`
	`datetime = instant / interval`

C	The implementation SHALL support an `instant` defined as specified by RFC 3339, 5.6, with the exception that the server is only required to support the `Z` UTC time notation, and not required to support local time offsets.
D	Time intervals unbounded at the start or at the end SHALL be supported using a double-dot (`..`) or an empty string for the start/end.

While the processing of the `datetime` parameter is specific to the resource and operation for which it is applied, there is a general set of requirements which all implementations must address.

REQUIREMENT 28	
IDENTIFIER	`/req/datetime/datetime-response`
INCLUDED IN	Requirements class 8: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime`
A	If the `datetime` parameter is provided by the client and supported by the server, then only resources that have a temporal geometry that intersects the temporal information in the `datetime` parameter SHALL be part of the result set. If a resource has multiple temporal properties, it is the decision of the server whether only a single temporal property is used to determine the extent or all relevant temporal properties.
B	The `datetime` parameter SHALL match all resources in the collection that are not associated with a temporal geometry.

"Intersects" means that the time (instant or period) specified in the parameter `datetime` includes a timestamp that is part of the temporal geometry of the resource (again, a time instant or period). For time periods this includes the start and end time.

NOTE: ISO 8601-2 (Date and time — Representations for information interchange — Part 2: Extensions) distinguishes unbounded start/end timestamps (double-dot) and unknown start/end timestamps (empty string). For queries, an unknown start/end has the same effect as an unbounded start/end.

Example 1 — A date-time: February 12, 2018, 23:20:52 UTC:
`datetime=2018-02-12T23:20:52Z`

For resources with a temporal property that is a timestamp (like `lastUpdate`), a date-time value would match all resources where the temporal property is identical.

For resources with a temporal property that is a date or a time interval, a date-time value would match all resources where the timestamp is on that day or within the time interval.

Example 2 — Intervals: February 12, 2018, 00:00:00 UTC to March 18, 2018, 12:31:12 UTC:
`datetime=2018-02-12T00:00:00Z/2018-03-18T12:31:12Z`

February 12, 2018, 00:00:00 UTC or later:

`datetime=2018-02-12T00:00:00Z/..`

March 18, 2018, 12:31:12 UTC or earlier:

`datetime=../2018-03-18T12:31:12Z`

A template for the definition of the `datetime` parameter is available at datetime.yaml.

14.4. subset=time query parameter request and response

The `subset` parameter is a common building block that is shared with other OGC API Standards. The `subset` parameter can be specified as an interval between lower and higher bound values or as a single value. An example of an interval is "between 1st of January 2021 and last day of May 2021" `subset=time("2021-01-01":"2021-05-31")`. An example of single value is "1st of January 2021 at 8am o'clock" `subset=time("2021-01-01T08:00:00")`. The `time` dimension can be used within the value specified for this parameter and should ideally match with the axis abbreviation specified in the CRS definition.

The `subset` parameter is defined as follows:

REQUIREMENT 29	
IDENTIFIER	`/req/datetime/subset-definition`
INCLUDED IN	Requirements class 8: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime`

A	A `subset` parameter with the following characteristics (using an Augmented Backus Naur Form (ABNF) fragment) SHALL be supported:
	`SubsetSpec = "subset" "=" "time" "(" intervalOrSingle ")"`
	`intervalOrSingle = interval / single`
	`interval = low ":" high`
	`low = single`
	`high = single`
	`single = number / text / "*"`
	
	Where:
	`number` is an integer or floating-point number, and
	`text` is some general ASCII text (such as a time and date notation in RFC 3339 section 5.6) enclosed in double quotes (`"`, ASCII code 0x42).
	An asterisk (`*`) can be used for `low` or `high` to indicate the minimum/maximum value.
	A single asterisk can be used to indicate the high value.
	Support for `*` is required for time, but optional for spatial and other dimensions.

B	The implementation SHALL support an axis name `time`.
C	The implementation SHALL return a 4xx status code if the axis name is not `time`, is not a recommended alias for time (`Time`, `t` and `T`), and is not recognized in the context of another requirements class, such as spatial and general subsetting.
D	If the `intervalOrSingle` values fall entirely outside the range of valid values defined for the identified axis, a 204 or 404 status code SHALL be returned

REQUIREMENT 29	
E	Coordinates SHALL be interpreted as values for the CRS specified in the temporal extent, or Gregorian UTC time if it is not specified in the temporal extent.
F	When Gregorian UTC time is used, the implementation SHALL support time expressed using RFC 3339 section 5.6, with only support for the UTC (`Z`) notation required (support for local time offsets is optional), as well as the following additional partial date and time formats: • only a year: `yyyy` • a year and a month: `yyyy-mm` • a year, a month and a day: `yyyy-mm-dd` • a year, a month, a day and an hour: `yyyy-mm-ddThhZ` • a year, a month, a day, an hour and a minute: `yyyy-mm-ddThh:mmZ`
G	The implementation SHALL support a `*` value indicating the earliest available time (for `low`) or the latest available time (for `high` and when used as a single time instant).
H	Multiple `subset` parameters SHALL be interpreted as if all dimension subsetting values were provided in a single `subset` parameter (comma separated)[1]. [1] Example: `subset=Lat(-90:90)&subset=Lon(-180:180)&subset=time("2018-02-12T23:20:52Z")` is equivalent to `subset=Lat(-90:90),Lon(-180:180),time("2018-02-12T23:20:52Z")`

NOTE 1: When the `intervalOrSingle` values fall partially outside of the range of valid values defined by the CRS for the identified axis, the service is expected to return the non-empty portion of the resource resulting from the subset.

NOTE 2: For the operation of returning a map, there normally is no value in preserving dimensionality, therefore a *slicing* operation (using the *point* notation) is usually equivalent to a *trimming* operation (using the *interval* notation) when the low and high bounds of an interval are the same. Therefore, use of the point notation is encouraged in these cases.

NOTE 3: For typical 2D map requests, a slicing or trimming operation for the temporal dimension may be provided, where that temporal dimension is flattened on the map. For a response format such as a video animation, the temporal dimension would not need to be flattened. For special maps like the graph of a vegetation index over time for a particular spatial location, the temporal axis may be presented as a vertical or horizontal axis on the map.

NOTE 4: If no subset parameter is used for the time dimension and no datetime parameter is used, the implementation is free to return a default temporal range.

While the processing of the `subset` parameter is specific to the resource and operation for which it is applied, there is a general set of requirements which all implementations must address.

REQUIREMENT 30	
IDENTIFIER	`/req/datetime/subset-response`

REQUIREMENT 30	
INCLUDED IN	Requirements class 8: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime`
A	Only that part of the resource that falls within the bounds of the subset expression SHALL be returned.

REQUIREMENT 31	
IDENTIFIER	`/req/datetime/axis`
INCLUDED IN	Requirements class 8: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime`
A	To subset a generic time dimension, the server SHALL support “time” as axisname in the subset parameter

Example 1 — A date-time (subset): February 12, 2018, 23:20:52 UTC:

```
subset=time("2018-02-12T23:20:52Z")
```

For resources with a temporal property that is a timestamp (such as `lastUpdate`), a date-time value would match all resources where the temporal property is identical.

For resources with a temporal property that is a date or a time interval, a date-time value would match all resources where the timestamp is on that day or within the time interval.

Example 2 — Intervals (subset): February 12, 2018, 00:00:00 UTC to March 18, 2018, 12:31:12 UTC:

```
subset=time("2018-02-12T00:00:00Z":"2018-03-18T12:31:12Z")
```

February 12, 2018, 00:00:00 UTC or later:

```
subset=time("2018-02-12T00:00:00Z":*)
```

March 18, 2018, 12:31:12 UTC or earlier:

```
subset=time(*:"2018-03-18T12:31:12Z")
```

14.5. Actual date & time response header

RECOMMENDATION 14	
IDENTIFIER	`/rec/datetime/actual-datetime`

A	The server SHOULD add an HTTP header with `OGCAPI-datetime` as a name and a temporal geometry as a value, to indicate the instant or the temporal interval of the content of the resource. The temporal geometries value shall conform to the following syntax (using ABNF):
	`interval = instant "/" instant`
	`datetime = instant / interval`

STATEMENT	The syntax of `instant` is specified by RFC 3339, 5.6.

14.6. Closest date & time permission

PERMISSION 5	
IDENTIFIER	`/per/datetime/closest`
A	In case the requested map is not available in the exact requested datetime the closest or last previous time for which data is available MAY be returned by the server.

NOTE: An Earth Observation use case where this permission is useful is to allow retrieving a map of the last datetime where imagery is available, considering that a certain geographic area may only be observed at an interval of "every few days" and availability may be irregular and conditioned by clouds.

14.7. Response

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core).

REQUIREMENT 32	
IDENTIFIER	`/req/datetime/map-success`
INCLUDED IN	Requirements class 8: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime`

REQUIREMENT 32	
A	The content of that response SHALL be consistent with the requested datetime.

14.8. Error conditions

A general summary of the HTTP status codes can be found in OGC API — Features — Part 1: Core, version 1.0 as well as in the OGC API – Common – Part 1: Core Standard.

If the parameter value for `datetime` is outside the domain of the collections, the status code of the response will be `400`.

If the request returns no data (e.g., in combination with other types of subsetting), the status code of the response will be `204` or `404`. If permission /per/datetime/closest is implemented, this situation is unlikely to occur.

15

REQUIREMENTS CLASS "GENERAL SUBSETTING"

15 REQUIREMENTS CLASS "GENERAL SUBSETTING"

15.1. Overview

REQUIREMENTS CLASS 9: REQUIREMENTS CLASS 'GENERAL SUBSETTING'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/general-subsetting`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.9: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/general-subsetting`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 33: `/req/general-subsetting/uniform-additional-dimensions` Requirement 34: `/req/general-subsetting/subset-definition`

The General Subsetting requirements class defines the way a subset of the map resource in additional dimensions beyond spatial and temporal can be specified using the `subset` parameter. This functionality is defined as common building blocks shared with other OGC API Standards.

15.2. Describing the extent of the additional dimensions

When implementing the General Subsetting requirements class, additional dimensions included in an extent description must use a uniform approach to describe the lower and upper bounds of those dimensions based on the `intervals` property. An example is found in an OGC API collection description response described in OGC API — Common — Part 2 and follows the JSON schema in https://schemas.opengis.net/ogcapi/maps/part1/1.0/openapi/schemas/common-geodata/collectionDesc.yaml. This is the same as with the temporal dimension.

REQUIREMENT 33	
IDENTIFIER	/req/general-subsetting/uniform-additional-dimensions
INCLUDED IN	Requirements class 9: https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/general-subsetting
A	Wherever applicable (e.g., the collection or dataset description), the extent of any additional dimension(s) beyond temporal and spatial SHALL be described in a way similar to the temporal dimension, using the name of the dimension as a key and an object as value, with that object containing an interval property, a URI for the semantic definition, a unit of measure if applicable, and a grid definition if applicable, as specified in *extent-uad.yaml*.

NOTE: The subset-crs mentioned in Spatial subsetting can only be used in combination with a spatial subset.

15.3. subset query parameter

The subset parameter can be specified as an interval between lower and higher bound values or as a single value. An example of an interval is "between 500.0 and 1013.0 hPa" subset=pressure(500:1013). An example of a single value is "750 hPa" subset=pressure(750). The dimensions that can be used within the value specified for this parameter are the axis abbreviations specified in the CRS definition. All additional dimensions listed in the extent as valid dimensions to be used with the subset parameter must be supported.

REQUIREMENT 34	
IDENTIFIER	/req/general-subsetting/subset-definition
INCLUDED IN	Requirements class 9: https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/general-subsetting

A	A subset parameter with the following characteristics (using an Augmented Backus Naur Form (ABNF) fragment) SHALL be supported:
	`SubsetSpec = "subset" "=" axisName "(" intervalOrSingle ")"`
	`intervalOrSingle = interval / single`
	`interval = low ":" high`
	`low = single`
	`high = single`
	`single = number / text / "*"`
	
	`Where:`
	`` `axisName` is an abbreviation identifying an axis starting by a lowercase or uppercase letter A-Z, optionally followed by any number of alphanumeric characters ``
	`` `number` is an integer or floating-point number, and ``
	`` `text` is some general ASCII text (such as a time and date notation in RFC 3339 section 5.6) enclosed in double-quotes (`"`, ASCII code 0x42). ``

REQUIREMENT 34	
	An asterisk (`*`) can be used for `low` or `high` to indicate the minimum/maximum value. For an additional temporal dimension, a single asterisk may be supported to indicate the latest available time. Support for `*` is required for time, but optional for spatial and other dimensions.
B	Any name of the *additional* dimensions in the extent of the collection SHALL be supported as an axis name.
C	A 4xx error status code SHALL be returned if an axis name does not correspond to the name of one of the *additional* dimensions in the extent of the collection, or recognized in the context of another requirements class.
D	If the *intervalOrSingle* values fall entirely outside the range of valid values defined for the identified axis, a 204 or 404 status code SHALL be returned.
E	For an axis that can wrap around, a *low* value greater than *high* SHALL be supported to indicate an extent crossing that wrapping point.
F	Multiple subset parameters SHALL be interpreted as if all dimension subsetting values were provided in a single `subset` parameter (comma separated)[1]. [1] Example: `subset=Lat(-90:90)&subset=Lon(-180:180)&subset=atm_pressure_hpa(500)` is equivalent to `subset=Lat(-90:90),Lon(-180:180),atm_pressure_hpa(500)`

NOTE 1: When the `intervalOrSingle` values fall partially outside of the range of valid values defined by the CRS for the identified axis, the service is expected to return the non-empty portion of the resource resulting from the subset.

NOTE 2: For the operation of returning a map, there normally is no value in preserving dimensionality. Therefore, a *slicing* operation (using the *point* notation) is usually equivalent to a *trimming* operation (using the *interval* notation) when the low and high bounds of an interval are the same. Therefore, use of the point notation is encouraged in these cases.

NOTE 3: For typical 2D map requests, a slicing or trimming operation for additional dimensions may be provided, where those additional dimensions are flattened on the map. For a response format where an additional dimension can be preserved (e.g., atmospheric pressure represented in a 3D representation), that additional dimension would not need to be flattened. For special maps like graphs, an additional dimension may be presented as a vertical or horizontal axis on the map.

15.4. Response

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core).

15.5. Error conditions

A general summary of the HTTP status codes can be found in the OGC API – Common – Part 1: Core Standard.

If the parameter value for the additional dimensions is outside the domain of the collections, the status code of the response will be `400`.

Otherwise, if the request returns no data (e.g., in combination with other types of subsetting), the status code of the response will be `204` or `404`.

16

REQUIREMENTS CLASS "COORDINATE REFERENCE SYSTEM"

16 REQUIREMENTS CLASS "COORDINATE REFERENCE SYSTEM"

16.1. Overview

REQUIREMENTS CLASS 10: REQUIREMENTS CLASS 'COORDINATE REFERENCE SYSTEM'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/crs`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.10: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/crs`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 35: `/req/crs/crs-definition` Requirement 36: `/req/crs/map-success`

In WMS 1.3 the CRS parameters had two purposes: To indicate the CRS of the data rendered in the map and the CRS of the coordinates of the BBOX parameter. In the Maps API Standard, these two purposes have been separated. This section describes how to request a map in a specific CRS.

For *OGC API — Maps* and *OGC API — Coverages*, a native CRS (also called a *storage CRS*) is defined. The `Content-Crs:` header (see Overview) specifies the CRS used in the response to avoid confusion.

16.2. Operation

The Maps API core requirements class defines how to retrieve a map. The CRS requirements class specifies parameters needed to retrieve a map in a particular output CRS.

16.2.1. Parameter `crs`

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core).

REQUIREMENT 35	
IDENTIFIER	`/req/crs/crs-definition`
INCLUDED IN	Requirements class 10: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/crs`

A	The map operation SHALL support a parameter `crs` with the characteristics defined in the OpenAPI Specification 3.0 fragment:
	`crs:`
	`  name: crs`
	`  in: query`
	`  description: A coordinate reference system of the map response. A list of all supported CRS values can be found under the collection metadata.`
	`  required: false`
	`  schema:`
	`  type: string`
	`  example: https://www.opengis.net/def/crs/OGC/1.3/CRS84`

B	Any of the CRSs listed in the collection (or collections) description SHALL be supported. If the list of supported CRS is not present, only https://www.opengis.net/def/crs/OGC/1.3/CRS84 SHALL be supported.
C	If the spatial subsetting requirements class is supported, the `bbox-crs` and the `subset-crs` SHALL additionally support the value specified in the `crs` parameter.
D	CRS expressed as URIs or as safe CURIEs SHALL be supported.

NOTE 1: When no `crs` parameters are specified, please refer to the Maps API core conformance class to know about the default CRS.

See how to determine the native (storage) CRS for the collection and the dataset in Requirements Class “Collection Map” and Requirements Class “Dataset Map” respectively.

NOTE 2: The default CRS of the BBOX is https://www.opengis.net/def/crs/OGC/1.3/CRS84 but the default CRS of the map is the native (storage) CRS

PERMISSION 6	
IDENTIFIER	`/per/crs/crs-curie`
A	For the `crs` parameter, unsafe CURIE without square brackets MAY be supported by the implementation.

NOTE 3: This makes the notation compatible with WMS.

16.2.2. Response

REQUIREMENT 36	
IDENTIFIER	`/req/crs/map-success`
INCLUDED IN	Requirements class 10: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/crs`
A	The content of the map response SHALL be consistent with the requested CRS.

16.2.3. Error conditions

A general summary of the HTTP status codes can be found in OGC API — Features — Part 1: Core, version 1.0 as well as in the OGC API – Common – Part 1: Core Standard.

If the parameter value for `crs` is not valid for the collections, the status code of the response will be `400`.

17

REQUIREMENTS CLASS "ORIENTATION"

17 REQUIREMENTS CLASS "ORIENTATION"

17.1. Overview

REQUIREMENTS CLASS 11: REQUIREMENTS CLASS 'ORIENTATION'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/orientation`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.11: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/orientation`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 37: `/req/orientation/orientation` Requirement 38: `/req/orientation/response-headers`

This requirements class defines the ability for clients to request a map whose content is rotated by a counterclockwise orientation specified in degrees, resulting in the viewing perspective being rotated by that same orientation in a clockwise direction.

17.2. Operation

17.2.1. Parameter `orientation`

REQUIREMENT 37	
IDENTIFIER	`/req/orientation/orientation`
INCLUDED IN	Requirements class 11: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/orientation`

REQUIREMENT 37	
A	If rotation of the map content is desired, an orientation parameter SHALL be supported that specifies the amount by which to rotate a map, expressed as counterclockwise degrees. This results in the viewing perspective being rotated by that same orientation in a clockwise direction.
B	If an `orientation` parameter is not specified, a zero-orientation value SHALL be assumed.
C	The orientation SHALL be applied to the map with the center of the selected spatial subset as the pivot point, or the center of the map if none is specified.
D	If an `orientation` parameter is used together with `subset` or `bbox` spatial subsetting parameter, the counterclockwise orientation SHALL be applied to the four corners of the clipping box associated to that subset, as if the equivalent `center`, `width` and `height` spatial subsetting query parameters were used instead. This avoids leaving empty corners in the final rotated map image.

17.2.2. Response headers

REQUIREMENT 38	
IDENTIFIER	`/req/orientation/response-headers`
INCLUDED IN	Requirements class 11: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/orientation`
A	For responses to a map request where the `orientation` query parameter was used, a response header `Content-Orientation: [value in decimal degrees]` corresponding to the orientation of the map SHALL be returned.
B	For responses to a map request where the `orientation` query parameter was used, the `Content-Bbox` response header SHALL reflect the bounding box in the map output CRS prior to the orientation being applied.

Example of a request and associated response headers:

Example 1 — Example of a request

```
/map?orientation=40
```

Example 2 — Example of associated response headers

```
Content-Crs: <https://www.opengis.net/def/crs/EPSG/0/3995>
Content-Orientation: 40
Content-Bbox: -10000,-4000,10000,4000
```

17.2.3. Response

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core).

17.2.4. Error conditions

A general summary of the HTTP status codes can be found in *OGC API — Features — Part 1: Core*, version 1.0 as well as in the OGC API – Common – Part 1: Core Standard.

If the parameter value for the orientation is not a valid orientation, the status code of the response will be `400`.

18

REQUIREMENTS CLASS "CUSTOM PROJECTION CRS"

18 REQUIREMENTS CLASS "CUSTOM PROJECTION CRS"

18.1. Overview

REQUIREMENTS CLASS 12: REQUIREMENTS CLASS 'CUSTOM PROJECTION CRS'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/projection`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 39: `/req/projection/crs-proj-method` Requirement 40: `/req/projection/crs-proj-params` Requirement 41: `/req/projection/crs-proj-center-definition` Requirement 42: `/req/projection/crs-datum` Requirement 43: `/req/projection/response-headers` Requirement 44: `/req/projection/projections-resource` Requirement 45: `/req/projection/projections-response`

The Custom Projection CRS requirements class defines the ability for clients to specify a custom projection CRS based on parameters defining a projection operation method, values for corresponding parameters, as well as a datum. This class also provides a parameter facilitating the task of selecting a projection center across different projections, for which the relevant latitude and longitude parameters differ. Angles are specified in degrees, whereas identifiers for methods, parameters and datums can be specified as URIs or safe CURIEs. Additionally, this requirements class defines a resource at `/projectionsAndDatums` listing the available operation methods and their associated parameters, as well as the possible datums that can be used as values for these parameters.

18.2. Operation

18.2.1. Parameter `crs-proj-method`

The operation method refers mainly to the equations that relate geographic coordinates to projected coordinates and vice-versa.

REQUIREMENT 39	
IDENTIFIER	`/req/projection/crs-proj-method`
INCLUDED IN	Requirements class 12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`
A	A `crs-proj-method` parameter supporting selection of a projection operation method SHALL be supported.
B	CURIEs in addition to URI to specify the projection method SHALL be supported.

18.2.2. Parameter `crs-proj-params`

The operation method may have some parameters, such as the standard parallel, to set. This query parameter supports setting those values. In some simple cases this query parameter might not be necessary, and clients can consider using `crs-proj-center` instead.

REQUIREMENT 40	
IDENTIFIER	`/req/projection/crs-proj-params`
INCLUDED IN	Requirements class 12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`
A	A `crs-proj-params` parameter SHALL be supported that enables selection of one or more values for operation method parameters, with values in between parentheses ( ) following the CURIE (or URI) of a projection parameter, and different parameters separated by value. For example, `crs-proj-params=[epsg-parameter:8823](40),[epsg-parameter:8824](90)`.
B	In addition to CURIEs, URIs to specify the projection parameters SHALL be supported.

18.2.3. Parameter `crs-proj-center`

To facilitate the task of centering a custom projection on the area of interest, with parameters varying greatly between projections, the `crs-proj-center` parameter automatically sets the most appropriate parameters for the specified center latitude and longitude.

REQUIREMENT 41	
IDENTIFIER	`/req/projection/crs-proj-center-definition`
INCLUDED IN	Requirements class 12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`
A	A `crs-proj-center` parameter of the form `Lat(centerLat),Lon(centerLon)` SHALL be supported to facilitate the selection of the most relevant projection parameters to center a custom projection.
B	The projection-center `Lat` value SHALL be mapped to the first matching operation method parameter available for the selected operation method of the projection query parameter, in this order: • [epsg-parameter:8832] Latitude of standard parallel (PROJ: *lat_ts*) • [epsg-parameter:8823] Latitude of first standard parallel, if only a single standard parallel can be set — i.e., no [epsg-parameter:8824] parameter is available (PROJ: *lat_ts*) • [epsg-parameter:8801] Latitude of natural origin (PROJ: *lat_0*) • [epsg-parameter:8811] Latitude of projection center (PROJ: *lat_0*)
C	The projection-center `Lon` value SHALL be mapped to the first matching operation method parameter available for the selected operation method of the projection query parameter, in this order: • [epsg-parameter:8802] Longitude of natural origin (PROJ: *lon_0*) • [epsg-parameter:8812] Longitude of projection center (PROJ: *lon_0*) • [epsg-parameter:8833] Longitude of origin (PROJ: *lon_0*)

NOTE: For some projections where there is not a clear single center latitude parameter, such as those with multiple standard parallels, clients should use `crs-proj-params` to unambiguously set each parameter instead.

18.2.4. Parameter `crs-datum`

REQUIREMENT 42	
IDENTIFIER	`/req/projection/crs-datum`
INCLUDED IN	Requirements class 12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`

REQUIREMENT 42	
A	A `crs-datum` parameter as a URI allowing to select a datum for the output CRS SHALL be supported.
B	CURIEs in addition to URI to specify the datum parameter SHALL be supported.
C	If a `crs-datum` parameter is not specified, the native (storage) CRS datum SHALL be assumed (the WGS84 ensemble `[epsg-datum:6326]` datum is assumed if the native CRS is not declared).

18.2.5. Response headers

REQUIREMENT 43	
IDENTIFIER	`/req/projection/response-headers`
INCLUDED IN	Requirements class 12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`
A	For responses to a map request where the `crs-proj-method` query parameter was used, a response header `Content-Crs-Method: <[URI]>` including the URI of the projection operation method SHALL be returned.
B	A response header `Content-Crs-Method-Params: <URI>=[value]; ...` SHALL be returned. This includes the URI of the projection operation parameters and its value for each parameter specified using the `crs-proj-method` or `crs-proj-center` query parameters.
C	For responses to a map request where the `crs-datum` query parameter was used, a response header `Content-Crs-Datum: <[URI]>` corresponding to the URI of the projection operation method SHALL be returned.
D	For responses to a map request specifying the `crs-proj-method` query parameter, a `Content-Crs` response header SHALL NOT be included.
E	For responses to a map request specifying the `crs-proj-method` query parameter, the CRS of the `Content-Bbox` response header coordinates SHALL be in the custom CRS defined by this operation method and its parameters.

Example of a request and associated response headers:

Example 1 — Example of a request

```
/map?
   crs-proj-method=[epsg-method:9840]&
   crs-proj-center=Lat(30),Lon(40)&
   crs-datum=[epsg-datum:6326]
```

Example 2 — Example of associated response headers

```
Content-Crs-Method: <https://www.opengis.net/def/method/EPSG/0/9840>
Content-Crs-Method-Params: <https://www.opengis.net/def/parameter/EPSG/0/8801>=
30; <https://www.opengis.net/def/parameter/EPSG/0/8802>=40
Content-Crs-Datum: <https://www.opengis.net/def/datum/EPSG/0/6326>
```

```
Content-Bbox: -1000,-800,1000,800
```

18.2.6. Response

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core).

18.2.7. Error conditions

A general summary of the HTTP status codes can be found in OGC API — Features — Part 1: Core, version 1.0 as well as in the OGC API – Common – Part 1: Core Standard.

If the parameter value for the parameters is not valid, the status code of the response will be `400`.

18.3. Available projections

18.3.1. Available projections operation

The resource at `/projectionsAndDatums` supports a `GET` operation, with JSON as an available representation, returning a list of the supported projection operation methods and their associated parameters, as well as a list of available datums, applying to all map retrieval operations of the API.

REQUIREMENT 44	
IDENTIFIER	`/req/projection/projections-resource`
INCLUDED IN	Requirements class 12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`
A	A GET operation at `/projectionsAndDatums` providing at minimum a JSON representation SHALL be supported.

18.3.2. Available projections response

The custom projections resource includes a `methods` property listing the available projections operation methods, corresponding to the valid values for the `crs-proj-method` query parameter,

and their associated parameters, corresponding to the valid values for the `crs-proj-params` parameters.

The custom projections resource also includes a `datums` property listing the available datums corresponding to the valid values for the `crs-datum` query parameter.

REQUIREMENT 45	
IDENTIFIER	`/req/projection/projections-response`
INCLUDED IN	Requirements class 12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`
A	The implementation SHALL include in its response for the `/projectionsAndDatums` resource the complete list of custom CRS projection operation methods supported for map retrieval operations.
B	The implementation SHALL include in its response for the `/projectionsAndDatums` resource the complete list of custom CRS datums supported for map retrieval operations.
C	In the JSON representation, the list of supported projection operation methods SHALL be provided as a dictionary (associative array) value for a `methods` property associating operation method objects (including optional `title` and `description` properties) to the corresponding identifiers to be used as values for the `crs-proj-method` query parameter.
D	These operation method identifiers SHALL be safe CURIEs when a registered URI exists for the method.
E	In the JSON representation, the list of supported datums SHALL be provided as a dictionary (associative array) value for a `datums` property associating datum objects to the corresponding identifiers to be used as values for the `crs-datum` query parameter.
F	These datum identifiers SHALL be safe CURIEs when a registered URI exists for the datum.
G	The datum object SHALL include an `ellipsoid` property specifying the safe CURIE for the associated ellipsoid and may contain additional optional `title` and `description` properties.
H	In the JSON representation, the operation method objects SHALL include all valid parameters for that method as a dictionary (associative array) value for a `parameters` property to method parameters object (including optional `title` and `description` properties) to the corresponding identifiers to be used as values for the `crs-proj-params` query parameter.
I	These method parameters SHALL be safe CURIEs when a registered URI exists for the parameter. To avoid repeating the same parameter, those objects may use a JSON pointer (`$ref`) to a top-level `parameters` property in the same custom projections JSON document.
J	In the JSON representation, the operation method objects SHALL include `centerLatParam` and/or `centerLonParam` properties (as applicable) whose values SHALL be the identifiers corresponding to the parameters for which values specified for the `crs-proj-center` query parameter will be mapped, in a manner consistent with requirement `/req/projection/crs-proj-center-definition`.

NOTE: Refer to the OpenAPI definition for the `/projectionsAndDatums` path resource for the schema corresponding to these requirements.

Example — Example JSON response for `/projectionsAndDatums` resource listing available projections, parameters and datums

```
{
   "methods":
   {
      "[epsg-method:9802]":
      {
         "title": "Lambert Conic Conformal",
         "description": "Lambert Conic Conformal projection (2 standard
parallels)",
         "parameters":
         {
            "[epsg-parameter:8823]": { "$ref": "#/parameters/[epsg-
parameter:8823]" },
            "[epsg-parameter:8824]": { "$ref": "#/parameters/[epsg-
parameter:8824]" },
         },
         "centerLatParam": "[epsg-parameter:8823]"
      },
      "[epsg-method:9805]":
      {
         "title": "Mercator",
         "description": "Mercator projection",
         "parameters":
         {
            "[epsg-parameter:8823]": { "$ref": "#/parameters/[epsg-
parameter:8823]" },
            "[epsg-parameter:8802]": { "$ref": "#/parameters/[epsg-
parameter:8802]" }
         },
         "centerLatParam": "[epsg-parameter:8823]",
         "centerLonParam": "[epsg-parameter:8802]"
      },
      "[epsg-method:9807]":
      {
         "title": "Transverse Mercator",
         "description": "Transverse Mercator projection",
         "parameters":
         {
            "[epsg-parameter:8801]": { "$ref": "#/parameters/[epsg-
parameter:8801]" },
            "[epsg-parameter:8802]": { "$ref": "#/parameters/[epsg-
parameter:8802]" }
         },
         "centerLatParam": "[epsg-parameter:8801]",
         "centerLonParam": "[epsg-parameter:8802]"
      },
      "[epsg-method:9820]":
      {
         "title": "Lambert Azimuthal Equal Area",
         "description": "Lambert Azimuthal Equal Area projection",
         "parameters":
         {
            "epsg-parameter:8801": { "$ref": "#/parameters/[epsg-parameter:8801]"
 },
            "epsg-parameter:8802": { "$ref": "#/parameters/[epsg-parameter:8802]"
 }
```

```
            },
            "centerLatParam": "[epsg-parameter:8801]",
            "centerLonParam": "[epsg-parameter:8802]"
         },
         "[epsg-method:9829]":
         {
            "title": "Polar Stereographic",
            "description": "Polar Stereographic projection",
            "parameters":
            {
               "[epsg-parameter:8832]": { "$ref": "#/parameters/[epsg-
parameter:8832]" },
               "[epsg-parameter:8833]": { "$ref": "#/parameters/[epsg-
parameter:8833]" },
            },
            "centerLatParam": "[epsg-parameter:8832]",
            "centerLonParam": "[epsg-parameter:8833]"
         },
         "[epsg-method:9840]":
         {
            "title": "Orthographic",
            "description": "Orthographic projection",
            "parameters":
            {
               "epsg-parameter:8801": { "$ref": "#/parameters/[epsg-parameter:8801]"
 },
               "epsg-parameter:8802": { "$ref": "#/parameters/[epsg-parameter:8802]"
 }
            },
            "centerLatParam": "[epsg-parameter:8801]",
            "centerLonParam": "[epsg-parameter:8802]"
         },
         "mollweide":
         {
            "title": "Mollweide",
            "description": "Mollweide projection",
            "parameters":
            {
               "epsg-parameter:8802": { "$ref": "#/parameters/[epsg-parameter:8802]"
 }
            },
            "centerLonParam": "[epsg-parameter:8802]"
         }
      },
      "parameters":
      {
         "[epsg-parameter:8801]": { "title": "Latitude of natural origin" },
         "[epsg-parameter:8802]": { "title": "Longitude of natural origin" },
         "[epsg-parameter:8823]": { "title": "Latitude of 1st standard parallel" },
         "[epsg-parameter:8824]": { "title": "Latitude of 2nd standard parallel" },
         "[epsg-parameter:8832]": { "title": "Latitude of standard parallel" },
         "[epsg-parameter:8833]": { "title": "Longitude of origin" }
      },
      "datums":
      {
         "[epsg-datum:6230]":
         {
            "title": "European Datum 1950",
            "description": "European Datum 1950",
            "ellipsoid": "[epsg-ellipsoid:7022]"
         },
         "[epsg-datum:6326]":
         {
```

```
            "title": "WGS84",
            "description": "World Geodetic System 1984 ensemble",
            "ellipsoid": "[epsg-ellipsoid:7030]"
         }
      }
   }
```

19

REQUIREMENTS CLASS "COLLECTION MAP"

19 REQUIREMENTS CLASS "COLLECTION MAP"

19.1. Overview

REQUIREMENTS CLASS 13: REQUIREMENTS CLASS 'COLLECTION MAP'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collection-map`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.13: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/collection-map`
PREREQUISITES	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core` https://www.opengis.net/spec/ogcapi-common-2/1.0/req/collections
NORMATIVE STATEMENTS	Requirement 46: `/req/collection-map/desc-links` Requirement 47: `/req/collection-map/desc-crs` Requirement 48: `/req/collection-map/map-operation`

The Collection Map requirements class specifies how to get maps from particular resources that contain geospatial data. Common resources that can contain geospatial data are the ones at the endpoint `/collections/{collectionId}` defined by the *OGC API — Common* or *OGC API — Features* Standards.

19.2. General

RECOMMENDATION 15	
IDENTIFIER	`/rec/collection-map/api-common`
A	Developers implementing the OGC API – Maps Standard should consider using the requirements classes specified in the https://www.opengis.net/spec/ogcapi-common-1/1.0/req/core (OGC API — Common version 1.0).

The Collection Maps requirements class depends on the *OGC API — Common — Part 2 — Geospatial data* "collections" requirements classes but remains flexible and does not require using *OGC API — Common — Part 1: Core*. This allows for other API architectures outside the OGC API framework to implement OGC API – Maps requirements classes. However, OGC API servers are normally expected to implement *OGC API — Common — Part 1*. If so, in practice, this means that the landing page and the conformance page follow *OGC API — Common — Part 1: Core*. It is also possible to combine this building block with *OGC API — Features — Part 1: Core* version 1.0 that is not tied to *OGC API — Common*, which can also be substituted for the *OGC API — Common — Part 2* dependency.

19.3. Geospatial data resources

The Maps API Standard does not specify how geospatial resources are exposed in the API or if they can be retrieved as raw geospatial data (e.g., feature items or coverage cell values). The Maps API Standard expects that other OGC API Standards will define how to expose additional access mechanisms for these geospatial resources. Examples of such standards, all of which also implement *OGC API — Common — Part 2: Geospatial data* defining the `/collections` and `/collections/{collectionId}` paths, are:

- *OGC API — Tiles* (Tiled vector data and tiled coverage data)
- *OGC API — Features* (feature collections and features)
- *OGC API — Coverages* (coverages and data cubes)
- *OGC API — EDR* (retrieval of spatiotemporal data)
- *OGC API — Processes — Part 3: Workflows and Chaining* ("Collection Output" requirements class)

The geospatial resources managed by these OGC API Standards (tiled vector data and tiled coverage data, feature collections and features, coverages and data cubes, spatiotemporal data and collection outputs as a result of a process execution) can be modified or complemented by other resources creating new endpoints indirectly giving access to the modified or complemented resources. Other OGC API Standards will define these mechanisms. An example of doing that is:

- *OGC API — Styles* which defines how to complement a geospatial resource by adding a style to it. The new resource is an overlap of both resources.

NOTE 1:The concept of a geospatial resource path substitutes the concept of "layer" in WMS but it intends to give a better integration between data visualization and data access.

REQUIREMENT 46

IDENTIFIER	`/req/collection-map/desc-links`
INCLUDED IN	Requirements class 13: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collection-map`
A	The OGC API collection description SHALL include a link with relation type `https://www.opengis.net/def/rel/ogc/1.0/map` (or `[ogc-rel:map]`) and the href pointing to the map resource for this collection.

PERMISSION 7

IDENTIFIER	`/per/collection-map/desc-links`
A	The collection can be exposed as a map totally or partially.

REQUIREMENT 47

IDENTIFIER	`/req/collection-map/desc-crs`
INCLUDED IN	Requirements class 13: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collection-map`
A	The `crs` property in the collection object of a geospatial collection SHALL contain URI or safe CURIEs for the list of CRSs supported by the server for that collection.
B	If the collection is available more efficiently (e.g., if it is stored in the server in that CRS) using a particular CRS (the native CRS, also *called storage CRS*) that is not https://www.opengis.net/def/crs/OGC/1.3/CRS84, a `storageCrs` property in the collection object of a geospatial collection SHALL be the URI or the safe CURIE for that CRS.
C	If a `storageCrs` property is used and that is not https://www.opengis.net/def/crs/OGC/1.3/CRS84, an overall bounding box (and optional inner bounding boxes for sparse data) SHALL be provided in a `storageCrsBbox` property within the `spatial` dimension of the `extent` following the same schema as the CRS84 `bbox`.

RECOMMENDATION 16

IDENTIFIER	`/rec/collection-map/storage-epoch`
A	If the collection is available more efficiently using a particular CRS (exposed in the `storageCrs` parameter) that is a dynamic coordinate reference system, the property `storageCrsCoordinateEpoch` in the collection object of the geospatial collection SHOULD provide the epoch of that CRS.

Example — Fragment of a collection with a links array with one item of the array pointing to a map.

```
{
  "id": "buildings",
  "title": "Buildings in the city of Bonn",
  "description": "This collection contains buildings",
  "attribution": "OpenStreetMap",
  "extent": {
    "spatial" : {
      "bbox" : [ [ -2.2830363, 39.8188272, 6.6350523, 42.7010011 ] ]
      "storageCrsBbox" : [ [ 47736, 4421022, 797736, 4734022 ] ]
    }
  },
  "crs": ["[EPSG:32631]", "[EPSG:23031]", "[EPSG:4326]"],
  "storageCrs": "[EPSG:32631]",
  "storageCrsCoordinateEpoch": 2022.3,
  "links": [
    ...
    {
      "href": "https://data.example.com/collections/buildings/map",
      "rel": "https://www.opengis.net/def/rel/ogc/1.0/map",
      "type": "image/png",
    }
  ]
}
```

NOTE 2: In the WMS Standard, layers have a hierarchical dependency. At the time this version of the Maps API Standard was approved, neither *OGC API — Features* nor *OGC API — Common* defined such a concept. However, there was a draft proposal to establish a hierarchical relation between collections based on a `parent` property and/or an `up` link relation type in the collection description. If a collection represented a hierarchy, then the Collections Selection requirements class could allow the selection of a subset of the collections it consists of to be included using the `collections` parameter.

19.4. Geospatial Data Resource Map

The core requirements class defines a map resource that is associated with an operation that contains the necessary information to later formulate a map request. Nevertheless, the Maps API core does not specify how to retrieve a map resource representing another resource. This section provides one possible workflow to support that use case from a geospatial data resource offered by an implementation of the Maps API.

19.4.1. Map path

REQUIREMENT 48	
IDENTIFIER	/req/collection-map/map-operation
INCLUDED IN	Requirements class 13: https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collection-map
A	Every OGC API collection available as a map SHALL support an HTTP GET operation to a URL /collections/{collectionId}/map to retrieve a map from that collection resource.

This requirements class does not specify any additional query parameter for the GET request.

19.4.2. Response

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core).

A successful response is a map that represents the collection of the geospatial data resource.

20

REQUIREMENTS CLASS "DATASET MAP"

20 REQUIREMENTS CLASS "DATASET MAP"

20.1. Overview

REQUIREMENTS CLASS 14: REQUIREMENTS CLASS 'DATASET MAP'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/dataset-map`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.14: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/dataset-map`
PREREQUISITES	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core` https://www.opengis.net/spec/ogcapi-common-1/1.0/req/core https://www.opengis.net/spec/ogcapi-common-1/1.0/req/landing-page
NORMATIVE STATEMENTS	Requirement 49: `/req/dataset-map/landingpage` Requirement 50: `/req/dataset-map/desc-extent` Requirement 51: `/req/dataset-map/desc-crs` Requirement 52: `/req/dataset-map/operation`

The Maps API core requirements class defines how to retrieve a map from a resource but does not detail which resources can be "mapped". The Dataset Map requirements class defines how to retrieve a map resource from the dataset represented by the service. In other words, it provides the path to retrieve a map resource from the dataset behind the API endpoint. This is achieved by adding the map path to the root (a.k.a. the landing page) of the service.

20.2. General

The Dataset Map describes how to serve maps for a dataset provided through a Web API instance implementing *OGC API — Common — Part 1* requirements classes or OGC API — Features — Part 1: Core, version 1.0 as an alternative.

20.3. Dataset API landing page

The landing page provides links to start exploring the resources offered by the API endpoint. The landing page mainly consists of a list of links to root resources. The Maps API Standard extension specifies a new link in the landing page for getting a dataset map.

20.3.1. Response

The root path of an OGC API offering access to geospatial data represents a dataset that will be exposed in different ways by the API.

Sub-resources can be added to the root path of an OGC API endpoint to complement it by offering new endpoints indirectly giving access to the complemented dataset. Other OGC API Standards will define this mechanism. An example is OGC API — Styles which defines how to complement a geospatial resource by adding a style to it. The new resource is an overlap of the dataset and the resource.

REQUIREMENT 49	
IDENTIFIER	`/req/dataset-map/landingpage`
INCLUDED IN	Requirements class 14: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/dataset-map`
A	The deployed API endpoint landing page SHALL include a link with relation type `https://www.opengis.net/def/rel/ogc/1.0/map` (or `[ogc-rel:map]`) to the dataset map URL at `/map`.

In the landing page, in JSON format, the links follow the link schema defined in the OGC API — Common or in the OGC API — Features Standards. Below you can find an example fragment of the response to an OGC API — Maps landing page showing the link to the dataset map.

REQUIREMENT 50	
IDENTIFIER	`/req/dataset-map/desc-extent`
INCLUDED IN	Requirements class 14: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/dataset-map`
A	An extent CRS SHALL be provided in an “extent” property of the API landing page following the same schema as the “extent” property for the collection (see OGC API — Common — Part 2: Geospatial Data).

NOTE: The spatial bounding box is provided in https://www.opengis.net/def/crs/OGC/1.3/CRS84 for Earthly data.

REQUIREMENT 51

IDENTIFIER	`/req/dataset-map/desc-crs`
INCLUDED IN	Requirements class 14: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/dataset-map`
A	The `crs` property in the landing page of a dataset SHALL contain URIs or safe CURIEs for the list of CRSs supported by the dataset as a whole.
B	If the dataset is available more efficiently using a particular CRS that is not https://www.opengis.net/def/crs/OGC/1.3/CRS84, a `storageCrs` property in the landing page of a dataset SHALL have the URI or the safe CURIE for that CRS as a value. For example, it may be more efficient to retrieve a map in the same CRS used to store the data. Note that the native CRS is also called the *storage CRS*.
C	If a `storageCrs` property is used and that is not https://www.opengis.net/def/crs/OGC/1.3/CRS84, an overall bounding box (and optional inner bounding boxes for sparse data) SHALL be provided in a `storageCrsBbox` property within the `spatial` dimension of the `extent` following the same schema as the CRS84 `bbox`.

RECOMMENDATION 17

IDENTIFIER	`/rec/dataset/storage-epoch`
A	If the dataset is available more efficiently using a particular CRS (exposed in the `storageCrs` parameter) that is a dynamic coordinate reference system, the property `storageCrsCoordinateEpoch` in the landing page of the dataset SHOULD provide the epoch of that CRS.

Example — API Landing Page fragment that advertises the dataset map, the storageCrs and the extent of it.

```
{
  "title": "Dataset of the city of Bonn",
  "description": "Dataset containing collections such as buildings",
  "attribution": "OpenStreetMap",
  "extent": {
    "spatial" : {
         "bbox" : [ [ -2.2830363, 39.8188272, 6.6350523, 42.7010011 ] ],
         "storageCrsBbox" : [ [ 47736, 4421022, 797736, 4734022 ] ]
    }
  },
  "crs": ["EPSG:32631", "EPSG:23031", "EPSG:4326"],
  "storageCrs": "EPSG:32631",
  "storageCrsCoordinateEpoch": 2022.3,
  "links": [
    ...,
    {
      "href": "https://data.example.org/map",
      "rel": "https://www.opengis.net/def/rel/ogc/1.0/map",
      "type": "image/png",
      "title": "Map combining collections in the dataset"
    }
```

```
  ]
}
```

20.4. Dataset maps

20.4.1. Map path

REQUIREMENT 52	
IDENTIFIER	`/req/dataset-map/operation`
INCLUDED IN	Requirements class 14: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/dataset-map`
A	The Maps API implementation SHALL support an HTTP GET operation for the `/map` URL to retrieve a map from the dataset API endpoint in the default style.

The operation request does not define extra parameters. Servers can advertise the requirements class https://www.opengis.net/spec/ogcapi-tiles-1/1.0/req/collections-selection to add a `collections` parameter that can be used to include only parts of the dataset in the map representation.

20.4.2. Response

A successful GET response is described in the core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core).

A successful response is a map resource that represents the entire dataset. In a Web API providing access to a complex dataset comprised of several geospatial data resources, there are reasons for being selective in the amount of information included. This can be achieved by applying a manual filter described in the Requirements Class “Collection Selection” or as an automatic decision by the server side.

RECOMMENDATION 18	
IDENTIFIER	`/rec/dataset-map/geodata-selection`
A	When it is possible and sensible, all geospatial data resources (`/collections`) supporting the requested CRS SHOULD be represented in the map response.

PERMISSION 8

IDENTIFIER	`/per/dataset-map/geodata-selection`
A	If it is not possible and sensible to represent all geospatial data resources (`/collections`) in a map (e.g. it compromises performance or maps are become packed with too many elements), the server is allowed to select only the most significant geospatial data resources.

21

REQUIREMENTS CLASS "STYLED MAPS"

21 REQUIREMENTS CLASS "STYLED MAPS"

21.1. Overview

REQUIREMENTS CLASS 15: REQUIREMENTS CLASS 'STYLED MAPS'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/styled-map`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.15: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/styled-map`
PREREQUISITE	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
NORMATIVE STATEMENTS	Requirement 53: `/req/styled-map/desc-links` Requirement 54: `/req/styled-map/map-operation`

The Styles Map requirements class specifies how to get maps from particular resources with a style applied by *OGC API — Styles* 1.0.

NOTE: This mechanism replaces the parameter `STYLES` in WMS 1.3 and adds the possibility to have a generic style for all layers of the service at once.

21.2. Styled resources

Geospatial resources can be modified or complemented by styles creating new endpoints giving access to the modified or complemented resources. The way this is done to a dataset resource or to a collection is specified in the *OGC API – Styles* candidate Standard (as of December 2023).

REQUIREMENT 53	
IDENTIFIER	`/req/styled-map/desc-links`

REQUIREMENT 53	
INCLUDED IN	Requirements class 15: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/styled-map`
A	If the deployed API endpoint has a mechanism to expose links associated with styled geospatial resources (e.g., the OGC API — Styles list of styles at `/styles` for a dataset or at `/collections/{collectionId}/styles` for a collection), those styled resources SHALL include a link with link relation `https://www.opengis.net/def/rel/ogc/1.0/map` (or `[ogc-rel:map]`) and the href pointing to the map associated with that styled resource.

Example — Fragment of a collection with a links array with one item of the array pointing to a map.

```
{
  "id": "buildings",
  "title": "Buildings in the city of Bonn",
  "description": "This collection contains buildings",
  "attribution": "OpenStreetMap",
  "extent": {
    ...
  }
  "links": [
    ...
    {
      "href": "https://data.example.com/collections/buildings/styles/dark/map",
      "rel": "https://www.opengis.net/def/rel/ogc/1.0/map",
      "type": "image/png",
    }
  ]
}
```

21.3. Styled Resource Map

Styled resources are defined by the draft OGC API – Styles candidate Standard (as of December 2023). Nevertheless, that draft does not specify how to retrieve a map resource representing a styled resource. This section explains how to do this.

21.3.1. Map path

REQUIREMENT 54	
IDENTIFIER	`/req/styled-map/map-operation`
INCLUDED IN	Requirements class 15: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/styled-map`

REQUIREMENT 54	
A	Every resource for which a styled map is available SHALL support an HTTP GET operation to a `../styles/{styleId}/map` URL to retrieve a map for a particular style (e.g., `/collections/{collectionId}/styles/{styleId}` for a styled collection map or `/styles/{styleId}/map` for a styled dataset map).

The Maps API Standard does not specify any additional query parameter in the GET request.

21.3.2. Response

A successful GET response is described in the Maps API core class (https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core).

A successful response is a map that represents the styled resource.

22

REQUIREMENTS CLASSES FOR ENCODINGS

22 REQUIREMENTS CLASSES FOR ENCODINGS

The *OGC API — Maps* Standard is designed to support mapped data that can potentially be encoded and provided in existing geospatial formats or new formats that could be developed in the future. Web API deployments may adopt these encodings and declare conformance to them in the list of conformance classes supported by the API. The requirements in this section do not limit the number of encodings offered by a Maps API deployment. The intent is to provide a minimum set of encodings that could be implemented by any deployed Maps APIs as well as to provide a practical way to test conformance to this Standard. For each of these encodings, a requirements class is defined. An API implementing OGC API – Maps classes is free to support other encodings and data formats. These other formats may be more convenient for given use cases even though they may not be listed as supported conformance classes by the API. In addition, the declaration of an encoding in the supported conformance classes does not mean that all the resources provided by an implementation of an OGC API Standard should support all of them. Partial support could be conditioned by the nature of the data behind each collection.

22.1. Overview

Six requirements classes for map responses of *OGC API — Maps* implementations supporting various encodings are defined:

- PNG
- JPEG
- JPEG XL
- TIFF
- SVG
- HTML

NOTE 1:If maps are provided as tiles, this section should be ignored and the equivalent section in the *OGC API – Tiles — Part 1: Core* Standard should be used instead.
NOTE 2:None of the encodings specified here are mandatory and an implementation of the Maps API Standard may implement none of them but implement other encodings instead.

22.2. Requirements Class “PNG”

PNG can be used for simple and immediate visualization in a web browser. For this use case, selecting an encoding that can be natively interpreted by the web browser is fundamental. The PNG format is one of the most popular formats. The PNG format is defined by the ISO Information technology Computer graphics and image processing ISO/IEC 15948:2004 Information technology — Computer graphics and image processing — Portable Network Graphics (PNG): Functional specification, also available from the World Wide Web Consortium (W3C) at https://www.w3.org/TR/PNG.

PNG supports lossless data compression. PNG supports palette-based images (with palettes of 24-bit RGB or 32-bit RGBA colors), grayscale images (with or without an alpha channel for transparency), and full-color non-palette-based RGB or RGBA images. The PNG working group designed the format for transferring images on the Internet, not for professional-quality print graphics. Therefore, non-RGB color spaces such as CMYK are not supported.

REQUIREMENTS CLASS 16: REQUIREMENTS CLASS ‘PNG’	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/png`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.16: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/png`
PREREQUISITES	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core` ISO/IEC 15948 Portable Network Graphics (PNG)
NORMATIVE STATEMENT	Requirement 55: `/req/png/content`

REQUIREMENT 55	
IDENTIFIER	`/req/png/content`
INCLUDED IN	Requirements class 16: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/png`
A	Every 200-response of the server with the media type `image/png` SHALL be a PNG document representing only one map.
B	The colors of the PNG SHALL represent the geospatial features or coverage values in the map.
C	The alpha channel of the PNG SHALL be used when partial transparency is required

REQUIREMENT 55	
D	All maps representing parts of the same resource or resources and using the same style SHALL follow the same portrayal rules

NOTE: The way the colors in the PNG format are mapped to geospatial features or coverage values is out of scope for the Maps API Standard. However, a common set of portrayal rules for all maps representing part of the same resource is essential, as maps are normally represented one after the other as a response to actions of the user (e.g. zoom and pan).

22.3. Requirements Class "JPEG"

JPEG can be used for simple and immediate visualization in a web browser. In these circumstances, selecting an encoding that can be natively interpreted by the web browser is fundamental. The JPEG format is another popular format used in web applications. JPEG is defined by the ITU-T Recommendation T.81 and the ISO/IEC 10918-1 Standard.

The JPEG compression algorithm operates at its best on photographs and paintings with smooth variations of tone and color. It is best used for color and grayscale still images, but not for binary images. JPEG is also the most common format used by digital cameras. However, JPEG is not well suited for line drawings and other textual or iconic graphics, where the sharp contrasts between adjacent pixels can cause noticeable artifacts. Such images are better saved in a lossless graphics format such as PNG because JPEG uses a lossy compression method, which reduces image fidelity. As such, it is inappropriate for exact reproduction of imaging data.

REQUIREMENTS CLASS 17: REQUIREMENTS CLASS 'JPEG'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/jpeg`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.17: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/jpeg`
PREREQUISITES	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core` ISO/IEC 10918-1
NORMATIVE STATEMENT	Requirement 56: `/req/jpeg/content`

REQUIREMENT 56	
IDENTIFIER	`/req/jpeg/content`
INCLUDED IN	Requirements class 17: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/jpeg`
A	Every 200-response of the server with the media type `image/jpeg` SHALL be a JPEG document representing only one map.
B	The colors of the JPEG SHALL represent geospatial features and/or coverage values in the map.
C	All maps representing parts of the same resource or resources and using the same style SHALL follow the same portrayal rules.

NOTE 1: The way the colors in the JPEG are mapped to geospatial features or coverage values is out of scope of the Maps API Standard. However, a common set of portrayal rules for all maps representing part of the same resource is essential, as maps are normally represented one after the other as a response to actions of the user (e.g., zoom and pan).

NOTE 2: JPEG is an ideal format for some remote sensing imagery. The use of JPEG to represent linear features or color solid polygons is not recommended.

22.4. Requirements Class "JPEG XL"

JPEG XL is a relatively newer image format (compared to PNG and JPEG) with support for both lossy and lossless compression, alpha channels, high bit depths, high image quality, high compression ratio as well as high performance encoding and decoding. Free and open source libraries are available to read and write the format such as libjxl, released under a 3-clause BSD license. Although JPEG XL is not yet widely supported in web browsers at the time of writing this document, this situation is likely to change, and this requirements class is included due to the numerous advantages of JPEG XL over the traditional JPEG and PNG encoding, such as avoiding the need for compromising between high compression ratio, image quality and translucency. JPEG XL is defined by the multi-part ISO standard Information technology JPEG XL — image coding system, including ISO/IEC 18181-1:2024 Part 1: Core coding system and ISO/IEC 18181-2:2024 Part 2: File format.

REQUIREMENTS CLASS 18: REQUIREMENTS CLASS 'JPEG XL'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/jpegxl`
TARGET TYPE	Web API

REQUIREMENTS CLASS 18: REQUIREMENTS CLASS 'JPEG XL'	
CONFORMANCE CLASS	Conformance class A.18: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/jpegxl`
PREREQUISITES	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core` ISO/IEC 18181-1 JPEG XL image coding system — Part 1: Core coding system ISO/IEC 18181-2 JPEG XL image coding system — Part 2: File format
NORMATIVE STATEMENT	Requirement 57: `/req/jpegxl/content`

REQUIREMENT 57	
IDENTIFIER	`/req/jpegxl/content`
INCLUDED IN	Requirements class 18: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/jpegxl`
A	Every 200-response of the server with the media type `image/jxl` SHALL be a JPEG XL file representing only one map.
B	The JPEG XL SHALL be a color image representing the geospatial features or coverage values in the map.
C	All maps representing parts of the same resource or resources and using the same style SHALL follow the same portrayal rules.

NOTE 1: The way the colors in the JPEG XL are mapped to geospatial features or coverage values is out of scope of the Maps API Standard. However, a common set of portrayal rules for all maps representing part of the same resource is essential, as maps are normally represented one after the other as a response to actions of the user (e.g., zoom and pan).

NOTE 2: JPEG XL is an ideal file format for all map types, due to its support for high image quality (including the possibility of lossless encoding), high compression ratio, high bit depth and translucency.

22.5. Requirements Class "TIFF"

One use case for maps is to distribute coverage values as regular grids. In these circumstances, selecting an encoding able to store grid values in their original format and eventually compress them using lossless compression is the right solution. The TIFF format is one of the older formats but is still one of the most popular formats to preserve arrays of data values and is defined by the Adobe Systems TIFF v6 specification.

TIFF is a flexible, adaptable file format for handling images and data. The ability to store data in a lossless format makes a TIFF file a useful image archive: Unlike standard JPEG files, a TIFF file using lossless compression such as PackBits or LZW compression.

REQUIREMENTS CLASS 19: REQUIREMENTS CLASS 'TIFF'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tiff`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.19: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/tiff`
PREREQUISITES	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core` TIFF Revision 6.0
NORMATIVE STATEMENT	Requirement 58: `/req/tiff/content`

REQUIREMENT 58	
IDENTIFIER	`/req/tiff/content`
INCLUDED IN	Requirements class 19: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tiff`
A	Every 200-response of the server with the media type `image/tiff` SHALL be a TIFF document representing only one map.
B	The TIFF file SHALL represent colors by using an image palette or RGB combination.
C	All maps representing parts of the same resource or resources and using the same style SHALL follow the same portrayal rules or represent data with the same reference and units of measure.

NOTE: TIFF is an ideal support for geospatial grid data values in its original format. This is not allowed for OGC API — Maps and it is expected that OGC API endpoints dealing with coverages will support this mode.

RECOMMENDATION 19	
IDENTIFIER	`/rec/tiff/geotiff`
A	A TIFF encoding SHOULD include georeference information using the OGC GeoTIFF Standard.

22.6. Requirements Class “SVG”

SVG is a commonly used format for representing vector objects on the web. SVG is simple to understand and well supported by tools and software libraries. Modern web browsers can render the SVG format directly and can also react to events generated by the user on the features visualized. That is why it is a good alternative for creating more interactive maps.

REQUIREMENTS CLASS 20: REQUIREMENTS CLASS ‘SVG’	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/svg`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.20: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/svg`
PREREQUISITES	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core` Scalable Vector Graphics (SVG) 2
NORMATIVE STATEMENT	Requirement 59: `/req/svg/content`

REQUIREMENT 59	
IDENTIFIER	`/req/svg/content`
INCLUDED IN	Requirements class 20: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/svg`
A	Every 200-response of the server with the media type `image/svg+xml` SHALL be an SVG document representing only a map.
B	The SVG coordinates inside the map SHALL start at 0,0 and end in the width and height of the request.

PERMISSION 9	
IDENTIFIER	`/per/svg/overflow`
A	SVG content of a map can contain features that are partially outside of the requested map bounding box (represented by negative coordinates or coordinates bigger than the requested width and height).

22.7. Requirements Class “HTML”

Returning an HTML page with a full, operational map browser that allows for interactive navigation over the map (e.g., zoom and pan) is convenient for easy exploration of the data or for testing purposes.

REQUIREMENTS CLASS 21: REQUIREMENTS CLASS ‘HTML’	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/html`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.21: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/html`
PREREQUISITES	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core` HTML Living Standard
NORMATIVE STATEMENT	Requirement 60: `/req/html/content`

REQUIREMENT 60	
IDENTIFIER	`/req/html/content`
INCLUDED IN	Requirements class 21: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/html`
A	Every 200-response of the server with the media type `text/html` SHALL be an HTML document representing the geospatial data as maps.

NOTE 1: An HTML page can contain links to other resources. In particular, it can contain image sources linked to map resources as image/png or image/jpeg that will represent the actual data.

NOTE 2: An HTML page may contain MapML elements to represent the map (https://maps4html.org/MapML/spec/).

RECOMMENDATION 20	
IDENTIFIER	`/rec/html/content`

RECOMMENDATION 20	
A	An HTML encoding MAY internally use requests to maps in image/png or image/jpeg format using this OGC API, to present the map images (e.g. in img src tags) to start and as a response to the actions of the user (e.g. zoom and pan).

23

REQUIREMENTS CLASS “API OPERATIONS”

23 REQUIREMENTS CLASS "API OPERATIONS"

23.1. Overview

REQUIREMENTS CLASS 22: REQUIREMENTS CLASS 'API OPERATIONS'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/api-operations`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.22: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/api-operations`
PREREQUISITE	https://www.opengis.net/spec/ogcapi-common-1/1.0/req/landing-page
NORMATIVE STATEMENTS	Requirement 61: `/req/api-operations/completeness` Requirement 62: `/req/api-operations/operation-id`

The "API Operations" requirements class defines requirements for providing a definition of a Web API implementing the Maps API Standard using an API definition language. This requirements class is intended to be combined with another requirements class specifying requirements for providing an API definition in a particular API definition language and / or version, such as the OpenAPI 3.0 requirements class defined in OGC API — Common — Part 1 :Core, or another eventual requirements class for OpenAPI 3.1.

23.2. Web API description

The API definition provides a description of the complete list of API resources available at an endpoint. Reading this description, an application would have the full picture of the resources that the API implementation provides, how to retrieve resources, and what responses are expected for successful and unsuccessful requests. Without an API description or documentation, an application would be forced to traverse all links, starting with the landing page, to get an equivalent full list of resources.

The *OpenAPI Specification 3.0 (oas30)* requirements class from *OGC API — Common — Part 1: Core* provides many details on general requirements that can be used in conjunction with this requirements class. A later version of OGC API — Common may describe how to declare support

for other versions of OpenAPI. This requirements class depends on the *OGC API — Common* landing page requirements class which provides details on how to request an API definition.

23.2.1. Response

23.2.1.1. Completeness

The API definition resulting as a response to this request needs to take into consideration the relevant resources specified in the Maps API Standard.

REQUIREMENT 61	
IDENTIFIER	`/req/api-operations/completeness`
INCLUDED IN	Requirements class 22: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/api-operations`
A	The API definition SHALL provide paths for all map, custom projections, tileset, tilesets list and tile resources provided by the API instance.
B	The resource paths defined in the API definition SHALL be consistent with the links to the same resources provided by the landing page, collections, tileset and tilesets list resources.
C	The resource paths defined in the API definition SHALL provide the description of the parameters that the map, tileset and tile resources need to operate that are specified in corresponding conformance classes.

23.2.1.2. Reusable API components

Reusable components for creating API definitions for implementations of this OGC API — Maps Standard using OpenAPI 3.0 can be found at https://schemas.opengis.net/ogcapi/maps/part1/1.0/openapi .

A server implementation of the *OGC API – Maps Standard* can use the content in the openapi directory to generate a response for the API description using OpenAPI 3.0. The *ogcapi-maps-1.yaml* file includes paths and components. An implementation should only include the paths that are implemented and remove the references to the paths that are not implemented. The components part includes parameters, responses and schemas that can be reused as-is. The `api` directory contains JSON files that are templates with enumerated values for collections, styles and tile matrix sets. A particular implementation of the Maps API should enumerate the actual resources exposed by the API deployment in the same way. The server can select to dynamically implement responses to /api/* (where * is replaced by all-collections, styles,…) or hardcode the /api/* files with the actual list of resource identifiers in the enumerations.

To improve performance, the whole content of this directory can be bundled into a single document by executing a tool such as *swagger-cli*. This document can be served for the *OGC API — Common — Part 1* `service-desc` link from the landing page.

23.2.1.3. Path Operation Identifiers

The API definition provides a client application with a set of paths that the client can use to interact with the API endpoint and get new resources. The API description of each path provides a description of what parameters to use in the request and what to expect in the response. However, the Maps API Standard does not propose a fixed set of paths so there is an issue identifying the requirements classes pertaining to each path in an API instance. In other words, the API description alone does not provide enough information by itself. Therefore, there is a need to associate the resource paths in the API definition with the resources defined in the various requirements classes. The Maps API Standard proposes a suffix mechanism to be applied to the operation identifiers of the path (e.g., the `operationId` property in OpenAPI Specification 3.0) to list the requirements classes pertaining to each path. Each path should have a unique operationId suffix, so it is expected that the API instance provides a prefix to the proposed suffixes that make each operation identifier unique.

REQUIREMENT 62	
IDENTIFIER	`/req/api-operations/operation-id`
INCLUDED IN	Requirements class 22: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/api-operations`
A	For API definitions with a concept of operation identifiers, the paths defined in the API definition SHALL have an operation identifier value ending with the relevant dot-separated suffix corresponding to the resource as specified in API operation identifier suffixes.

Table 11 — API operation identifier suffixes

ORIGIN	STYLED	RESOURCE	OPERATION ID SUFFIXES
root		Projections	`getCustomCRSProjections`
With the origins described in this document			
DataSet[5]		Map[1]	`.dataset.getMap`
DataSet[5]		TileSetsList[4]	`.dataset.map.getTileSetsList`
DataSet[5]		TileSet[3]	`.dataset.map.getTileSet`
DataSet[5]		Tile[2]	`.dataset.map.getTile`

ORIGIN	STYLED	RESOURCE	OPERATION ID SUFFIXES
DataSet[5]	Styled[7]	Map[1]	`.dataset.style.getMap`
DataSet[5]	Styled[7]	TileSetsList[4]	`.dataset.style.map.getTileSetsList`
DataSet[5]	Styled[7]	TileSet[3]	`.dataset.style.map.getTileSet`
DataSet[5]	Styled[7]	Tile[2]	`.dataset.style.map.getTile`
Collection[6]		Map[1]	`.collection.getMap`
Collection[6]		TileSetsList[4]	`.collection.map.getTileSetsList`
Collection[6]		TileSet[3]	`.collection.map.getTileSet`
Collection[6]		Tile[2]	`.collection.map.getTile`
Collection[6]	Styled[7]	Map[1]	`.collection.style.getMap`
Collection[6]	Styled[7]	TileSetsList[4]	`.collection.style.map.getTileSetsList`
Collection[6]	Styled[7]	TileSet[3]	`.collection.style.map.getTileSet`
Collection[6]	Styled[7]	Tile[2]	`.collection.style.map.getTile`
With other potential origins[8]			
other		Map[1]	`#.getMap`
other		TileSetsList[4]	`#.map.getTileSetsList`
other		TileSet[3]	`#.map.getTileSet`
other		Tile[2]	`#.map.getTile`
other	Styled[7]	Map[1]	`#.style.getMap`
other	Styled[7]	TileSetsList[4]	`#.style.map.getTileSetsList`
other	Styled[7]	TileSet[3]	`#.style.map.getTileSet`
other	Styled[7]	Tile[2]	`#.style.map.getTile`

ORIGIN	STYLED	RESOURCE	OPERATION ID SUFFIXES

[1] The *Map* resource is defined in requirements class “Core”.

[2] The *Tile* resource is defined in the *OGC API — Tiles — Part 1: Core* “Core” requirements class.

[3] The *TileSet* resource is defined in the *OGC API — Tiles — Part 1: Core* “TileSet” requirements class.

[4] The *TileSetsList* resource is defined in the *OGC API — Tiles — Part 1: Core* “TileSets List” requirements class. Map tilesets are defined in the *Map Tilesets* requirements class_ and depend on *OGC API — Tiles — Part 1: Core*.

[5] The *DataSet* origin is defined in requirements class “Dataset Maps” and depends on *OGC API — Common — Part 1: Core*.

[6] The *Collection* origin is defined in requirements class “Collection Maps” and depends on the *Collections* requirements class defined in *OGC API — Common — Part 2: Geospatial data*.

[7] Styled tilesets rely on the ability to list styles defined in *OGC API — Styles*.

[8] ‘#’ represents an optional *origin* that could be defined in another relevant standard.

24

REQUIREMENTS CLASS "CORS"

24 REQUIREMENTS CLASS "CORS"

24.1. Overview

REQUIREMENTS CLASS 23: REQUIREMENTS CLASS 'CORS'	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/cors`
TARGET TYPE	Web API
CONFORMANCE CLASS	Conformance class A.23: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/cors`
PREREQUISITE	https://www.w3.org/TR/2020/SPSD-cors-20200602/
NORMATIVE STATEMENT	Requirement 63: `/req/cors/cors`

Many current implementations of map clients are done in HTML and JavaScript on HTTPS. A common situation is that a single JavaScript client combines maps from different Web APIs and domains. In this case, Cross-Origin Resource Sharing (CORS) is applied by web browsers preventing access of data from a domain that is different from the Web API, if this is not explicitly allowed by the server headers. The CORS requirements class requires the implementation of CORS support for the Web APIs.

24.2. CORS for JavaScript clients

REQUIREMENT 63	
IDENTIFIER	`/req/cors/cors`
INCLUDED IN	Requirements class 23: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/cors`
A	The implementation SHALL support CORS as defined by W3C (https://www.w3.org/TR/2020/SPSD-cors-20200602/).

NOTE 1: By supporting CORS, an implementation allows JavaScript clients (e.g. Web Browser applications) to read the map from a domain different than the OGC API — Maps endpoint.

NOTE 2: It is not the purpose of this Standard to specify how CORS is implemented and that may change as web browsers and servers become more concerned about security (for example, currently a PNG in a <img src=""> is not affected by CORS but a PNG retrieved by the JavaScript fetch() function is affected). It is worth noting that a server that wants to authorize a JavaScript client application from another domain (under the CORS rules) should use the Access-Control-Allow-Origin: header to list which domains are authorized to read the map (or use * to indicate that all domains are authorized). Services that want to comply to the open data sharing principles should be willing to allow all clients reading the data even if they are in other domains. However, this is up to the server implementers.

A

ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE

ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE

A.1. Conformance Class “Core”

CONFORMANCE CLASS A.1	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core`
REQUIREMENTS CLASS	Requirements class 1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.1: `/conf/core/map-op` Abstract test A.2: `/conf/core/map-response` Abstract test A.3: `/conf/core/conformance-success`

A.1.1. Abstract Test for Requirement Map Operation

ABSTRACT TEST A.1	
IDENTIFIER	`/conf/core/map-op`
REQUIREMENT	Requirement 1: `/req/core/map-op`
TEST PURPOSE	Verify that the implementation supports the map retrieval operation
TEST METHOD	**Given:** a geospatial data resource conforming to the Maps API Standard, with an API path including `.../map...` or discovered following a link with relation type `[ogc-rel:map]` **When:** performing a GET operation on the `/map` resource with a supported media type specified in the `Accept:` header (e.g., `Accept: image/png,image/jpeg`) **Then:**

ABSTRACT TEST A.1	
	- assert that the implementation supports retrieving map resources at one or more `.../map` URL.

A.1.2. Abstract Test for Requirement Map Response

ABSTRACT TEST A.2	
IDENTIFIER	`/conf/core/map-response`
REQUIREMENT	Requirement 2: `/req/core/map-response`
TEST PURPOSE	Verify that the implementation's response for the map retrieval operation is correct
TEST METHOD	**Given:** a map resource that was successfully retrieved for Abstract test A.1: `/conf/core/map-op` **When:** retrieving that resource for the map operation **Then:** - assert that the response has an HTTP status code 200, - assert that the map response is in the storage CRS specified in the description of the geospatial resource (such as the associated collection or dataset), or https://www.opengis.net/def/crs/OGC/1.3/CRS84 if none is specified, - assert that the headers of the response include the `Content-Crs:` header with the URI or the safe CURIEs of the CRS used to render the map, except if the content is in the https://www.opengis.net/def/crs/OGC/1.3/CRS84 CRS, - assert that the headers of the response include a `Content-Bbox:` header with the actual geospatial boundary of the rendered map, - assert that the `Content-Bbox:` coordinates are in the response CRS (indicated in the `Content-Crs:` or https://www.opengis.net/def/crs/OGC/1.3/CRS84 if it is not present) and contains four comma-separated numbers representing the lower-left and upper-right corners of the response honoring the CRS coordinates order, - assert that if the geospatial resource has a temporal aspect (e.g., if a temporal extent is defined in the collection description or in the dataset landing page), the headers of the response include a `Content-Datetime:` header with the actual datetime instant or datetime interval of the rendered map. - assert that a `Content-Datetime:` response header representing a temporal interval, if present, uses the notation `time-instant / time-instant`, - assert that if `Content-Datetime:` response header is present, time instants, whether a single value or the `time-instant` of an interval, are expressed as defined in RFC 3339, section 5.6 format, using either UTC time or local time offsets, but also allowing for the additional formats `yyyy`, `yyyy-mm`, `yyyy-mm-dd`, `yyyy-mm-ddThhZ` (or e.g., `yyyy-mm-ddThh-04:00`) and `yyyy-mm-ddThh:mmZ` (or e.g., `yyyy-mm-ddThh-04:00`), - assert that the body of the response contains a map encoded in the negotiated format.

A.1.3. Abstract Test for Requirement Map Conformance Success

ABSTRACT TEST A.3	
IDENTIFIER	`/conf/core/conformance-success`
REQUIREMENT	Requirement 3: `/req/core/conformance-success`
TEST PURPOSE	For implementations having a mechanism to advertise conformance classes, verify that it reports conformance to this Standard correctly.
TEST METHOD	**Given:** a conformance resource in a recognized format, such as the OGC API JSON `/conformance` resource **When:** retrieving that resource from the API endpoint **Then:** - assert that the list of conformance classes that are supported by this API implementation instance includes at minimum https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/core. This list of conformance classes can also be used to select which other abstract tests to perform as listed in the Conformance class URIs and indicated in the tests preconditions.

A.2. Conformance Class “Map Tilesets”

CONFORMANCE CLASS A.2	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/tilesets`
REQUIREMENTS CLASS	Requirements class 2: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tilesets`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.4: `/conf/tilesets/desc-links` Abstract test A.5: `/conf/tilesets/tiles-parameters`

A.2.1. Abstract Test for Requirement desc-links

ABSTRACT TEST A.4	
IDENTIFIER	`/conf/tilesets/desc-links`
REQUIREMENT	Requirement 4: `/req/tilesets/desc-links`
TEST PURPOSE	Verify that the implementation supports map tilesets
TEST METHOD	**Given:** a geospatial data resource conforming to this Standard, to "Map Tilesets", to OGC *API — Tiles* and providing a description resource including links **When:** retrieving the geospatial data resource description **Then:** - assert that the geospatial data resource (e.g., collection or landing page description's `links` property) includes a link with the `href` pointing to a tileset list supported that presents a tile aspect of this geospatial data resource and with rel: `[ogc-rel:tilesets-map]`

A.2.2. Abstract Test for Requirement tiles-parameters

ABSTRACT TEST A.5	
IDENTIFIER	`/conf/tilesets/tiles-parameters`
REQUIREMENT	Requirement 5: `/req/tilesets/tiles-parameters`
TEST PURPOSE	Verify that the implementation supports relevant parameters for map tilesets
TEST METHOD	**Given:** a geospatial data resource conforming to this Standard, to "Map Tilesets", to OGC *API — Tiles*, and to *Maps* requirements classes introducing parameters relevant for map tiles **When:** retrieving the map tiles with parameters for the *background*, *display resolution*, *spatial subsetting* (only for `subset` and `subset-crs` parameters, and only if a vertical dimension is available), *general subsetting*, and *scaling* requirements classes **Then:** - assert that tiles responses reflect the relevant map parameters used for the requests

NOTE: This conformance class depends on *OGC API — Tiles — Part 1: Core* "Tilesets List" conformance class to which the implementation must also conform.

A.3. Conformance Class "Background"

CONFORMANCE CLASS A.3	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/background`
REQUIREMENTS CLASS	Requirements class 3: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/background`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.6: `/conf/background/bgcolor-definition` Abstract test A.7: `/conf/background/transparent-definition` Abstract test A.8: `/conf/background/void-color-definition` Abstract test A.9: `/conf/background/void-transparent-definition` Abstract test A.10: `/conf/background/map-success`

A.3.1. Abstract Test for Requirement `bgcolor` parameter definition

ABSTRACT TEST A.6	
IDENTIFIER	`/conf/background/bgcolor-definition`
REQUIREMENT	Requirement 6: `/req/background/bgcolor-definition`
TEST PURPOSE	Verify that the implementation supports the `bgcolor` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving a map without `bgcolor` parameter, with `bgcolor` using a hexadecimal value and with `bgcolor` using a W3C Web Color name **Then:** - assert that the map operation supports a `bgcolor` parameter in hexadecimal red-green-blue color value (from 00 to FF, FF representing 255) for the background color of the map. For a six-digit hexadecimal value, the first and second digits specify the intensity of red. The third and fourth digits specify the intensity of green. The fifth and sixth digits specify the intensity of blue, - assert that the map operation supports a `bgcolor` parameter in case-insensitive W3C web color name for the background color of the map, - assert that if `bgcolor` is not specified, and either `transparent` is set to `false` or the output format cannot encode transparency, and there is an style defined the server uses the background color specified by the requested style, - assert that if `bgcolor` is not specified, and either `transparent` is set to `false` or the output format cannot encode transparency, and there is no style used or the style do not specify a background color, the background color is set to 0xFFFFFF.

A.3.2. Abstract Test for Requirement `transparent` parameter definition

ABSTRACT TEST A.7	
IDENTIFIER	`/conf/background/transparent-definition`
REQUIREMENT	Requirement 7: `/req/background/transparent-definition`
TEST PURPOSE	Verify that the implementation supports the `transparent` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving a map for all combinations of (no `transparent` parameter, transparent=false,` `transparent=true)` and with and without `bgcolor`parameter **Then:** - assert that the server interprets `transparent` as a Boolean indicating whether the background of the map should be transparent, - assert that, if `transparent` is not specified and a `bgcolor` is not specified, the server assumes a value of `true`, - assert that, if `transparent` is not specified and a `bgcolor` is specified, the server assumes a value of `false`, - assert that, if `transparent` is `true` and a `bgcolor` is specified, the server uses 0 for the background's opacity.

A.3.3. Abstract Test for Requirement `void-color` parameter definition

ABSTRACT TEST A.8	
IDENTIFIER	`/conf/background/void-color-definition`
REQUIREMENT	Requirement 8: `/req/background/void-color-definition`
TEST PURPOSE	Verify that the implementation supports the `void-color` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving a map without `void-color` parameter, with `void-color` using a hexadecimal value and with `void-color` using a W3C Web Color name **Then:** - assert that the map operation supports a `void-color` parameter which can be a hexadecimal red-green-blue color value (from 00 to FF, FF representing 255) for the parts of the map outside of the valid areas of the projection / CRS. For a six-digit hexadecimal value, the first and second digits specify the intensity of red. The third and fourth digits specify the intensity of green. The fifth and sixth digits specify the intensity of blue, - assert that the map operation supports a case-insensitive W3C web color name the parts of the map outside of the valid areas of the projection / CRS, - assert that if `void-color` is not specified, the same color value as for `bgcolor` (specified or default) is used for the parts of the map outside of the valid areas of the projection / CRS.

A.3.4. Abstract Test for Requirement `void-transparent` parameter definition

ABSTRACT TEST A.9	
IDENTIFIER	`/conf/background/void-transparent-definition`
REQUIREMENT	Requirement 9: `/req/background/void-transparent-definition`
TEST PURPOSE	Verify that the implementation supports the `void-transparent` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving a map for all combinations of (no `void-transparent` parameter, void-transparent=false,``void-transparent=true)` and with and without `void-color`parameter **Then:** - assert that the server interprets `void-transparent` as a Boolean indicating whether the parts of the map outside of the valid areas of the projection / CRS should be transparent, - assert that, if `void-transparent` is not specified, the server assumes the same value as for `transparent` (specified or default).

A.3.5. Abstract Test for Requirement Background Map Success

ABSTRACT TEST A.10	
IDENTIFIER	`/conf/background/map-success`
REQUIREMENT	Requirement 10: `/req/background/map-success`
TEST PURPOSE	Verify that the implementation's response for the map retrieval operation with a background color and/or transparent parameter is correct
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** for all combinations of (no `transparent` parameter, transparent=false,``transparent=true)` and (without `bgcolor`parameter, with `bgcolor`using a hexadecimal value and with `bgcolor`using a W3C Web Color name) **Then:** - assert that the color of the map in the areas with no data is exactly the one specified in the `bgcolor`, - assert that the color in parts of the map outside of the valid areas of the projection / CRS is the one specified by `void-color`, or otherwise default to the same as the background color (whether specified by `bgcolor` or default), - assert that the transparency setting in parts of the map outside of the valid areas of the projection / CRS is the one specified by `void-transparent`, or otherwise default to the same as the background transparency setting (whether specified by `transparent` or default),

ABSTRACT TEST A.10	
	- assert that, in case the output format allows it and in the absence of the `transparent` parameter (or if it is `false`), the opacity (alpha value) of the map in the areas with no data is exactly 100%, if `transparent` is `false` or 0% if `transparent` is `true` (if the renderer supports anti-aliasing, at the edges between data and no-data areas, the opacity is allowed to have a value between 0% and 100%).

A.4. Conformance Class "Collection Selection"

CONFORMANCE CLASS A.4	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/collections-selection`
REQUIREMENTS CLASS	Requirements class 4: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collections-selection`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.11: `/conf/collections-selection/collections-parameter` Abstract test A.12: `/conf/collections-selection/collections-response`

A.4.1. Abstract Test for Requirement `collections` parameter definition

ABSTRACT TEST A.11	
IDENTIFIER	`/conf/collections-selection/collections-parameter`
REQUIREMENT	Requirement 11: `/req/collections-selection/collections-parameter`
TEST PURPOSE	Verify that the implementation supports the `collections` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core and that is understood to consist of multiple collections (e.g., a dataset advertising support for Dataset Map and featuring multiple collections) **When:** retrieving a map using the `collections` parameter with one and multiple *collectionsIds* **Then:** - assert that an operation that acts on a resource consisting of multiple geospatial data sub-resources (e.g., a resource derived from a root dataset) supports an optional parameter `collections` as an array of comma-separated collection id strings,

ABSTRACT TEST A.11	
	- assert that the parameter `collections` is supported by maps originating from resources consisting of multiple geospatial data sub-resources that can be addressed by identifiers (e.g. dataset map at `{datasetAPI}/maps/`), - assert that implementations support both a comma-separated list of geospatial resource identifiers (e.g., collectionId's) and full URLs to local geospatial resources.

A.4.2. Abstract Test for Requirement Collection Selection Response

ABSTRACT TEST A.12	
IDENTIFIER	`/conf/collections-selection/collections-response`
REQUIREMENT	Requirement 12: `/req/collections-selection/collections-response`
TEST PURPOSE	Verify that the implementation responds correctly to map requests using the `collections` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core and that is understood to consist of multiple collections (e.g., a dataset advertising support for Dataset Map and featuring multiple collections) **When:** retrieving a map using the `collections` parameter with one and multiple *collectionsIds* **Then:** - assert that only collections of geospatial data enumerated in the values of the `collections` parameter are used to generate the responses for the resource (map) to which they apply, - assert that if there is more than one collection name and the style applied does not specify otherwise, the collections are rendered in the result in an order starting with the first (leftmost) collection and ending with the last (rightmost).

A.5. Conformance Class "Scaling"

CONFORMANCE CLASS A.5	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/scaling`
REQUIREMENTS CLASS	Requirements class 5: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/scaling`
TARGET TYPE	Web API

CONFORMANCE CLASS A.5	
CONFORMANCE TESTS	Abstract test A.13: `/conf/scaling/width-definition` Abstract test A.14: `/conf/scaling/height-definition` Abstract test A.15: `/conf/scaling/scale-denominator-definition`

A.5.1. Abstract Test for Requirement `width` parameter definition

ABSTRACT TEST A.13	
IDENTIFIER	`/conf/scaling/width-definition`
REQUIREMENT	Requirement 13: `/req/scaling/width-definition`
TEST PURPOSE	Verify that the implementation supports the (scaling) `width` parameter correctly for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using `width` parameter for different values, as well as the same `bbox` parameter if spatial subsetting is supported, with and without `height` parameter, with and without `mm-per-pixel` parameter if display resolution is supported **Then:** - assert that the `width` value is interpreted as the horizontal size (columns) of the viewport where the response will be presented in pixel units (number of pixels), - assert that an HTTP 4xx error is returned if the `width` number is not a positive integer number, - assert that an error is returned if the value of the `width` exceeds the `maxWidth` property specified in the `x-OGC-limits.maps` object included in the service metadata, - assert that an HTTP 4xx error is returned if the value of the `width` (specified or calculated) times `height` (specified or calculated) exceeds a `maxPixels` property from a `x-OGC-limits.maps` object included in the service metadata, - assert that an HTTP 4xx error be returned if the `width` parameter is used together with the `bbox` (or `subset` for spatial dimensions) as well as the `scale-denominator` parameter, - assert that an HTTP 4xx error is returned if the `width` parameter is used together with the `scale-denominator` parameter and the implementation does not also support the “Subsetting” requirements class, - assert that, when the `width` parameter is omitted, the implementation uses an appropriate width which accurately reflects the default or requested scale established as the ratio between the horizontal dimension of the viewport and the corresponding size of the physical world, specifically for the local subset (bounding box) of the map being returned, and taking into consideration the default (0.28 mm/pixel) or specified display resolution (`mm-per-pixel`).

A.5.2. Abstract Test for Requirement `height` parameter definition

ABSTRACT TEST A.14	
IDENTIFIER	`/conf/scaling/height-definition`
REQUIREMENT	Requirement 14: `/req/scaling/height-definition`
TEST PURPOSE	Verify that the implementation supports responds the (scaling) `height` parameter correctly for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using `height` parameter for different values, as well as the same `bbox` parameter if spatial subsetting is supported, with and without `width` parameter, with and without `mm-per-pixel` parameter if display resolution is supported **Then:** - assert that the `height` value is interpreted as the vertical size (rows) of the viewport where the response will be presented in pixel units (number of pixels), - assert that an HTTP 4xx error is returned if the `height` value is not a positive integer number, - assert that an HTTP 4xx error is returned if the value of the `height` exceeds the `maxHeight` property specified in the `x-OGC-limits.maps` object included in the service metadata, - assert that an HTTP 4xx error is returned if the value of the `width` (specified or calculated) times `height` (specified or calculated) exceeds a `maxPixels` property from a `x-OGC-limits.maps` object included in the service metadata, - assert that an HTTP 4xx error is returned if the `height` parameter is used together with the `bbox` (or `subset` for spatial dimensions) as well as the `scale-denominator` parameter, - assert that an HTTP 4xx error is returned if the `height` parameter is used together with the `scale-denominator` parameter and the implementation does not also support the “Subsetting” requirements class, - assert that, when the `height` parameter is omitted, the implementation uses an appropriate height which accurately reflects the default or requested scale established as the ratio between the vertical dimension of the viewport and the corresponding size of the physical world, specifically for the local subset (bounding box) of the map being returned.

A.5.3. Abstract Test for Requirement `scale-denominator` parameter definition

ABSTRACT TEST A.15	
IDENTIFIER	`/conf/scaling/scale-denominator-definition`
REQUIREMENT	Requirement 15: `/req/scaling/scale-denominator-definition`
TEST PURPOSE	Verify that the implementation supports the `scale-denominator` parameter correctly for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `scale-denominator` parameter, combining all possibilities with and without `width` and/or `height` parameters, with and without `bbox` and `center` parameter if

ABSTRACT TEST A.15	
	spatial subsetting is supported, with and without `mm-per-pixel` parameter if display resolution is supported **Then:** - assert that the `scale-denominator` value is interpreted as the number of real-world units corresponding to one of the same unit on the map (as printed or displayed), considering only the local subset of the map being returned, based on the selected (e.g., from display resolution requirements class) or default (0.28 mm/pixel) display resolution, - assert that the implementation establishes the correspondence between real-world units and pixel units based on the equation: *physicalMetersPerPixel*= (`mm-per-pixel` / 1000 mm/m) * `scale-denominator`, where the *physicalMetersPerPixel* are not necessarily the same as the CRS units (even if those units are expressed in meters) for the region of that CRS consisting of the map subset being returned, - assert that an HTTP 4xx error is returned if the `scale-denominator` parameter is used together with `width` and/or `height` and the implementation does not declare conformance to the *spatial subsetting* requirements class (which specifies that the `width` and `height` parameters can also take on a subsetting role), - assert that an HTTP 4xx error is returned if the `scale-denominator` parameter is used together with `width` and/or `height` as well as a `bbox` (or equivalent `subset` parameter for a spatial dimension), - assert that, if the `scale-denominator` parameter is omitted, the implementation computes it as needed (for purposes such as applying scale-dependent symbology rules) based on the default or selected dimensions, display resolution, and the spatial subset of the map to return, - assert that, for implementations also supporting “Subsetting”, when the spatial subset of the map is not specified in the request, the `scale-denominator` value (default or specified) is used to compute this bounding box, taking into consideration the display resolution as well as the default or specified dimensions.

A.6. Conformance Class “Display Resolution”

CONFORMANCE CLASS A.6	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/display-resolution`
REQUIREMENTS CLASS	Requirements class 6: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/display-resolution`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.16: `/conf/display-resolution/mm-per-pixel-definition` Abstract test A.17: `/conf/display-resolution/map-success`

A.6.1. Abstract Test for Requirement `mm-per-pixel` parameter definition

ABSTRACT TEST A.16	
IDENTIFIER	`/conf/display-resolution/mm-per-pixel-definition`
REQUIREMENT	Requirement 16: `/req/display-resolution/mm-per-pixel-definition`
TEST PURPOSE	Verify that the implementation supports the `mm-per-pixel` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `mm-per-pixel` parameter, for different styles if styled maps are supported, combining all possibilities with and without `width` and/or `height` parameters, with and without `bbox` and `center` parameter if spatial subsetting is supported, with and without `mm-per-pixel` parameter if display resolution is supported **Then:** - assert that the implementation interprets `mm-per-pixel` as the size (in millimeters) of a rendering device pixel, - assert that an HTTP 4xx error is returned if the `mm-per-pixel` value is not a positive number, - assert that, if the parameter `mm-per-pixel` is not provided, the server assumes that the pixel size is 0.28 millimeters (90.71 pixels per inch).

A.6.2. Abstract Test for Requirement Display Resolution Map Success

ABSTRACT TEST A.17	
IDENTIFIER	`/conf/display-resolution/map-success`
REQUIREMENT	Requirement 17: `/req/display-resolution/map-success`
TEST PURPOSE	Verify that the implementation responds correctly to map requests using the `mm-per-pixel` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `mm-per-pixel` parameter, for different styles if styled maps are supported, combining all possibilities with and without `width` and/or `height` parameters, with and without `bbox` and `center` parameter if spatial subsetting is supported, with and without `mm-per-pixel` parameter if display resolution is supported **Then:** - assert that for an implementation supporting the Maps API *scaling* requirements class, the implementation uses the `mm-per-pixel` value instead of the default 0.28 mm/pixel when establishing the relationship between the dimensions of the output image, the scale and the spatial extent of the map,

ABSTRACT TEST A.17	
	- assert that the `mm-per-pixel` value is used instead of the default 0.28 mm/pixel when establishing scale for the purpose of applying styling and symbology rules to the map. For example, this needs to be considered for scale-dependent rule selectors as well as for graphical units in real world units (e.g., meters) or display units (e.g., millimeters).

A.7. Conformance Class "Spatial Subsetting"

CONFORMANCE CLASS A.7	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/spatial-subsetting`
REQUIREMENTS CLASS	Requirements class 7: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/spatial-subsetting`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.18: `/conf/spatial-subsetting/bbox-crs` Abstract test A.19: `/conf/spatial-subsetting/subset-crs` Abstract test A.20: `/conf/spatial-subsetting/center-crs` Abstract test A.21: `/conf/spatial-subsetting/bbox-definition` Abstract test A.22: `/conf/spatial-subsetting/subset-definition` Abstract test A.23: `/conf/spatial-subsetting/subset-response` Abstract test A.24: `/conf/spatial-subsetting/center-definition` Abstract test A.25: `/conf/spatial-subsetting/width-height` Abstract test A.26: `/conf/spatial-subsetting/map-success`

A.7.1. Abstract Test for Requirement `bbox-crs` parameter definition

ABSTRACT TEST A.18	
IDENTIFIER	`/conf/spatial-subsetting/bbox-crs`
REQUIREMENT	Requirement 18: `/req/spatial-subsetting/bbox-crs`
TEST PURPOSE	Verify that the implementation supports the `bbox-crs` parameter for specifying the CRS of the `bbox` parameter correctly
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core

ABSTRACT TEST A.18	
	When: retrieving maps using `bbox` and `bbox-crs` parameters for different values, as well as different values for the `crs` parameter if supported and applicable, **Then:** - assert that the map retrieval operation supports a query parameter `bbox-crs` with a string, - assert that for Earth centric data, the implementation supports https://www.opengis.net/def/crs/OGC/1.3/CRS84 as a value, - assert that if the bbox-crs is not indicated https://www.opengis.net/def/crs/OGC/1.3/CRS84 is assumed, - assert that if the storage (native) CRS is known, the storage CRS is supported as a value. Other conformance classes may allow additional values (see `crs` parameter definition), - assert that the CRS expressed as URIs or as safe CURIEs is supported, - assert that if the `bbox` parameter is not used, the `bbox-crs` is ignored.

A.7.2. Abstract Test for Requirement `subset-crs` parameter definition

ABSTRACT TEST A.19	
IDENTIFIER	`/conf/spatial-subsetting/subset-crs`
REQUIREMENT	Requirement 19: `/req/spatial-subsetting/subset-crs`
TEST PURPOSE	Verify that the implementation supports the `subset-crs` parameter for specifying the CRS of the `subset` parameter correctly
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using `subset` and `subset-crs` parameters for different values (using the correct spatial axes), as well as different values for the `crs` parameter if supported and applicable, **Then:** - assert that the map operation supports a parameter `subset-crs` with a string, - assert that for Earth centric data, https://www.opengis.net/def/crs/OGC/1.3/CRS84 is supported as a value, - assert that if the `subset-crs` is not indicated, https://www.opengis.net/def/crs/OGC/1.3/CRS84 is assumed, - assert that if the storage (native) CRS is known, the storage CRS is supported as a value. Other requirements classes may allow additional values (see crs parameter definition), - assert that CRS expressed as URIs or as safe CURIEs is supported, - assert that if no `subset` parameter referring to an axis of the CRS is used, the `subset-crs` is ignored.

A.7.3. Abstract Test for Requirement `center-crs` parameter definition

ABSTRACT TEST A.20	
IDENTIFIER	`/conf/spatial-subsetting/center-crs`
REQUIREMENT	Requirement 20: `/req/spatial-subsetting/center-crs`
TEST PURPOSE	Verify that the implementation supports the `center-crs` parameter for specifying the CRS of the `center` parameter correctly
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using `center` and `center-crs` parameters for different values, as well as different values for the `crs` parameter if supported and applicable, **Then:** - assert that the map retrieval operation supports a parameter `center-crs` with a string, - assert that for Earth centric data, https://www.opengis.net/def/crs/OGC/1.3/CRS84 is supported as a value, - assert that if the `center-crs` is not used, https://www.opengis.net/def/crs/OGC/1.3/CRS84 is assumed, - assert that if the storage (native) CRS is known, the storage CRS is supported as a value, - assert that CRS expressed as URIs or as safe CURIEs are supported, - assert that if no `center` parameter is used, the `center-crs` is ignored.

A.7.4. Abstract Test for Requirement `bbox` parameter definition

ABSTRACT TEST A.21	
IDENTIFIER	`/conf/spatial-subsetting/bbox-definition`
REQUIREMENT	Requirement 21: `/req/spatial-subsetting/bbox-definition`
TEST PURPOSE	Verify that the implementation supports the `bbox` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `bbox` parameter (with and without the `bbox-crs` parameter), **Then:** - assert that the map operation supports a parameter `bbox` with a comma-separated list of four or six floating point numbers. If the bounding box consists of six numbers, the first three numbers are the coordinates of the lower bound corner of a three-dimensional bounding box and the last three are the coordinates of the upper bound corner. The axis order is determined by the `bbox-crs` parameter value or longitude and latitude if the parameter is missing (https://www.opengis.net/def/crs/OGC/1.3/CRS84 axis order for a 2D bounding box, https://www.opengis.net/def/crs/OGC/1.3/CRS84h for a 3D bounding box). For example in https://www.opengis.net/def/crs/OGC/1.3/CRS84 the order is left_long, lower_lat, right_long, upper_lat,

ABSTRACT TEST A.21	
	- assert that if the `bbox` parameter is used together with the `center` and/or with a `subset` parameter including any of the dimensions corresponding to those of the map bounding box, the server returns a 4xx client error.

A.7.5. Abstract Test for Requirement spatial subsetting `subset` parameter definition

ABSTRACT TEST A.22	
IDENTIFIER	`/conf/spatial-subsetting/subset-definition`
REQUIREMENT	Requirement 22: `/req/spatial-subsetting/subset-definition`
TEST PURPOSE	Verify that the implementation supports the `subset` parameter for spatial subsetting
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `subset` parameter (with and without the `subset-crs` parameter, for the correct spatial axes), **Then:** - assert that the axis names `Lat` and `Lon` are supported for geographic CRS and `E` and `N` for projected CRS, which are to be interpreted as the best matching spatial axis in the CRS definition, - assert that if a third spatial dimension is supported (if the resource's spatial extent bounding box is three dimensional), the implementation also supports a `h` dimension. This is the elevation above the ellipsoid in EPSG:4979 or CRS84h for geographic CRS and `z` for projected CRS, which are interpreted as the vertical axis in the CRS definition, - assert that a 4xx error status code is returned if an axis name in the subset parameter value does not correspond to one of the axes of the subsetting CRS, is not an alias recommended to be supported (`lat`, `Latitude`, `latitude`, `lon`, `long`, `Long`, `Longitude`, `longitude`, `e`, `easting`, `Easting`, `x`, `X`, `n`, `y`, `Y`, `Northing`, `northing`, `z`, `Z`, `H`), and is not defined in the context of another supported requirements class, - assert that if the *interval* values fall entirely outside the range of valid values defined for the identified axis, a 204 or 404 status code is returned, - assert that, for a CRS where an axis can wrap around, such as subsetting across the dateline (anti-meridian) in a geographic CRS, a *low* value greater than *high* is supported to indicate an extent crossing that wrapping point, - assert that coordinates are interpreted as values for the named axis of the CRS specified in the `subset-crs` parameter value or in https://www.opengis.net/def/crs/OGC/1.3/CRS84 (https://www.opengis.net/def/crs/OGC/1.3/CRS84h for vertical dimension) if the `subset-crs` parameter is missing, - assert that if the `subset` parameter including any of the dimensions corresponding to those of the map bounding box is used with a `bbox` and/or `center` parameter, the server returns a 4xx error, - assert that multiple `subset` parameters are interpreted, as if all dimension subsetting values were provided in a single `subset` parameter (comma separated).

A.7.6. Abstract Test for Requirement map subset response

ABSTRACT TEST A.23	
IDENTIFIER	`/conf/spatial-subsetting/subset-response`
REQUIREMENT	Requirement 23: `/req/spatial-subsetting/subset-response`
TEST PURPOSE	Verify that the implementation responds correctly to map requests using the `subset` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `subset` (with and without the `subset-crs` parameter) **Then:** - assert that only the part of the resource that falls within the bounds of the subset interval and/or corresponds to the single point value is returned.

A.7.7. Abstract Test for Requirement `center` parameter definition

ABSTRACT TEST A.24	
IDENTIFIER	`/conf/spatial-subsetting/center-definition`
REQUIREMENT	Requirement 24: `/req/spatial-subsetting/center-definition`
TEST PURPOSE	Verify that the implementation supports the `center` parameter correctly
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `center` parameter (with and without the `center-crs` parameter), **Then:** - assert that a `center` parameter is supported to specify the center of the subset of the map to include, with coordinates in the CRS specified in the `center-crs` parameter value or in https://www.opengis.net/def/crs/OGC/1.3/CRS84 if the `center-crs` parameter is missing, - assert that if the `center` parameter is used together with the `bbox` and/or with a `subset` parameter including any of the dimensions corresponding to those of the map bounding box, the server returns a 4xx client error.

A.7.8. Abstract Test for Requirement subsetting `width` and `height` parameters definition

ABSTRACT TEST A.25	
IDENTIFIER	`/conf/spatial-subsetting/width-height`
REQUIREMENT	Requirement 25: `/req/spatial-subsetting/width-height`
TEST PURPOSE	Verify that the implementation supports the `width` and `height` parameter for spatial subsetting when used together with the `center` and/or the `scale-denominator` parameters
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `center` parameter together, with the `width` and/or `height` (with and without the `center-crs` parameter), with and without the `scale-denominator` parameter if scaling is supported **Then:** - assert that when the `center` parameter and/or the `scale-denominator` parameter is used, or if the *scaling* conformance class is not supported, a `width` and `height` parameter specifying the subset of the map to return around the specified or default center of the map is supported, - assert that the scale of the map is considered whether returning the map at a native scale or resampled (e.g., using the *scaling* conformance class `scale-denominator` parameter), as well as the display resolution (either the default 0.28 mm/pixel, or the one specified by the `mm-per-pixel` parameter of the *display resolution* conformance class).

A.7.9. Abstract Test for Requirement map subset success

ABSTRACT TEST A.26	
IDENTIFIER	`/conf/spatial-subsetting/map-success`
REQUIREMENT	Requirement 26: `/req/spatial-subsetting/map-success`
TEST PURPOSE	Verify that the implementation responds correctly to map requests using subsetting parameters (`bbox`, `subset` or `center`)
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `bbox` (with and without the `bbox-crs` parameter), `subset` (with and without the `subset-crs` parameter), and `center` parameter (with and without the `center-crs` parameter, with the `width` and/or `height` parameter, with and without the `scale-denominator` parameter if scaling is supported **Then:** - assert that the content of the response represents elements inside or intersecting with the spatial extent of the geographical area of the map identified with the subsetting coordinates.

A.8. Conformance Class “Date and Time”

CONFORMANCE CLASS A.8	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/datetime`
REQUIREMENTS CLASS	Requirements class 8: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/datetime`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.27: `/conf/datetime/datetime-definition` Abstract test A.28: `/conf/datetime/datetime-response` Abstract test A.29: `/conf/datetime/subset-definition` Abstract test A.30: `/conf/datetime/subset-response` Abstract test A.31: `/conf/datetime/axis` Abstract test A.32: `/conf/datetime/map-success`

A.8.1. Abstract Test for Requirement `datetime` parameter definition

ABSTRACT TEST A.27	
IDENTIFIER	`/conf/datetime/datetime-definition`
REQUIREMENT	Requirement 27: `/req/datetime/datetime-definition`
TEST PURPOSE	Verify that the implementation supports the `datetime` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `datetime` parameter **Then:** - assert that the implementation supports an `instant` defined as specified by RFC 3339, 5.6, with the exception that the server is only required to support the `Z` UTC time notation, and not required to support local time offsets, - assert that time intervals unbounded at the start or end are supported using a double-dot (`..`) or an empty string for the start/end.

A.8.2. Abstract Test for Requirement `datetime` parameter response

ABSTRACT TEST A.28	
IDENTIFIER	`/conf/datetime/datetime-response`
REQUIREMENT	Requirement 28: `/req/datetime/datetime-response`
TEST PURPOSE	Verify that the implementation responds correctly to map requests using the `datetime` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `datetime` parameter **Then:** - assert that if the `datetime` parameter is provided by the client and supported by the server, then only resources that have a temporal geometry that intersects the temporal information in the `datetime` parameter are part of the result set, - assert that using a `datetime` parameter all resources that does not have a temporal information associate are present in the map.

A.8.3. Abstract Test for Requirement temporal `subset` parameter definition

ABSTRACT TEST A.29	
IDENTIFIER	`/conf/datetime/subset-definition`
REQUIREMENT	Requirement 29: `/req/datetime/subset-definition`
TEST PURPOSE	Verify that the implementation supports temporal subsetting using the `subset` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `subset` parameter with the `time` axis **Then:** - assert that the implementation supports a `subset` parameter, consisting of an axis name (`time`) followed by parentheses within which can be specified an interval consisting of two “time coordinate”s separated by a colon (`:`), or a single “time coordinate”, where a “time coordinate” consists of a single number, a string or an asterisk (`\*`), - assert that the implementation supports an axis name `time`, - assert that the implementation returns a 4xx status code if the axis name is not `time` (or its recommended aliases `Time`, `t` and `T`) and is not recognized in the context of another requirements class, - assert that if “time coordinates” fall entirely outside the range of valid values defined for the identified axis, a 204 or 404 status code is returned, - assert that “time coordinates” are interpreted as values for the CRS specified in the temporal extent, or Gregorian UTC time if it is not specified in the temporal extent, - assert that when Gregorian UTC time is used, the implementation supports time expressed using RFC 3339 section 5.6, with only support for the UTC (`Z`) notation required (meaning that support

ABSTRACT TEST A.29	
	for local time offsets should not be tested), as well as the following additional partial date and time formats: `yyyy`, `yyyy-mm`, `yyyy-mm-dd`, `yyyy-mm-ddThhZ`, `yyyy-mm-ddThh:mmZ`, - assert that the implementation supports a `*` value indicating the earliest available time (for `low`) or the latest available time (for `high` and when used as a single time instant), - assert that multiple `subset` parameters is interpreted as if all dimension subsetting values were provided in a single `subset` parameter (comma separated).

A.8.4. Abstract Test for Requirement temporal subset response

ABSTRACT TEST A.30	
IDENTIFIER	`/conf/datetime/subset-response`
REQUIREMENT	Requirement 30: `/req/datetime/subset-response`
TEST PURPOSE	Verify that the implementation responds correctly to temporal subsetting requests using the `subset` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `subset` parameter with the `time` axis **Then:** - assert that only the part of the resource that falls within the bounds of the subset interval and/or corresponds to the single point value is returned.

A.8.5. Abstract Test for Requirement temporal axis

ABSTRACT TEST A.31	
IDENTIFIER	`/conf/datetime/axis`
REQUIREMENT	Requirement 31: `/req/datetime/axis`
TEST PURPOSE	Verify that the implementation supports the `time` axis for temporal subsetting using the `subset` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `subset` parameter with the `time` axis **Then:** - assert that to subset a generic time dimension, the server supports `time` as axis name in the `subset` parameter.

A.8.6. Abstract Test for Requirement temporal subsetting success

ABSTRACT TEST A.32	
IDENTIFIER	`/conf/datetime/map-success`
REQUIREMENT	Requirement 32: `/req/datetime/map-success`
TEST PURPOSE	Verify that the implementation responds correctly to temporal subsetting requests
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `subset` parameter with the `time` axis **Then:** - assert that the content of that response is consistent with the requested datetime.

A.9. Conformance Class "General Subsetting"

CONFORMANCE CLASS A.9	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/general-subsetting`
REQUIREMENTS CLASS	Requirements class 9: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/general-subsetting`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.33: `/conf/general-subsetting/uniform-additional-dimensions` Abstract test A.34: `/conf/general-subsetting/subset-definition`

A.9.1. Abstract Test for Requirement uniform additional dimensions

ABSTRACT TEST A.33	
IDENTIFIER	`/conf/general-subsetting/uniform-additional-dimensions`
REQUIREMENT	Requirement 33: `/req/general-subsetting/uniform-additional-dimensions`

ABSTRACT TEST A.33	
TEST PURPOSE	Verify that the implementation describes additional dimensions in a uniform manner
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core for which an extent description is available (e.g., conforming successfully to either the “Collection Map” or “Dataset Map”) **When:** retrieving the description of the data resource e.g., the collection (for “Collection Map”) or the landing page (for “Dataset Map”) **Then:** - assert that the extent of any additional dimension(s) beyond temporal and spatial is described in a way similar to the temporal dimension, using the name of the dimension as a key and an object as value, with that object containing an `interval` property, a URI for the semantic `definition`, a `unit` of measure if applicable, and a `grid` definition if applicable, as specified in *extent-uad.yaml*.

A.9.2. Abstract Test for Requirement general subsetting `subset` parameter

ABSTRACT TEST A.34	
IDENTIFIER	`/conf/general-subsetting/subset-definition`
REQUIREMENT	Requirement 34: `/req/general-subsetting/subset-definition`
TEST PURPOSE	Verify that the implementation supports general subsetting using the `subset` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps using the `subset` parameter for an additional dimension besides space and time **Then:** - assert that the implementation supports a `subset` parameter, consisting of an axis name followed by an interval or single value within parentheses, where an interval is separated by a colon (`:`), - assert that all additional dimensions described in the extent of the collection are supported as axis name, - assert that a 4xx error status code is returned if an axis name not corresponding to the name of one of the *additional* dimensions in the extent of the collection, and not corresponding to the required or recommended aliases in the spatial subsetting and temporal subsetting requirements classes is used, - assert that when the *intervalOrSingle* values fall entirely outside the range of valid values defined for the identified axis, a 204 or 404 status code is returned, - assert that for an axis that can wrap around, a *low* value greater than *high* are supported to indicate an extent crossing that wrapping point, - assert that multiple subset parameters are interpreted as if all dimension subsetting values were provided in a single `subset` parameter (comma separated).

A.10. Conformance Class “Coordinate Reference System”

CONFORMANCE CLASS A.10	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/crs`
REQUIREMENTS CLASS	Requirements class 10: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/crs`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.35: `/conf/crs/crs-definition` Abstract test A.36: `/conf/crs/map-success`

A.10.1. Abstract Test for Requirement `crs` parameter definition

ABSTRACT TEST A.35	
IDENTIFIER	`/conf/crs/crs-definition`
REQUIREMENT	Requirement 35: `/req/crs/crs-definition`
TEST PURPOSE	Verify that the implementation supports the output `crs` parameter for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps with the `crs` parameter for different available CRS and without **Then:** - assert that the map operation supports a `crs` string parameter, - assert that all CRSs listed in the collection (or collections) description are supported, or that `[OGC:CRS84]` is supported if no list of CRS is available, - assert that, if the spatial subsetting requirements class is also supported, the `bbox-crs` and the `subset-crs` also support those available CRS values when the same value is also specified in the `crs` parameter, - assert that the CRS values can be expressed either as URIs or as safe CURIEs.

A.10.2. Abstract Test for Requirement CRS map success

ABSTRACT TEST A.36	
IDENTIFIER	`/conf/crs/map-success`
REQUIREMENT	Requirement 36: `/req/crs/map-success`
TEST PURPOSE	Verify that the implementation responds correctly to map requests using the `crs` parameter
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps with the `crs` parameter for different available CRS and without **Then:** - assert that the content of the map response is consistent with the requested CRS.

A.11. Conformance Class “Orientation”

CONFORMANCE CLASS A.11	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/orientation`
REQUIREMENTS CLASS	Requirements class 11: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/orientation`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.37: `/conf/orientation/orientation` Abstract test A.38: `/conf/orientation/response-headers`

A.11.1. Abstract Test for Requirement `orientation` parameter

ABSTRACT TEST A.37	
IDENTIFIER	`/conf/orientation/orientation`
REQUIREMENT	Requirement 37: `/req/orientation/orientation`
TEST PURPOSE	Verify that the implementation supports the `orientation` parameter correctly for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps with the `orientation` parameter for different values and without **Then:**

ABSTRACT TEST A.37	
	- assert that an orientation parameter that specifies the amount by which to rotate a map is supported , expressed as counterclockwise degrees, resulting in the viewing perspective being rotated by that same orientation in a clockwise direction, - assert that when the `orientation` parameter is not specified, a zero orientation value is be assumed, - assert that the orientation is applied to the map with the center of the selected spatial subset as the pivot point, or the center of the map if none is specified, - assert that if an `orientation` parameter is used together with `subset` or `bbox` spatial subsetting parameter, the counterclockwise orientation is applied to the four corners of the clipping box associated to that subset, as if the equivalent `center`, `width` and `height` spatial subsetting query parameters were used instead, avoiding leaving empty corners in the final rotated map image.

A.11.2. Abstract Test for Requirement orientation response headers

ABSTRACT TEST A.38	
IDENTIFIER	`/conf/orientation/response-headers`
REQUIREMENT	Requirement 38: `/req/orientation/response-headers`
TEST PURPOSE	Verify that the implementation includes the correct response headers for map requests using the `orientation` parameter.
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving maps with the `orientation` parameter for different values and without **Then:** - assert that for responses to a map request where the `orientation` query parameter is used, a response header `Content-Orientation: [value in decimal degrees]` corresponding to the orientation of the map is returned, - assert that for responses to a map request where the `orientation` query parameter is used, the `Content-Bbox` response header reflects the bounding box in the map output CRS prior to the orientation being applied.

A.12. Conformance Class “Custom Projection CRS”

CONFORMANCE CLASS A.12	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/projection`
REQUIREMENTS CLASS	Requirements class 12: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/projection`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.39: `/conf/projection/crs-proj-method` Abstract test A.40: `/conf/projection/crs-proj-params` Abstract test A.41: `/conf/projection/crs-proj-center-definition` Abstract test A.42: `/conf/projection/crs-datum` Abstract test A.43: `/conf/projection/response-headers` Abstract test A.44: `/conf/projection/projections-resource` Abstract test A.45: `/conf/projection/projections-response`

A.12.1. Abstract Test for Requirement `crs-proj-method` parameter

ABSTRACT TEST A.39	
IDENTIFIER	`/conf/projection/crs-proj-method`
REQUIREMENT	Requirement 39: `/req/projection/crs-proj-method`
TEST PURPOSE	Verify that the implementation supports the `crs-proj-method` parameter correctly for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to Conformance class A.1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core` and passing Abstract test A.45: `/conf/projection/projections-response` **When:** retrieving maps with the `crs-proj-method` parameter for different available values as listed in `/projectionsAndDatums` **Then:** - assert that a `crs-proj-method` parameter supporting selection of a projection operation method is supported, - assert that CURIEs are supported in addition to URIs to specify the projection method.

A.12.2. Abstract Test for Requirement `crs-proj-params` parameter

ABSTRACT TEST A.40	
IDENTIFIER	`/conf/projection/crs-proj-params`

ABSTRACT TEST A.40	
REQUIREMENT	Requirement 40: `/req/projection/crs-proj-params`
TEST PURPOSE	Verify that the implementation supports the `crs-proj-params` parameter correctly for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to Conformance class A.1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core` and passing Abstract test A.45: `/conf/projection/projections-response` **When:** retrieving maps with the `crs-proj-method` parameter for different available values and different values of the associated method parameters (specified using the `crs-proj-params` query parameter) as listed in `/projectionsAndDatums` **Then:** - assert that a `crs-proj-params` parameter is supported that enables selection of one or more values for operation method parameters, with values in between parentheses `( )` following the URI of a projection parameter, and different parameters separated by value (e.g., `crs-proj-params=[epsg-parameter:8823](40),[epsg-parameter:8824](90)`), - assert that in addition to CURIEs, URIs are also supported to specify the projection parameters.

A.12.3. Abstract Test for Requirement `crs-proj-center` parameter

ABSTRACT TEST A.41	
IDENTIFIER	`/conf/projection/crs-proj-center-definition`
REQUIREMENT	Requirement 41: `/req/projection/crs-proj-center-definition`
TEST PURPOSE	Verify that the implementation supports the `crs-proj-center` parameter correctly for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to Conformance class A.1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core` and passing Abstract test A.45: `/conf/projection/projections-response` **When:** retrieving maps with the `crs-proj-method` parameter for different available values as listed in `/projectionsAndDatums` and the `crs-proj-center` parameter for different values **Then:** - assert that a `crs-proj-center` parameter of the form `Lat(centerLat),Lon(centerLon)` is supported to facilitate the selection of the most relevant projection parameters to center a custom projection, - assert that the projection center `Lat` value is mapped to the first matching operation method parameter available for the selected operation method of the projection query parameter, in the epsg-parameter order 8832, 8823, 8801, 8811, - assert that the projection-center `Lon` value is mapped to the first matching operation method parameter available for the selected operation method of the projection query parameter, in the epsg-parameter order 8802, 8812, 8833.

A.12.4. Abstract Test for Requirement `crs-datum` parameter

ABSTRACT TEST A.42	
IDENTIFIER	`/conf/projection/crs-datum`
REQUIREMENT	Requirement 42: `/req/projection/crs-datum`
TEST PURPOSE	Verify that the implementation supports the `crs-datum` parameter correctly for map requests
TEST METHOD	**Given:** a map resource that conformed successfully to Conformance class A.1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core` and passing Abstract test A.45: `/conf/projection/projections-response` **When:** retrieving maps with the `crs-datum` parameter for different available values as listed in `/projectionsAndDatums` **Then:** - assert that a `crs-datum` parameter as a URI allowing to select a datum for the output CRS is supported, - assert that CURIEs are supported in addition to URIs to specify the datum parameter, - assert that if a `crs-datum` parameter is not specified, the native (storage) CRS datum is assumed (the WGS84 ensemble `[epsg-datum:6326]` datum is assumed if the native CRS is not declared).

A.12.5. Abstract Test for Requirement custom CRS projection response headers

ABSTRACT TEST A.43	
IDENTIFIER	`/conf/projection/response-headers`
REQUIREMENT	Requirement 43: `/req/projection/response-headers`
TEST PURPOSE	Verify that the implementation responds to map requests using the `crs-proj-method` parameter and/or `crs-datum` with the correct response headers
TEST METHOD	**Given:** a map resource that conformed successfully to Conformance class A.1: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/core` and passing Abstract test A.45: `/conf/projection/projections-response` **When:** retrieving maps with the `crs-proj-method` parameter for different available values, different values of the associated method parameters (using both `crs-proj-center` and `crs-proj-params`), and different values for `crs-datum` as listed in `/projectionsAndDatums` **Then:**

ABSTRACT TEST A.43	
	- assert that for responses to a map request where the `crs-proj-method` query parameter was used, a response header `Content-Crs-Method: <[URI]>` including the URI of the projection operation method is returned, - assert that a response header `Content-Crs-Method-Params: <URI>=[value]; ...` is returned, including the URI of the projection operation parameters and its value for each parameter specified using the `crs-proj-method` or `crs-proj-center` query parameters, - assert that for responses to a map request where the `crs-datum` query parameter was used, a response header `Content-Crs-Datum: <[URI]>` corresponding to the URI of the projection operation method is returned, - assert that for responses to a map request specifying the `crs-proj-method` query parameter, a `Content-Crs` response header is not included, - assert that for responses to a map request specifying the `crs-proj-method` query parameter, the CRS of the `Content-Bbox` response header coordinates is in the custom CRS defined by this operation method and its parameters.

A.12.6. Abstract Test for Requirement `/projectionsAndDatums` resource

ABSTRACT TEST A.44	
IDENTIFIER	`/conf/projection/projections-resource`
REQUIREMENT	Requirement 44: `/req/projection/projections-resource`
TEST PURPOSE	Verify that the implementation supports retrieving the list of available projection operation methods, their parameters, and the list of available datums at `/projectionsAndDatums`
TEST METHOD	**Given:** an API implementation being tested **When:** retrieving the `/projectionsAndDatums` resource **Then:** - assert that a GET operation at `/projectionsAndDatums` providing a JSON representation is supported.

A.12.7. Abstract Test for Requirement `/projectionsAndDatums` response

ABSTRACT TEST A.45	
IDENTIFIER	`/conf/projection/projections-response`
REQUIREMENT	Requirement 45: `/req/projection/projections-response`

ABSTRACT TEST A.45	
TEST PURPOSE	Verify that the implementation responds correctly to a request for the `/projectionsAndDatums` resource, conforming to the JSON schema and using the correct URIs
TEST METHOD	**Given:** an API implementation being tested passing Abstract test A.44: `/conf/projection/projections-resource` **When:** retrieving the `/projectionsAndDatums resource` **Then:** - assert that the implementation includes in its response for the `/projectionsAndDatums` resource the complete list of custom CRS projection operation methods supported for map retrieval operations, - assert that the implementation includes in its response for the `/projectionsAndDatums` resource the complete list of custom CRS datums supported for map retrieval operations, - assert that in the JSON representation, the list of supported projection operation methods is provided as a dictionary (associative array) value for a `methods` property associating operation method objects (including optional `title` and `description` properties) to the corresponding identifiers to be used as values for the `crs-proj-method` query parameter, - assert that these operation method identifiers are safe CURIEs when a registered URI exists for the method, - assert that in the JSON representation, the list of supported datums are provided as a dictionary (associative array) value for a `datums` property associating datum objects to the corresponding identifiers to be used as values for the `crs-datum` query parameter, - assert that these datum identifiers are safe CURIEs when a registered URI exists for the datum, - assert that the datum object includes an `ellipsoid` property specifying the safe CURIE for the associated ellipsoid and may contain additional optional `title` and `description` properties, - assert that in the JSON representation, the operation method objects include all valid parameters for that method as a dictionary (associative array) value for a `parameters` property to method parameters object (including optional `title` and `description` properties) to the corresponding identifiers to be used as values for the `crs-proj-params` query parameter, - assert that these method parameters are safe CURIEs when a registered URI exists for the parameter (to avoid repeating the same parameter, those objects may use a JSON pointer (`$ref`) to a top-level `parameters` property in the same custom projections JSON document), - assert that in the JSON representation, the operation method objects include `centerLatParam` and/or `centerLonParam` properties (as applicable) whose values are the identifiers corresponding to the parameters for which values specified for the `crs-proj-center` query parameter will be mapped, in a manner consistent with requirement `/req/projection/crs-proj-center-definition`.

A.13. Conformance Class “Collection Map”

CONFORMANCE CLASS A.13	
IDENTIFIER	https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/collection-map
REQUIREMENTS CLASS	Requirements class 13: https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/collection-map
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.46: /conf/collection-map/desc-links Abstract test A.47: /conf/collection-map/desc-crs Abstract test A.48: /conf/collection-map/map-operation

A.13.1. Abstract Test for Requirement collection description links

ABSTRACT TEST A.46	
IDENTIFIER	/conf/collection-map/desc-links
REQUIREMENT	Requirement 46: /req/collection-map/desc-links
TEST PURPOSE	Verify that the implementation links correctly from the collection description resource to the map resource
TEST METHOD	**Given:** a collection from an API implementation conforming to OGC API — Common — Part 2: Geospatial Data "Collections" conformance class **When:** retrieving the JSON representation of the description for that collection **Then:** - assert that the OGC API collection description includes a link with relation type https://www.opengis.net/def/rel/ogc/1.0/map (or [ogc-rel:map]) and the href pointing to a the map resource for this collection.

A.13.2. Abstract Test for Requirement collection description CRS

ABSTRACT TEST A.47	
IDENTIFIER	/conf/collection-map/desc-crs
REQUIREMENT	Requirement 47: /req/collection-map/desc-crs
TEST PURPOSE	Verify that the implementation describes the supported CRS correctly in its collection description resources

ABSTRACT TEST A.47	
TEST METHOD	**Given:** an API implementation conforming to OGC API — Common — Part 2: Geospatial Data "Collections" conformance class **When:** retrieving the JSON representation of the description for that collection **Then:** - assert that the `crs` property in the collection object of a geospatial collection contains URIs or safe CURIEs for the list of CRSs supported by the server for that collection, - assert that if the collection is available more efficiently (e.g., if it is stored in the server in that CRS) using a particular CRS (the native CRS, also *called storage CRS*) that is not https://www.opengis.net/def/crs/OGC/1.3/CRS84, a `storageCrs` property in the collection object of a geospatial collection is the URI or the safe CURIE for that CRS, - assert that if a `storageCrs` property is used and that is not https://www.opengis.net/def/crs/OGC/1.3/CRS84, an overall bounding box (and optional inner bounding boxes for sparse data) is provided in a `storageCrsBbox` property within the `spatial` dimension of the `extent` following the same schema as the CRS84 `bbox`.

A.13.3. Abstract Test for Requirement collection map operation

ABSTRACT TEST A.48	
IDENTIFIER	`/conf/collection-map/map-operation`
REQUIREMENT	Requirement 48: `/req/collection-map/map-operation`
TEST PURPOSE	Verify that the implementation supports retrieving maps from an OGC API collection resource as defined in the OGC API – Common Standard.
TEST METHOD	**Given:** a collection correctly linking to a map resource as per /conf/collection-map/desc-links **When:** retrieving a map for that collection resource as per /conf/core **Then:** - assert that every OGC API collection available as a map supports an HTTP GET operation to a URL `/collections/{collectionId}/map` to retrieve a map from that collection resource.

A.14. Conformance Class "Dataset Map"

CONFORMANCE CLASS A.14	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/dataset-map`

CONFORMANCE CLASS A.14	
REQUIREMENTS CLASS	Requirements class 14: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/dataset-map`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.49: `/conf/dataset-map/landingpage` Abstract test A.50: `/conf/dataset-map/desc-extent` Abstract test A.51: `/conf/dataset-map/desc-crs` Abstract test A.52: `/conf/dataset-map/operation`

A.14.1. Abstract Test for Requirement dataset landing page

ABSTRACT TEST A.49	
IDENTIFIER	`/conf/dataset-map/landingpage`
REQUIREMENT	Requirement 49: `/req/dataset-map/landingpage`
TEST PURPOSE	Verify that the implementation supports linking properly from an OGC API landing page to a map resource
TEST METHOD	**Given:** a dataset provided by an API implementation conforming to OGC API — Common — Part 1: Core **When:** retrieving the JSON representation of the landing page description for that dataset **Then:** - assert that the deployed API endpoint landing page includes a link with relation type `https://www.opengis.net/def/rel/ogc/1.0/map` (or `[ogc-rel:map]`) to the dataset map URL at `/map`.

A.14.2. Abstract Test for Requirement dataset description extent

ABSTRACT TEST A.50	
IDENTIFIER	`/conf/dataset-map/desc-extent`
REQUIREMENT	Requirement 50: `/req/dataset-map/desc-extent`
TEST PURPOSE	Verify that the implementation describes the extent of the dataset correctly from the landing page
TEST METHOD	**Given:** a dataset provided by an API conforming to OGC API — Common — Part 1: Core **When:** retrieving the JSON representation of the landing page description for that dataset **Then:**

ABSTRACT TEST A.50	
	- assert that an extent CRS is provided in an “extent” property of the API landing page following the same schema as the “extent” property for the collection (see OGC API — Common — Part 2: Geospatial Data).

A.14.3. Abstract Test for Requirement dataset description CRS

ABSTRACT TEST A.51	
IDENTIFIER	`/conf/dataset-map/desc-crs`
REQUIREMENT	Requirement 51: `/req/dataset-map/desc-crs`
TEST PURPOSE	Verify that the implementation describes the supported CRS correctly in its landing page resource
TEST METHOD	**Given:** a dataset provided by an API conforming to OGC API — Common — Part 1: Core **When:** retrieving the JSON representation of the landing page description for that dataset **Then:** - assert that the `crs` property in the landing page of a dataset contains URIs or safe CURIEs for the list of CRSs supported by the dataset as a whole, - assert that if the dataset is available more efficiently using a particular CRS that is not https://www.opengis.net/def/crs/OGC/1.3/CRS84, a `storageCrs` property in the landing page of a dataset is the URI or the safe CURIE for that CRS as a value, - assert that if a `storageCrs` property is used and that is not https://www.opengis.net/def/crs/OGC/1.3/CRS84, an overall bounding box (and optional inner bounding boxes for sparse data) is provided in a `storageCrsBbox` property within the `spatial` dimension of the `extent` following the same schema as the CRS84 `bbox`.

A.14.4. Abstract Test for Requirement dataset map operation

ABSTRACT TEST A.52	
IDENTIFIER	`/conf/dataset-map/operation`
REQUIREMENT	Requirement 52: `/req/dataset-map/operation`
TEST PURPOSE	Verify that the implementation supports retrieving a ‘dataset maps’ resource exposed by the OGC API — Maps implementation
TEST METHOD	**Given:** an OGC API dataset correctly linking to a map resource as per /conf/dataset-map/landingpage **When:** retrieving a map for that dataset resource as per /conf/core

ABSTRACT TEST A.52	
	Then: - assert that the implementation supports an HTTP GET operation for the `/map` URL to retrieve a map from the dataset API endpoint in the default style.

A.15. Conformance Class “Styled Map”

CONFORMANCE CLASS A.15	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/styled-map`
REQUIREMENTS CLASS	Requirements class 15: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/styled-map`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.53: `/conf/styled-map/desc-links` Abstract test A.54: `/conf/styled-map/map-operation`

A.15.1. Abstract Test for Requirement styled map links

ABSTRACT TEST A.53	
IDENTIFIER	`/conf/styled-map/desc-links`
REQUIREMENT	Requirement 53: `/req/styled-map/desc-links`
TEST PURPOSE	Verify that the implementation links correctly from a style resource to a map resource
TEST METHOD	**Given:** a list of styles provided by an API implementation conforming to OGC API — Styles **When:** retrieving the JSON representation of that list of styles **Then:** - assert that if the deployed API endpoint has a mechanism to expose links associated with styled geospatial resources (e.g., the OGC API — Styles list of styles at `/styles` for a dataset or at `/collections/{collectionId}/styles` for a collection), those styled resources include a link with link relation `https://www.opengis.net/def/rel/ogc/1.0/map` (or `[ogc-rel:map]`) and the href pointing to the map associated with that styled resource.

A.15.2. Abstract Test for Requirement styled map operation

ABSTRACT TEST A.54	
IDENTIFIER	`/conf/styled-map/map-operation`
REQUIREMENT	Requirement 54: `/req/styled-map/map-operation`
TEST PURPOSE	Verify that the implementation supports retrieving maps from *OGC API — Styles* style resources
TEST METHOD	**Given:** a style correctly linking to a map resource as per /conf/styled-map/desc-links **When:** retrieving a map for that style as per /conf/core **Then:** - assert that every resource for which a styled map is available supports an HTTP GET operation to a `.../styles/{styleId}/map` URL to retrieve a map for a particular style (e.g., `/collections/{collectionId}/styles/{styleId}` for a styled collection map or `/styles/{styleId}/map` for a styled dataset map).

A.16. Conformance Class “PNG”

CONFORMANCE CLASS A.16	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/png`
REQUIREMENTS CLASS	Requirements class 16: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/png`
TARGET TYPE	Web API
CONFORMANCE TEST	Abstract test A.55: `/conf/png/content`

A.16.1. Abstract Test for Requirement PNG map content

ABSTRACT TEST A.55	
IDENTIFIER	`/conf/png/content`
REQUIREMENT	Requirement 55: `/req/png/content`
TEST PURPOSE	Verify that the implementation supports retrieving maps negotiating for PNG content
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving a PNG (`image/png`) representation of a map resource through HTTP content negotiation **Then:** - assert that every 200-response of the server with the media type `image/png` is a PNG document representing only one map, - assert that the colors of the PNG represent the geospatial features or coverage values in the map, - assert that the alpha channel of the PNG is used when partial transparency is required, - assert that all maps representing parts of the same resource or resources and using the same style follow the same portrayal rules.

A.17. Conformance Class “JPEG”

CONFORMANCE CLASS A.17	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/jpeg`
REQUIREMENTS CLASS	Requirements class 17: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/jpeg`
TARGET TYPE	Web API
CONFORMANCE TEST	Abstract test A.56: `/conf/jpeg/content`

A.17.1. Abstract Test for Requirement JPEG map content

ABSTRACT TEST A.56	
IDENTIFIER	`/conf/jpeg/content`

ABSTRACT TEST A.56	
REQUIREMENT	Requirement 56: `/req/jpeg/content`
TEST PURPOSE	Verify that the implementation supports retrieving maps negotiating for JPEG content
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving a JPEG (`image/jpeg`) representation of a map resource through HTTP content negotiation **Then:** - assert that every 200-response of the server with the media type `image/jpeg` is a JPEG document representing only one map, - assert that the colors of the JPEG represent geospatial features and/or coverage values in the map, - assert that all maps representing parts of the same resource or resources and using the same style follow the same portrayal rules.

A.18. Conformance Class “JPEG XL”

CONFORMANCE CLASS A.18	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/jpegxl`
REQUIREMENTS CLASS	Requirements class 18: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/jpegxl`
TARGET TYPE	Web API
CONFORMANCE TEST	Abstract test A.57: `/conf/jpegxl/content`

A.18.1. Abstract Test for Requirement JPEG XL map content

ABSTRACT TEST A.57	
IDENTIFIER	`/conf/jpegxl/content`
REQUIREMENT	Requirement 57: `/req/jpegxl/content`

ABSTRACT TEST A.57	
TEST PURPOSE	Verify that the implementation supports retrieving maps negotiating for JPEG XL content
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving a JPEG XL (`image/jxl`) representation of a map resource through HTTP content negotiation **Then:** - assert that every 200-response of the server with the media type `image/jxl` is a JPEG XL file representing only one map, - assert that the JPEG XL is a color image representing the geospatial features or coverage values in the map, - assert that all maps representing parts of the same resource or resources and using the same style follow the same portrayal rules.

A.19. Conformance Class “TIFF”

CONFORMANCE CLASS A.19	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/tiff`
REQUIREMENTS CLASS	Requirements class 19: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/tiff`
TARGET TYPE	Web API
CONFORMANCE TEST	Abstract test A.58: `/conf/tiff/content`

A.19.1. Abstract Test for Requirement TIFF map content

ABSTRACT TEST A.58	
IDENTIFIER	`/conf/tiff/content`
REQUIREMENT	Requirement 58: `/req/tiff/content`
TEST PURPOSE	Verify that the implementation supports retrieving maps negotiating for TIFF and/or GeoTIFF content

ABSTRACT TEST A.58	
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving a TIFF (`image/tiff`) and GeoTIFF (`image/tiff; application=geotiff`) representation of a map resource through HTTP content negotiation **Then:** - assert that every 200-response of the server with the media type `image/tiff` is a TIFF document representing only one map, - assert that the TIFF file represents colors by using an image palette or RGB combination, - assert that all maps representing parts of the same resource or resources and using the same style follow the same portrayal rules or represent data with the same reference and units of measure.

A.20. Conformance Class “SVG”

CONFORMANCE CLASS A.20	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/svg`
REQUIREMENTS CLASS	Requirements class 20: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/svg`
TARGET TYPE	Web API
CONFORMANCE TEST	Abstract test A.59: `/conf/svg/content`

A.20.1. Abstract Test for Requirement SVG map content

ABSTRACT TEST A.59	
IDENTIFIER	`/conf/svg/content`
REQUIREMENT	Requirement 59: `/req/svg/content`
TEST PURPOSE	Verify that the implementation supports retrieving maps negotiating for SVG content
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving an SVG (`image/svg+xml`) representation of a map resource through HTTP content negotiation

ABSTRACT TEST A.59	
	Then: - assert that every 200-response of the server with the media type `image/svg+xml` is an SVG document representing only a map, - assert that the SVG coordinates inside the map start at 0,0 and end in the width and height of the request.

A.21. Conformance Class “HTML”

CONFORMANCE CLASS A.21	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/html`
REQUIREMENTS CLASS	Requirements class 21: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/html`
TARGET TYPE	Web API
CONFORMANCE TEST	Abstract test A.60: `/conf/html/content`

A.21.1. Abstract Test for Requirement HTML map content

ABSTRACT TEST A.60	
IDENTIFIER	`/conf/html/content`
REQUIREMENT	Requirement 60: `/req/html/content`
TEST PURPOSE	Verify that the implementation supports retrieving maps negotiating for HTML content
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving an (`text/html`) HTML representation of a map resource HTTP content negotiation **Then:** - assert that every 200-response of the server with the media type `text/html` is an HTML document representing the geospatial data as maps.

A.22. Conformance Class “API Operations”

CONFORMANCE CLASS A.22	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/api-operations`
REQUIREMENTS CLASS	Requirements class 22: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/api-operations`
TARGET TYPE	Web API
CONFORMANCE TESTS	Abstract test A.61: `/conf/api-operations/completeness` Abstract test A.62: `/conf/api-operations/operation-id`

A.22.1. Abstract Test for Requirement API Operations completeness

ABSTRACT TEST A.61	
IDENTIFIER	`/conf/api-operations/completeness`
REQUIREMENT	Requirement 61: `/req/api-operations/completeness`
TEST PURPOSE	Verify that the implementation completely and correctly describes the map resources
TEST METHOD	**Given:** an API conforming to *OGC API — Common — Part 1: Core* “Landing Page” conformance class, **When:** retrieving the API description **Then:** - assert that the API definition provides paths for all map, custom projections, tileset, tilesets list and tile resources provided by the API instance, - assert that the resource paths defined in the API definition are consistent with the links to the same resources provided by the landing page, collections, tileset and tilesets list resources, - assert that the resource paths defined in the API definition provide the description of the parameters that the map, tileset and tile resources need to operate that are specified in corresponding conformance classes.

A.22.2. Abstract Test for Requirement API Operation identifiers

ABSTRACT TEST A.62	
IDENTIFIER	`/conf/api-operations/operation-id`
REQUIREMENT	Requirement 62: `/req/api-operations/operation-id`
TEST PURPOSE	Verify that the implementation uses the correct API operation identifier suffixes to identify the resources defined in the Maps API Standard
TEST METHOD	**Given:** an API implementation conforming to *OGC API — Common — Part 1: Core* "Landing Page" conformance class supporting an API definition language with a concept of operation identifiers **When:** retrieving the API description - assert that the paths defined in the API definition have an operation identifier value ending with the relevant dot-separated suffix corresponding to the resource as specified in API operation identifier suffixes.

A.23. Conformance Class "CORS"

CONFORMANCE CLASS A.23	
IDENTIFIER	`https://www.opengis.net/spec/ogcapi-maps-1/1.0/conf/cors`
REQUIREMENTS CLASS	Requirements class 23: `https://www.opengis.net/spec/ogcapi-maps-1/1.0/req/cors`
TARGET TYPE	Web API
CONFORMANCE TEST	Abstract test A.63: `/conf/cors/cors`

A.23.1. Abstract Test for Requirement CORS

ABSTRACT TEST A.63	
IDENTIFIER	`/conf/cors/cors`
REQUIREMENT	Requirement 63: `/req/cors/cors`
TEST PURPOSE	Verify that the implementation completely and correctly implement CORS

ABSTRACT TEST A.63	
TEST METHOD	**Given:** a map resource that conformed successfully to /conf/core **When:** retrieving resources, such as the map resource, defined in the supported requirements classes from a web page at a different origin **Then:** - assert that the implementation supports CORS as defined by W3C (https://www.w3.org/TR/2020/SPSD-cors-20200602/) for these resources.

B

ANNEX B (INFORMATIVE) EXAMPLES

B ANNEX B (INFORMATIVE) EXAMPLES

This annex provides a set of examples illustrating requests and responses for retrieving maps using capabilities defined by this OGC API — Maps Standard.

B.1. Default map

The first example shows the response of a map endpoint with a request with no parameters (https://maps.gnosis.earth/ogcapi/collections/OpenMapLocal/map).

The server is free to respond with any `bbox` and any `width` and `height`. In this case the server default behavior is to render the whole of Great Britain in a 631×1024 pixels canvas.



Figure B.1 — Great Britain data served as a map without specifying any parameter. From: Ordnance Survey OS OpenMap - Local product. Contains OS data © Crown copyright and database right 2023.

The headers of the response provide additional information on the bounding box (`Content-Bbox`) and the CRS of the image (`Content-Crs` is omitted here, indicating the default CRS84).

```
HTTP/1.1 200 OK
Expires: Tue, 19 Nov 2024 09:58:40 GMT
Access-Control-Allow-Origin: *
Vary: Accept, Accept-Encoding, Prefer
Content-Type: image/png
Content-Bbox: -8.756498,49.814737,1.848147,60.948016
Content-Length: 538037
```

B.2. Collection description

This map resource is attached to an *OGC API — Common — Part 2* collection origin, which is available from the parent resource path:

https://maps.gnosis.earth/ogcapi/collections/OpenMapLocal

An example JSON collection description response for the OS OpenMap Local collection is below:


```
{
   "id" : "OpenMapLocal",
   "title" : "OS OpenMap - Local",
   "description" : "OS OpenMap - Local, by the OrdnanceSurvey",
   "attribution" : "<a href='https://www.ordnancesurvey.co.uk/products/os-open-
map-local'>OS OpenMap - Local</a>",
   "extent" : {
      "spatial" : {
         "bbox" : [ [ -8.7564981947375, 49.8147371614082,
                      1.8481470870055, 60.9480162305518 ] ]
   } },
   "minScaleDenominator" : 4265.4591676995678,
   "minCellSize" : 0.0000107288361,
   "storageCrs" : "https://www.opengis.net/def/crs/OGC/1.3/CRS84",
   "crs" : [
      "https://www.opengis.net/def/crs/OGC/1.3/CRS84",
      "https://www.opengis.net/def/crs/EPSG/0/4326",
      "https://www.opengis.net/def/crs/EPSG/0/3857",
      "https://www.opengis.net/def/crs/EPSG/0/3395"
   ],
   "links" : [
      { "rel" : "self", "title" : "Information about the OpenMapLocal data",
        "href" : "/ogcapi/collections/OpenMapLocal"
      },
      { "rel" : "[ogc-rel:map]", "title" : "Default map",
        "href" : "/ogcapi/collections/OpenMapLocal/map"
      },
      { "rel" : "[ogc-rel:tilesets-map]",
        "title" : "Map tilesets available for this collection",
        "href" : "/ogcapi/collections/OpenMapLocal/map/tiles"
      },
      { "rel" : "[ogc-rel:tilesets-vector]",
        "title" : "Multi-layer vector tilesets available for this collection",
        "href" : "/ogcapi/collections/OpenMapLocal/tiles"
      },
      { "rel" : "[ogc-rel:styles]", "title" : "Styles for OpenMapLocal",
        "href" : "/ogcapi/collections/OpenMapLocal/styles"
      }
   ]
}
```

B.3. Scaling and subsetting the map

The following examples are going to be centered on The O2 (formerly known as the Millenium Dome) shown here in a night aerial picture. Note that the roundness of the feature facilitates validating its aspect ratio in physical units.

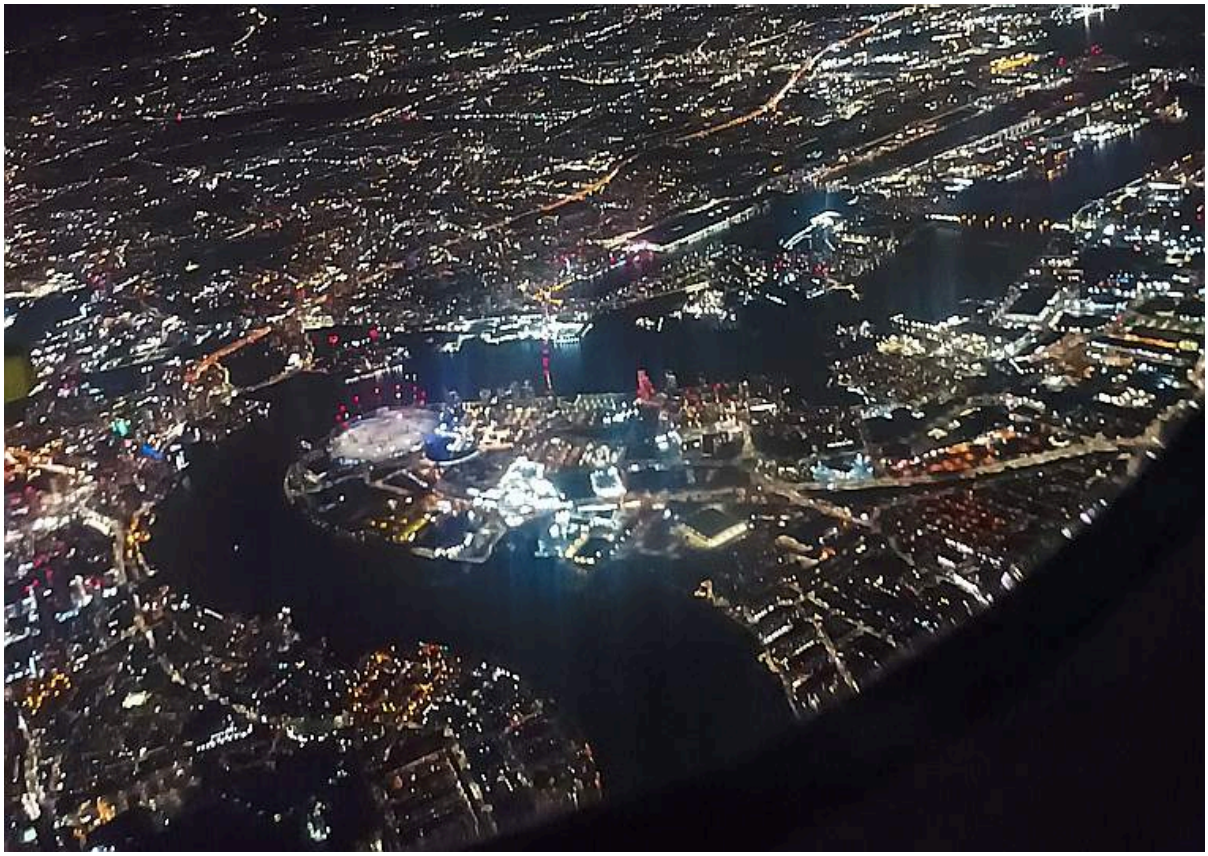


Figure B.2 — The O2 dome peninsula (image captured from a plane by an editor of this standard while working on this annex)

The following map request only specifies a `center` point parameter next to the O2 Dome:

https://maps.gnosis.earth/ogcapi/collections/OpenMapLocal/map?center=0,51.5

The server interprets the coordinates as CRS84 and decides to respond with a low scale denominator (high level of detail) suitable for the dataset and with reasonable default `width` and `height` (1024×1024 pixels). The response is shown in the following image.



Figure B.3 — Map of OS OpenMap - Local close to The O2 dome, specifying `center` at 51.5°N, 0°E. Contains OS data © Crown copyright and database right 2023.

The headers of the response provide additional information on the bounding box (`Content-Bbox`). Since the `Content-Crs` is not specified in this case, the client can assume CRS84.

```
HTTP/1.1 200 OK
Expires: Tue, 19 Nov 2024 09:57:36 GMT
Access-Control-Allow-Origin: *
Vary: Accept, Accept-Encoding, Prefer
```

```
Content-Type: image/png
Content-Bbox: -0.008805,51.494504,0.008805,51.505496
Content-Length: 188490
```

The following request is equivalent, using the value of that `Content-Bbox` as the value for the `bbox` parameter instead of using `center`, explicitly specifying the same `width` and `height` dimensions as those default values chosen by the server for the above request:

https://maps.gnosis.earth/ogcapi/collections/OpenMapLocal/map?bbox=-0.008805,51.494504,0.008805,51.505496&width=1024&height=1024

There is also an equivalent notation for the previous request that uses `subset` instead of `bbox`:

https://maps.gnosis.earth/ogcapi/collections/OpenMapLocal/map?subset=Lat(51.494504:51.505496),Lon(-0.008805:0.008805)&width=1024&height=1024

As explained in the scale and aspect ratio considerations section, clients wishing to retrieve identical responses from different implementations should specify either of these `bbox` or `subset` parameters, together with a `width` and `height`. A smaller image can be requested by specifying `height` of the image.

https://maps.gnosis.earth/ogcapi/collections/OpenMapLocal/map?center=0,51.5&height=512

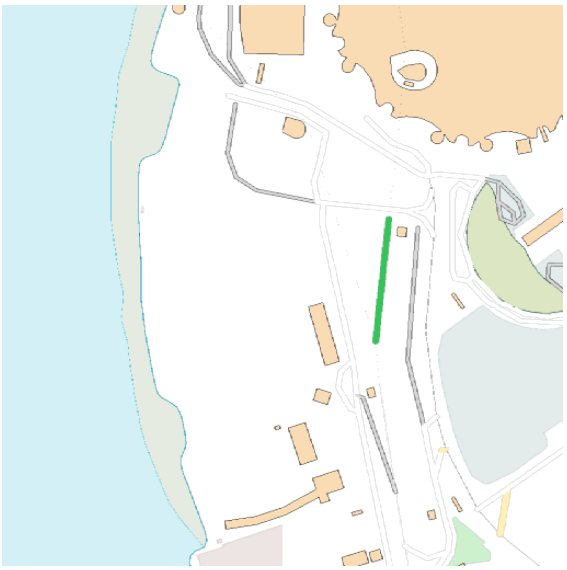


Figure B.4 — Map of OS OpenMap - Local centered on The O2 dome using `height=512`

The server would be free to act otherwise, but it automatically adjusted the width to also be 512. Notice that to preserve the same default scale with a smaller image, the spatial region (bounding box) was reduced accordingly. This behavior is particularly important when the client requests a specific scale, as in the following request which specifies the same `center` point parameter as before, but requests a map for a 1:8000 scale:

https://maps.gnosis.earth/ogcapi/collections/OpenMapLocal/map?center=0,51.5&height=512&scale-denominator=8000



Figure B.5 — Map of OS OpenMap - Local centered on The O2 dome at 1:8000 scale using `scale-denominator`. Contains OS data © Crown copyright and database right 2023.

The server responded with the same `width` and `height` (512×512 pixels). The headers of the response provide additional information on the bounding box of the image.

Clients can easily zoom in and out by simply changing the `scale-denominator` parameter as in the following images at different scales:



Figure B.6 — Map of OS OpenMap - Local centered on The O2 dome at 1:12,000 scale using `scale-denominator`. Contains OS data © Crown copyright and database right 2023.



Figure B.7 — Map of OS OpenMap - Local centered on The O2 dome at 1:20,000 scale using `scale-denominator`. Contains OS data © Crown copyright and database right 2023.



Figure B.8 — Map of OS OpenMap - Local centered on The O2 dome at 1:30,000 scale using `scale-denominator`. Contains OS data © Crown copyright and database right 2023.



Figure B.9 — Map of OS OpenMap - Local centered on The O2 dome at 1:50,000 scale using `scale-denominator`. Contains OS data © Crown copyright and database right 2023.

A rectangular image could be “forced” by also specifying the width of the image to be 1024, while keeping the rest of the parameters:

https://maps.gnosis.earth/ogcapi/collections/OpenMapLocal/map?center=0,51.5&scale-denominator=50000&width=1024&height=512



Figure B.10 — Wider 1024x512 map of OS OpenMap - Local centered on the O2 dome at 1:50,000 scale. Contains OS data © Crown copyright and database right 2023.

For this last request, specifying both the `width` and `height`, the `center`, the `scale-denominator`, combined with the fact that the default value of `mm-per-pixel` is defined as 0.28 mm/pixel, defines all the parameters necessary to make the subsetting and scaling *mostly* predictable by the client. As explained in the scale and aspect ratio considerations section, implementations may compute the dimensions and bounding boxes not explicitly specified slightly differently. Because of these potential differences, clients should always consider the bounding box information in the response headers for georeferencing purposes, as well as the actual dimensions of the image returned. This will also avoid problems in cases where the server may decide to correct the center or bounding box due to the values being out of range. The `center` and `scale-denominator` parameters are primarily intended as convenience parameters to let the server automatically compute ideal bounding boxes and dimensions, while specifying a spatial region using the `bbox` or `subset` parameter as well as `width` and `height` should result in more deterministic responses.

B.4. Temporal subsetting

Spatial datasets are often also organized with a temporal dimension in addition to two or three spatial dimensions (some of these datasets are sometimes called time series or datacubes).

The following example reuses the same subsetting and scaling from the earlier rectangular 1:50,000 map of London, and applies it to a Sentinel-2 collection of images. The `datetime` parameter selects a particular day of the time series (April 1st, 2022).

https://maps.gnosis.earth/ogcapi/collections/sentinel2-l2a/map?center=0,51.5&scale-denominator=50000&datetime=2022-04-01&width=1024&height=512



Figure B.11 — A map of Sentinel-2 data from April 1st, 2022 of the same area. From: Copernicus SENTINEL-2 operated by ESA.

There is an equivalent notation for the previous request that uses `subset` for the *time* axis instead of the `datetime` parameter. This subsetting axis can also be combined within a single `subset` parameter value together with subsetting for the `Lat` and `Lon` axes, instead of using `center` and `scale-denominator`, or `bbox`. Note that in this case, the time string should be enclosed in double quotes.

https://maps.gnosis.earth/ogcapi/collections/sentinel2-l2a/map?width=1024&height=512&subset=time("2022-04-01"),Lat(51.467787:51.532213),Lon(-0.103152:0.103152)

B.5. Styled maps

The following two example requests, for the same region and time of interest, illustrate two additional styles available from the same sentinel-2 datacube, in addition to its default Red, Green, Blue natural color style. The first style, symbolizes the Scene Classification Layer categories:

https://maps.gnosis.earth/ogcapi/collections/sentinel2-l2a/styles/scl/map?center=0,51.5&scale-denominator=50000&datetime=2022-04-01&width=1024&height=512

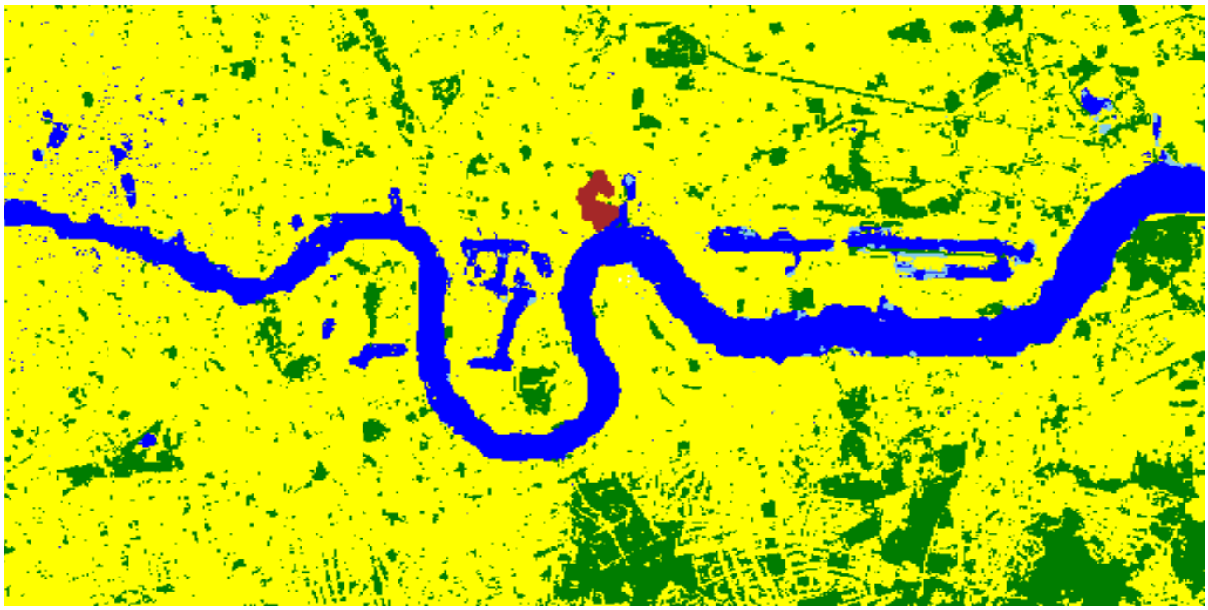


Figure B.12 — A map of a scene classification layer style for Sentinel-2 data from April 1st, 2022 of London. From: Copernicus SENTINEL-2 operated by ESA.

The next style, using style identifier *evi2*, represents an Enhanced Vegetation Index (EVI) calculated from bands B02 (blue), B04 (red) and B08 (near infrared):

https://maps.gnosis.earth/ogcapi/collections/sentinel2-l2a/styles/evi2/map?center=0,51.5&scale-denominator=50000&datetime=2022-04-01&width=1024&height=512



Figure B.13 — A map of an Enhanced Vegetation Index (EVI) style for Sentinel-2 data from April 1st, 2022 of London. From: Copernicus SENTINEL-2 operated by ESA.

The following example requests illustrate how to retrieve two different styles for a High Resolution (1 m) Digital Terrain Model (DTM) of the Red River in Manitoba, from Natural Resources Canada. Styles with identifiers *style1* and *style2* are available at `.../styles/{styleId}`, through *OGC API — Styles*, and provide links to map resources.

https://maps.gnosis.earth/ogcapi/collections/HRDEM-RedRiver:DTM:1m/styles/style1/map?center=-97.06,49.937&scale-denominator=28000&height=600&width=1000



Figure B.14 — Styled map of High Resolution DTM from Natural Resources Canada (style1)

https://maps.gnosis.earth/ogcapi/collections/HRDEM-RedRiver:DTM:1m/styles/style2/map?center=-97.06,49.937&scale-denominator=28000&height=600&width=1000



Figure B.15 — Styled map of High Resolution DTM from Natural Resources Canada, showing alternative *style2*

B.6. Additional dimensions

It is also common for spatial datasets, especially for weather and climate data, to feature additional dimensions beyond space and time, such as pressure levels, or additional time dimensions relating to forecasting. These can all be handled in a generic manner also using the `subset` parameter. The following two examples illustrate how to retrieve a map of the temperature for the whole world at *pressure* (an extra dimension) levels of 500 and 850 hectopascals:

https://maps.gnosis.earth/ogcapi/collections/climate:cmip5:byPressureLevel:temperature/map?subset=pressure(500)&datetime=2023-07-03&bgcolor=gray



Figure B.16 — A map of CMIP5 temperature data of the world at 500 hPa on July 3rd, 2023 (from Copernicus climate data store)

https://maps.gnosis.earth/ogcapi/collections/climate:cmip5:byPressureLevel:temperature/map?subset=pressure(850)&datetime=2023-07-03&bgcolor=gray

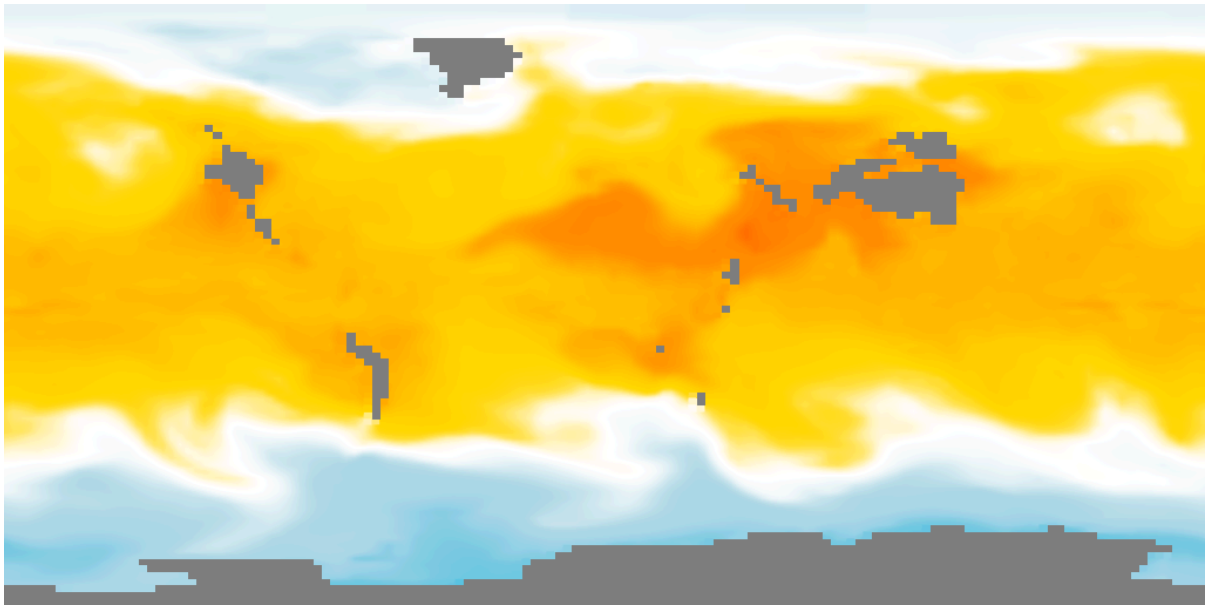


Figure B.17 — A map of CMIP5 temperature data of the world at 850 hPa on July 3rd, 2023 (from Copernicus climate data store)

The following examples illustrate how to retrieve a map of the relative humidity for the whole world at *pressure* (an extra dimension) levels of 500 and 975 hectopascals:

https://maps.gnosis.earth/ogcapi/collections/climate:era5:relativeHumidity/map?subset=pressure(500)&datetime=2023-04-06T23:00:00Z&bgcolor=skyBlue

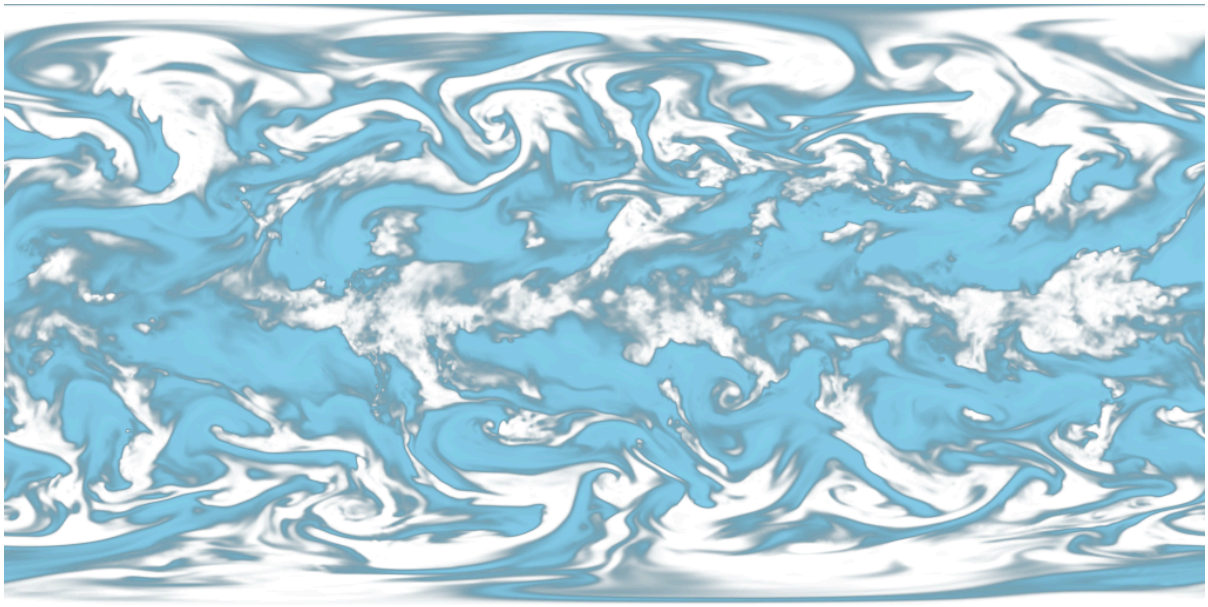


Figure B.18 — A map of ERA5 reanalysis data showing Relative Humidity of the whole world at 500 hPa on April 6th, 2023 at 23:00:00 UTC (from Copernicus climate data store)

https://maps.gnosis.earth/ogcapi/collections/climate:era5:relativeHumidity/map?subset=pressure(975)&datetime=2023-04-06T23:00:00Z&bgcolor=skyBlue

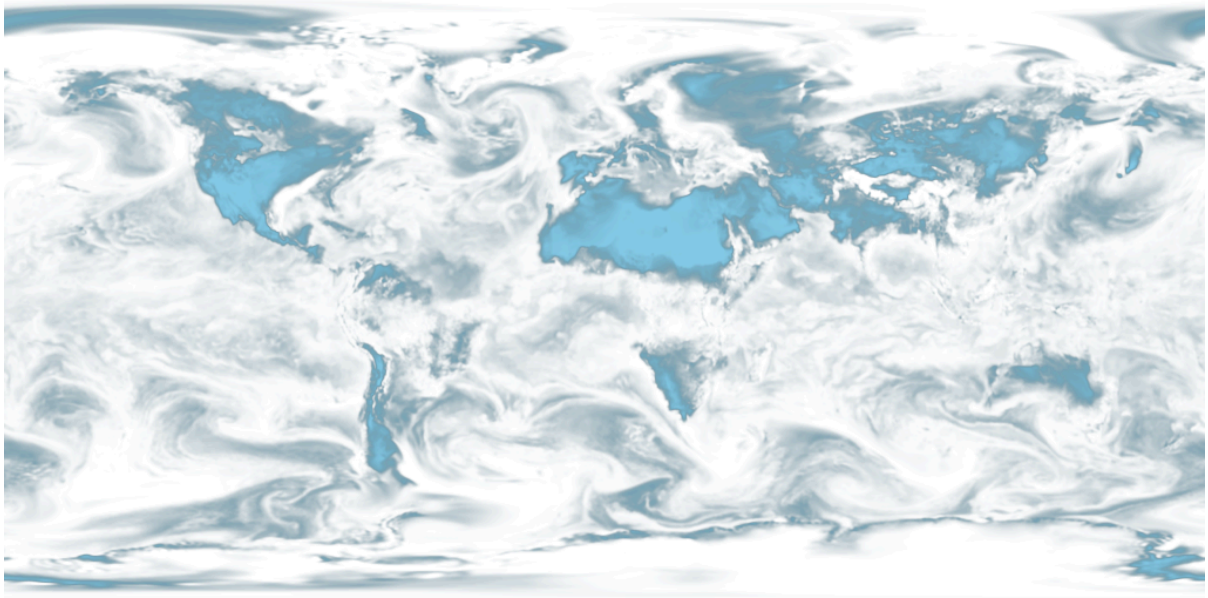


Figure B.19 — A map of ERA5 reanalysis data showing Relative Humidity of the whole world at 975 hPa on April 6th, 2023 at 23:00:00 UTC (from Copernicus climate data store)

The following JSON collection description for these relative humidity examples illustrates how to describe the extent of multidimensional datasets, including the details of both regular and irregular grids.

An example JSON collection description response for ERA5 relative humidity is below:


```
{
   "id" : "climate:era5:relativeHumidity", "title" : "ERA5 Relative Humidity",
   "attribution" : "<a href='https://cds.climate.copernicus.eu/cdsapp#!/dataset/
reanalysis-era5-pressure-levels'>Copernicus Climate Data Store</a>",
   "extent" : {
      "spatial" : {
         "bbox" : [ [ -180, -90, 180, 90 ] ],
         "grid" : [
            { "cellsCount" : 2049, "resolution" : 0.17578125 },
            { "cellsCount" : 1025, "resolution" : 0.17578125 }
         ]
      },
      "temporal" : {
         "interval" : [ [ "2023-04-01T00:00:00Z", "2023-04-06T23:00:00Z" ] ],
         "grid" : { "cellsCount" : 144, "resolution" : "PT1H" }
      },
      "pressure" : {
         "definition": "https://qudt.org/vocab/quantitykind/AtmosphericPressure",
         "unit" : "hPa",
         "interval" : [ [ 1.0, 1000.0 ] ],
         "grid" : {
            "cellsCount" : 37,
            "coordinates" : [ 1.0, 2.0, 3.0, 5.0, 7.0, 10.0, 20.0,
               30.0, 50.0, 70.0, 100.0, 125.0, 150.0, 175.0, 200.0,
               225.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0, 550.0,
               600.0, 650.0, 700.0, 750.0, 775.0, 800.0, 825.0, 850.0,
               875.0, 900.0, 925.0, 950.0, 975.0, 1000.0 ]
         }
      }
   },
   "minCellSize" : 0.17578125,
   "minScaleDenominator" : 69885283.0035897195339,
   "crs" : [
      "https://www.opengis.net/def/crs/OGC/1.3/CRS84",
      "https://www.opengis.net/def/crs/EPSG/0/4326",
      "https://www.opengis.net/def/crs/EPSG/0/3857",
      "https://www.opengis.net/def/crs/EPSG/0/3395"
   ],
   "storageCrs" : "https://www.opengis.net/def/crs/OGC/1.3/CRS84",
   "links" : [
      { "rel" : "self",
        "title" : "Information about the ERA5 Relative Humidity",
        "href" : "/ogcapi/collections/climate:era5:relativeHumidity"
      },
      { "rel" : "[ogc-rel:map]", "title" : "Default map",
        "href" : "/ogcapi/collections/climate:era5:relativeHumidity/map"
      },
      { "rel" : "[ogc-rel:tilesets-map]",
        "title" : "Map tilesets available for this collection",
        "href" : "/ogcapi/collections/climate:era5:relativeHumidity/map/tiles"
      },
      { "rel" : "[ogc-rel:styles]", "title" : "Styles for Relative Humidity",
        "href" : "/ogcapi/collections/climate:era5:relativeHumidity/styles"
      },
      { "rel" : "[ogc-rel:schema]", "title" : "Schema",
        "href" : "/ogcapi/collections/climate:era5:relativeHumidity/schema"
```

```
        },
        { "rel" : "[ogc-rel:coverage]", "title" : "Coverage for Relative Humidity",
          "href" : "/ogcapi/collections/climate:era5:relativeHumidity/coverage"
        },
        { "rel" : "[ogc-rel:tilesets-coverage]",
          "title" : "Coverage tilesets available for this collection",
          "href" : "/ogcapi/collections/climate:era5:relativeHumidity/tiles"
        }
    ]
}
```

B.7. Coordinate Reference Systems

While introducing the selection of an alternative output Coordinate Reference System (World Mercator, EPSG:3395) to the default native CRS (`storageCRS`) returned so far (CRS84), the following examples will zoom out significantly to a 1:20,000,000 scale. At the scales used in previous examples, the difference between those two CRSs would not be distinguishable, since the server automatically preserve scales in both dimensions, which makes the responses for those two CRSs almost visually equivalent on a local scale. These examples will illustrate the two CRS by requesting a map for the Blue Marble Next Generation (2004) from NASA Earth Observatory's Visible Earth, first explicitly requesting EPSG:4326 (whose main difference from CRS84 is that axis order is Latitude, Longitude).

https://maps.gnosis.earth/ogcapi/collections/blueMarble/map?center=0,51.5&scale-denominator=20000000&crs=[EPSG:4326]

Notice that the response header now includes the `Content-Crs:` header, and that the `Content-Bbox:` axis order now follows the latitude, longitude order:

```
Content-Crs: <https://www.opengis.net/def/crs/EPSG/0/4326>
Content-Bbox: 25.729221,-28.573112,77.270779,28.573112
```

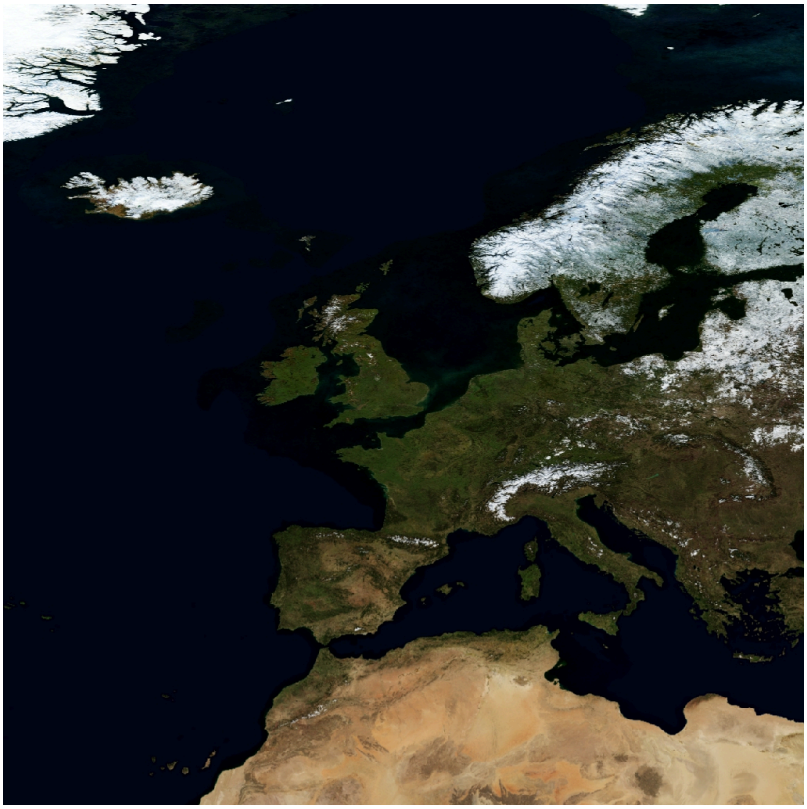


Figure B.20 — Map of NASA Earth Observatory's Blue Marble Next Generation (2004), in Plate Carrée (EPSG:4326) output `crs`

Now EPSG:3395 can be requested instead using:

https://maps.gnosis.earth/ogcapi/collections/blueMarble/map?center=0,51.5&scale-denominator=20000000&crs=[EPSG:3395]



Figure B.21 — Map of NASA Earth Observatory's Blue Marble Next Generation (2004), using World Mercator (EPSG:3395) output `crs`

The `Content-Crs:` contains the coordinates of the bounding box selected from the requested scale and default dimensions, which can be used to make a request that will generate an equivalent response.

```
Content-Crs: <https://www.opengis.net/def/crs/EPSG/0/3395>
Content-Bbox: -4596385.263861,2080129.089271,4596385.263861,11273386.415933
```

In order to also specify the bounding box in that EPSG:3395 CRS, the following request also makes use of the `bbox-crs` parameter, which otherwise always defaults to CRS84 (regardless of the native CRS or selected output CRS).

https://maps.gnosis.earth/ogcapi/collections/blueMarble/map?bbox-crs=[EPSG:3395]&bbox=-4596385.263861,2080129.089271,4596385.263861,11273386.415933&crs=[EPSG:3395]

B.8. Calculations to infer appropriate dimensions

In these examples, the client specifies a bounding box (using `bbox` or `subset`) from 30°N to 50°N and 0°E to 30°E and a `scale-denominator` of 1:10,000,000. The server needs to compute appropriate map image dimensions from these parameters that are provided (no default dimensions or center are used). The default 0.28 mm/pixel display resolution is used since `mm-per-pixel` is not specified. Two examples are given, one in a geographic Plate Carrée CRS (EPSG:4326) and one in a projected World Mercator CRS (EPSG:3395) (which uses a bounding box of the same geographic area transformed into coordinates in that CRS).

Regardless of the CRS, the number of physical meters that each pixel should represent can be computed with:

```
physicalMetersPerPixel = (mm-per-pixel / 1000 mm/m) * scale-denominator
physicalMetersPerPixel = (0.28 mm/pix / 1000 mm/m) * 10,000,000 = 2800 m/pix
```

B.8.1. Plate Carrée (EPSG:4326) Example

The first example is for an EPSG:4326 Plate Carrée CRS.

```
GET /collections/blueMarble/map?
   subset=Lat(30:50),Lon(0:30)&
   scale-denominator=10000000&
   crs=[EPSG:4326]

GET /collections/blueMarble/map?
   bbox=0,30,30,50&
   scale-denominator=10000000&
   crs=[EPSG:4326]
```

The latitude delta is 20°, whereas the longitude delta is 30°.

The implementation could assume the WGS84 111,319.49 meters / degree of latitude (`metersPerDegLat` below), and use the most equatorial latitude of the subset (30°N) to compute the numbers of meters / degree of longitude with:

```
metersPerDegLon = metersPerDegLat * cos(mostEquatorialLat)
metersPerDegLon = 111,319.49 m/deg * cos(30°) = 96,405.51 m/deg
```

The dimensions can then simply be computed (rounding to the nearest integer) with:

```
width  = deltaLon * metersPerDegLon / physicalMetersPerPixel
height = deltaLat * metersPerDegLat / physicalMetersPerPixel

width  = 30 deg *  96,405.51 m/deg / 2800 m/pix = 1033 pixels
height = 20 deg * 111,319.49 m/deg / 2800 m/pix =  795 pixels
```

B.8.2. World Mercator (EPSG:3395) Example

This second example is for the same geographical area, but for an EPSG:3395 World Mercator CRS instead.

```
GET /collections/blueMarble/map?
   subset=E(0:3339584.72),N(3482189.09:6413524.59)&
   scale-denominator=10000000&
   subset-crs=[EPSG:3395]&
   crs=[EPSG:3395]

GET /collections/blueMarble/map?
   bbox=0,3482189.09,3339584.72,6413524.59&
   scale-denominator=10000000&
   crs=[EPSG:3395]&
   bbox-crs=[EPSG:3395]
```

The easting delta is 3,339,584.72, whereas the northing delta is 2,931,335.50.

To correctly apply the scale, the ratio between CRS units and physical meters must be considered. This could be calculated for the center point: (E: 1,669,792.36, N: 4,947,856.84) which corresponds to (40.7514917°N, 15°E). One approach could be to project two points 1 degree of longitude apart around the center point, using the implementation's projection library, to obtain the number of CRS units per degree of longitude. Transforming (40.7514917°N, 14.5°E) to EPSG:3395 yields (E: 1,614,132.62, N: 4,947,856.85) and transforming (40.7514917°N, 15.5°E) yields (E: 1,725,452.11, N: 4,947,856.85). The resulting easting delta (`oneDegEastingDelta` below) is 111,319.49, which in this case can be recognized as the actual physical meters per degree at the equator, rather than at the center latitude used. Therefore, in this case implementations effectively need to apply the reverse correction that had to be applied for Plate Carrée when considering the numbers of true meters per Easting unit. First, `metersPerDegLon` can be computed as in the previous example:

```
metersPerDegLon = metersPerDegLat * cos(centerLat)
metersPerDegLon = 111,319.49 m/deg * cos(40.7514917°) = 84,329.856 m/deg
```

Then the meters per easting unit can be computed:

```
metersPerEastingUnit = metersPerDegLon / oneDegEastingDelta
metersPerEastingUnit = 84,329.856 m/deg / 111,319.49 m/deg = 0.75754799
```

Notice that in this particular case of World Mercator, this is simply:

```
metersPerEastingUnit = cos(centerLat)
metersPerEastingUnit = cos(40.7514917°) = 0.75754799
```

The implementation could also apply similar logic to compute the meters per northing unit:

```
metersPerNorthingUnit = metersPerDegLat / oneDegNorthingDelta
```

However, in the case of World Mercator, it could simply assume that `metersPerNorthingUnit` is equal to `metersPerEastingUnit`.

Finally, the map image dimensions can be computed with:

```
width  = deltaEasting  * metersPerEastingUnit  / physicalMetersPerPixel
height = deltaNorthing * metersPerNorthingUnit / physicalMetersPerPixel
```

```
width  = 3,339,584.72 m * 0.75754799 / 2800 m/pix = 904 pixels
height = 2,931,335.50 m * 0.75754799 / 2800 m/pix = 793 pixels
```

B.9. Calculations to infer appropriate bounding boxes

In these examples, the client specifies a `center`, a `scale-denominator`, and optionally `width` and / or `height` dimensions. If either the `width` or `height` is not specified, the server could pick default values, such as making the missing dimension equal to the one provided or selecting default values. The examples will assume that the client explicitly requested a 1024 x 768 map at a 1:10,000,000 scale for a location centered on (41.8902°N, 12.4922°E).

To compute the extent in CRS units, first the physical meters per pixel can be computed using the same formula as earlier (and same result in this case):

```
physicalMetersPerPixel = (mm-per-pixel / 1000 mm/m) * scale-denominator
physicalMetersPerPixel = (0.28 mm/pix / 1000 mm/m) * 10,000,000 = 2800 m/pix
```

B.9.1. Plate Carrée (EPSG:4326) Example

This first example requests a map in an EPSG:4326 output CRS, using that same CRS for specifying the center as well:

```
GET /collections/blueMarble/map?
   center=41.8902,12.4922&
   center-crs=[EPSG:4326]&
   scale-denominator=10000000&
   crs=[EPSG:4326]&
   width=1024&
   height=768
```

A simple approach to computing the bounding box is to extend away from the center in both directions by the distance in CRS units corresponding to half the respective pixel dimension.

To consider the latitude of the subsets for computing the longitude extent, the latitude extent will be computed first, using the inverse of the earlier height computation:

```
deltaLat = height * physicalMetersPerPixel / metersPerDegLat
deltaLat = 768 pix * 2800 m/pix / 111,319.49 m/deg = 19.317372 degrees
```

A constant 111,319.49 meters / degree of latitude is assumed here again, rather than the more accurate polynomial equation taking into consideration the ellipsoid eccentricity mentioned earlier, which would yield slightly different results.

The lower and upper latitudes of the bounding can then be easily computed by adding and subtracting half this delta to the requested center latitude:

```
lowerLat = 41.8902°N - 19.317372 deg / 2 = 32.231514°N
upperLat = 41.8902°N + 19.317372 deg / 2 = 51.548886°N
```

From this, the most equatorial latitude can be established to be 32.231514°N, which can then be used to compute the meters per degrees of longitude, exactly like for the dimension computation examples:

```
metersPerDegLon = metersPerDegLat * cos(mostEquatorialLat)
metersPerDegLon = 111,319.49 m/deg * cos(32.231514°) = 94,165.15 m/deg
```

and the longitude delta can then be computed similarly:

```
deltaLon = width * physicalMetersPerPixel / metersPerDegLon
deltaLon = 1024 pix * 2800 m/pix / 94,165.15 m/deg = 30.448632 degrees
```

and from this the left (West) and right (East) longitude bounds:

```
leftLon  = 12.4922°E - 30.448632 deg / 2 =  2.732116°W
rightLon = 12.4922°E + 30.448632 deg / 2 = 27.716516°E
```

which completes the bounding box calculation. Expressed in the default CRS84 (longitude, latitude) order, the `bbox` parameter would be:

```
bbox=-2.732116,32.231514,27.716516,51.548886.
```

B.9.2. World Mercator (EPSG:3395) Example

This second example requests a map in an EPSG:3395 output CRS, using that same CRS for specifying the center as well:

```
GET /collections/blueMarble/map?
   center=1390625.34,5116008.23&
   center-crs=[EPSG:3395]&
   scale-denominator=10000000&
   crs=[EPSG:3395]&
   width=1024&
   height=768
```

The center corresponds to the same point as the previous example (41.8902°N, 12.4922°E).

Using the same approach as for the earlier dimension computation examples, transforming test points one degree apart, the number of physical meters per easting and northing units can be computed specifically for the requested center point (E: 1390625.34, N: 5116008.23). Like earlier, in the specific case of World Mercator the `oneDegEastingDelta` and `oneDegNorthingDelta` is constant at 111,319.49 m/deg, which corresponds to the number of meters per degree at the equator. The number of physical meters per degree of longitude at the center latitude (41.8902°N) can be computed in the same way as previous examples:

```
metersPerDegLon = metersPerDegLat * cos(centerLat)
metersPerDegLon = 111,319.49 m/deg * cos(41.8902°) = 82,869.096 m/deg
```

Then the meters per easting unit can be computed:

```
metersPerEastingUnit = metersPerDegLon / oneDegEastingDelta
metersPerEastingUnit = 82,869.096 m/deg / 111,319.49 m/deg = 0.74442576
```

Again, in the case of World Mercator, this is simply:

```
metersPerEastingUnit = cos(centerLat)
metersPerEastingUnit = cos(41.8902°) = 0.74442576
```

The implementation could also apply similar logic to compute the meters per northing unit:

```
metersPerNorthingUnit = metersPerDegLat / oneDegNorthingDelta
```

And again, in this case it could simply assume that `metersPerNorthingUnit` is equal to `metersPerEastingUnit`.

Then the delta easting and northing can be computed using the inverse of the equations used for computing dimensions:

```
deltaEasting  = width  * physicalMetersPerPixel / metersPerEastingUnit
deltaNorthing = height * physicalMetersPerPixel / metersPerNorthingUnit
deltaEasting  = 1024 pix * 2800 m/pix / 0.74442576 = 3,851,559.355 m
deltaNorthing =  768 pix * 2800 m/pix / 0.74442576 = 2,888,669.516 m
```

and finally this bounding box can be extended away from the requested center point (E: 1,390,625.34, N: 5,116,008.23):

```
leftEasting   = 1,390,625.34 - 3,851,559.355 / 2 =  -535,154.34
lowNorthing   = 5,116,008.23 - 2,888,669.516 / 2 = 3,671,673.47
rightEasting  = 1,390,625.34 + 3,851,559.355 / 2 = 3,316,405.02
upperNorthing = 5,116,008.23 + 2,888,669.516 / 2 = 6,560,342.99
```

which completes the bounding box calculation. Expressed in EPSG:3395 together with the required `bbox-crs`, the parameters would be:

```
bbox=-535154.34,3671673.47,3316405.02,6560342.99&bbox-crs=[EPSG:3395].
```

C

ANNEX C (INFORMATIVE) EVOLUTION FROM OGC WEB MAP SERVICE

ANNEX C (INFORMATIVE) EVOLUTION FROM OGC WEB MAP SERVICE

C.1. From OGC Web Services to OGC Web APIs

OGC Web Service (OWS) Standards have historically implemented a Remote-Procedure-Call-over-HTTP architectural style using Extensible Markup Language (XML) for the metadata and capability payloads. This was state-of-the-art when initial versions of OGC Web Services were originally designed in the late 1990s and early 2000s. The resource-oriented RESTful Web API style is proposed as an alternative architectural style to the RPC pattern which was service-oriented. The Web Map Service 1.3 Standard did not describe a resource oriented architectural style or a Web API definition. This *OGC API — Maps* Standard specifies requirements for a Web API that follows the Web architecture and the W3C/OGC best practices for sharing Spatial Data on the Web as well as the W3C best practices for sharing Data on the Web.

The *OGC API — Maps — Part 1: Core* Standard provides the necessary elements to incorporate ***maps*** support into a broader Web API implementation. These elements can be incorporated in an API based on *OGC API — Features — Part 1: Core* or can be incorporated in a Web API implementation based on *OGC API — Common — Part 1: Core*. Both specify a kernel of a Web API approach to services that follows current resource-oriented architecture practices in the OGC. The *OGC API — Common* Standard provides the foundation upon which implementations of the OGC API Standards can be built. *OGC API — Common* can be combined with the *Maps API* Standard and other resource-specific OGC API Standards to build an OGC standards-based API implementation. However, the *Maps API* Standard is specified in a way that can extend *OGC API — Common* but does not make *OGC API — Common* mandatory. In this way, the *Maps API* Standard can be reused as a building block in other APIs that do not follow the OGC API pattern.

Beside the general alignment with the architecture of the Web (e.g., consistency with HTTP/HTTPS, hypermedia controls), another goal for OGC API Standards is modularization. This goal has several facets:

- Clear separation between Core requirements and more advanced capabilities. The *OGC API — Maps — Part 1: Core* Standard presents the requirements that are relevant for almost any organization wanting to share or use maps at a fine-grained level. Additional capabilities deemed useful for several communities will be specified as extensions to the Core API.
- Technologies that change more frequently are decoupled and specified in separate modules ("requirements classes" in OGC terminology). This enables, for example, the use/

re-use of new encodings for spatial data or Web API definition (such as new version of the OpenAPI description document).

- Modularization is not just about a single "service". OGC API Standards define building blocks that can be reused in API implementations in general. In other words, a server supporting *OGC API — Maps* should not be seen as a standalone service. Rather it should be viewed as a collection of Web API building blocks that together implement *OGC API — Maps* capabilities. A corollary for this is that it should be possible to implement an API that simultaneously conforms to conformance classes from the OGC API — Features, OGC API — Coverages, OGC API — Maps, OGC API — Tiles, and other future OGC API Standards.

This approach intends to support two types of client developers:

- Those that have never heard about the OGC. Developers should be able to create a client using the Web API definition without the need to adopt a specific OGC approach. For example, developers no longer need to read how to implement a *GetCapabilities* document, allowing them to focus on the geospatial aspects.
- Those that want to write a "generic" client that can access implementations of OGC API Standards. In other words, common capability and metadata resources are not specific to a particular API.

As a result of following a RESTful approach, OGC API implementations are not backwards compatible with OWS implementations per se. However, a design goal is to define OGC API Standards in a way that an OGC API interface can be mapped to an OWS implementation (where appropriate). OGC API Standards are intended to be simpler and more modern, but still an evolution from the previous versions and their implementations making the transition easy (e.g., by initially implementing facades in front of the current OWS services).

C.2. Comparison between *WMS* and *OGC API — Maps*

The OGC WMS and WMTS Standards share the concept of a map and the capability to create and distribute maps at a limited resolution and size. In WMS, the number of rows and columns can be selected by the user within limits and in WMTS the number of rows and columns of the response is predefined in the tile matrix set.

The concept of a map used in OGC API Standards is more abstract than the one used in WMS. Read more about the map concept in the Overview.

C.2.1. Functionality changes from WMS

One important enhancement in the *OGC API — Maps* Standard when compared to WMS is that requests are built out of modular building blocks, allowing different levels of complexity: From

a simple map request with no parameters to a precise selection of collections, spatiotemporal subsets, output dimensions, scale, background settings and so on.

Additionally, to accommodate different client preferences and use cases as well as maintain consistency with other OGC API data access mechanisms such as the *OGC API — Coverages* and *OGC API — Features* Standards, multiple equivalent subsetting mechanisms are supported such as subsetting by bounding box, date and time, generic dimension subset or center point.

Similarly, the ability to explicitly specify a scale denominator as a way to control down-sampling is also defined.

The concept of a display resolution (millimeters per pixel) is also introduced as an alternative to the default 0.28 mm/pixel specified in WMS, enabling a better control on how a server renders symbology suitable for devices and printed maps of different resolutions.

A flexible mechanism to define custom CRS and re-orient the map is also introduced, offering a more comprehensive approach based on coordinate operation (see https://docs.ogc.org/as/18-005r4/18-005r4.html, definition 3.1.8) methods, parameters and datums (implying a particular ellipsoid) compared to the *AUTO2:* namespace for CRSs as specified in the WMS Standard. The *AUTO2* namespace is used for "automatic" coordinate reference systems.

Finally, *OGC API — Maps* can also be used to represent a vertical dimension. This can be done either by returning a 3D data format, by selecting a vertical subset of features to be rendered on a 2D output, or by rendering a vertical profile. Some examples include:

- a 3D map to be displayed on virtual reality devices,
- a 1D graph of NDVI over time for a particular point,
- a 1D graph of a temperature over a vertical profile for a given spatial point,
- intensity of rain along a latitude and vertical dimension,
- a video over a temporal axis,
- a video of temperature representing different elevation levels in different frames of the video.

C.2.2. Correspondence between WMS map metadata and OGC API Standards

In the WMS Standard, the `GetCapabilities` response provides some metadata about the server and individual `layer` sections that inform the client about layer characteristics and some restrictions useful to formulate a successful map query. In the OGC API — Maps Standard, the equivalent metadata is provided via the landing page, the list of collections, the collections details, the API definition, and the service-meta link from the landing page. Implementers of Web APIs are encouraged to make use of the mechanisms provided by other standards of the OGC API family to communicate the relevant metadata to the client.

The following table provides a reference to where some of metadata aspects at the service level are specified.

Table C.1 — Where some "service" metadata elements are specified

NAME IN <SERVICE> WMS 1.3	WHERE IN THE API	PROPERTY	SPECIFIED IN
Title	service metadata[2]	title	OGC API — Common — Part 1
Name fixed to "WMS"	N/A		
Abstract	service metadata[2]	description	OGC API — Common — Part 1
OnlineResource	landing page	links	OGC API — Common — Part 1
Keywords	service metadata[2]	x-keywords	OGC API — Common — Part 1, Annex B.4
LayerLimit	service metadata[2]	x-OGC-limits.maps.max Collections	This standard
MaxWidth MaxHeight	service metadata	x-OGC-limits.maps.max Width x-OGC-limits.maps. maxHeight x-OGC-limits. maps.maxPixels [1]	This standard Requirements Class "Scaling"
Fees	N/A		
AccessConstraints	N/A		

[1] `x-OGC-limits.maps.maxWidth`, `x-OGC-limits.maps.maxHeight` and `x-OGC-limits.maps.maxPixels` are intended to control the work load of the server by providing limitations in size of the outputs of the subset. `width` and `height` parameters in *OGC API — Maps* (defined in Requirements Class "Scaling") control the size of the response and its resolution. The core of OGC API — Maps does not provide explicit limits on the size and resolution, but the server is free to respond with an error to avoid work overload. `width` and `height` parameters are commonly related to the size of the device screen. The fact that new devices are being built with more and more available display pixels. As such, specifying a reasonable limit on the server side based on today's technology may become too restrictive for future devices. [2] service metadata may be provided as an extension of the `info` section of the Open API document as indicated in OGC API — Common — Part 1, Annex B.4

The following table provides a reference to where some of layer metadata aspects are specified.

Table C.2 — Where some "layer" metadata elements are specified

NAME IN WMS 1.3 <LAYER>	WHERE IN THE API	PROPERTY	SPECIFIED IN
Title	collection response	title	OGC API — Common — Part 2
Name	collection response	id	OGC API — Common — Part 2
Abstract	collection response	description	OGC API — Common — Part 2
Keywords	collection response	keywords	OGC API — Records (Local Resources Catalogue)
Style	style response	id	OGC API — Styles — Part 1
EX_GeographicBoundingBox	collections response	extent.spatial.bbox	OGC API — Common — Part 2
CRS	collection response	storageCrs	This standard
BoundingBox	collection response	extent.spatial.storageCrsBbox[1]	This standard
minScaleDenominator maxScale Denominator	collection response	minScaleDenominator maxScale Denominator	Possibly in OGC API — Common — Part 2
Sample Dimensions	collection response	extent [2]	This standard
MetadataURL	collection response	link with rel describedBy	OGC API Common — Part 2
Attribution	collection response	attribution	OGC API — Common — Part 2
Identifier AuthorityURL	collection response	externalIds	OGC API — Records (Local Resources Catalogue)
FeatureListURL	items response		OGC API — Features provides this capability
DataURL			OGC API — Features, Coverages and EDR provide download capabilities
queryable [3]			OGC API — Features, Coverages and EDR provide query capabilities

NAME IN WMS 1.3 <LAYER>	WHERE IN THE API	PROPERTY	SPECIFIED IN
cascaded [4] noSubsets [5] fixed Width [6] fixedHeight [7]	N/A		

[1] In WMS it was possible to specify one bounding box for each supported output CRS. In OGC API — Maps, it is only provided for the native CRS (storageCrs). [2] If extra dimensions are supported the range of values are defined in additional properties of the 'extent' of the collection. [3] No equivalent functionality to GetFeatureInfo is provided so this flag is not applicable to OGC API — Maps. Please use the query capabilities of other OGC API Standards instead. [4] The `cascaded` XML attribute in WMS is removed because no practical use has been seen. `cascaded` indicated if a map was generated by the addressed service or by another service assisting the first one. [5] The `noSubsets` XML attribute in WMS was used to indicate lack of subsetting support. The client will know if the server does not support the *spatial subsetting*, *date and time* (for temporal subsetting) or *general subsetting* conformance class by inspecting its conformance declaration. [6] The `fixedWidth` XML attribute in WMS was used to indicate lack of scaling support. The client will know if the server does not support the *scaling* conformance class by inspecting its conformance declaration. [7] The `fixedHeight` XML attribute in WMS was used to indicate lack of scaling support. The client will know if the server does not support the *scaling* conformance class by inspecting its conformance declaration.

NOTE 1: The supported formats for map resources, or more precisely the media types of the supported encodings, can also be determined from the API definition. The desired encoding is selected using HTTP content negotiation. In addition to the parameters specified by the core, other parameters should be added.

NOTE 2: The `opaque` XML attribute in WMS was rarely useful and has been removed. This attribute indicated whether the map data represents features that probably do not completely fill space and shows the background opaque (true) or transparent (false).

C.2.3. No equivalent to *GetFeatureInfo* as part of the OGC API — Maps — Part 1

The OGC Web Map Service *GetFeatureInfo* operation provides the capability for clients to implement some simple level of user interaction with the map. In essence the user can focus on a point on the map (e.g., by clicking on it) and the client will request from the server some textual information related to the elements represented at that point of the map (a functionality sometimes called "query by location"). If the elements represented in the map are simple features, the result should be related to their properties (attributes). If the map represents a coverage, the result should report the value of the coverage in that position (eventually, if the coverage is multidimensional, it could be a e.g., time series graphic or a vertical profile). The format of the actual response is left to the discretion of the server.

GetFeatureInfo was first proposed in the 2000 version of the OGC Web Map Service Standard. In that environment *GetFeatureInfo* provided an easy to implement solution for the first step to "queryable" maps.

The new OGC API Standards emerged in a completely different context where most web content is dynamic and JavaScript is now a powerful programming language for the Web. Most

simplistic implementations of WMS *GetFeatureInfo* resulted in an imperfect presentation of the attribute text. Users demand much more than query by location. Now, the integration of the different building blocks defined in OGC API Standards can be provided by default. A map is connected to a collection (or a dataset) that is probably also offered as features with *OGC API — Features* or as coverage with *OGC API — Coverages* — all from the same API landing page. Furthermore, *OGC API — Environmental Data Retrieval (EDR)* also provides a point query, similar to *GetFeatureInfo* as well as much more advanced queries by polygons, trajectories or corridors.

Implementers of map clients are encouraged to implement support for OGC API Standards in addition to *OGC API — Maps* to provide a functionality similar to *GetFeatureInfo*. Instead of building a request to a map point in map coordinates (I, J), implementers should use point narrow bounding boxes in CRS coordinates. For example:

- In *OGC API — Features*, map coordinates should be transformed to Lon,Lat WGS84 in the client side and implement an HTTP GET request to `/collections/{collectionId}/items?bbox=Lon,Lat,Lon,Lat`.
- In *OGC API — Coverages*, map coordinates should be transformed to native coordinates and use `/collections/{collectionId}/coverage?bbox=x,y,x,y` or the equivalent "subset" query.
- In *OGC API — EDR*, map coordinates should be transformed to a CRS that the API supports `/collections/{collectionId}/position?coords=POINT(x y)` or by adding a radius query `/collections/{collectionId}/radius?coords=POINT(x y)&within=20&within-units=km`.

The use of *OGC API — Tiles* and serving vector tiled content directly also makes creating visualizations with query capabilities directly on the client side possible. Since tiled vector data can contain features, their attributes can be presented to the user when clicked, and a different style can be applied to highlight that selected feature.

NOTE 1: Even if the *OGC API — Maps — Part 1* does not provide a direct *GetFeatureInfo* equivalent, there is a strong tradition of *GetFeatureInfo* implementations that suggests a possible *OGC API — Maps* Standard future "part" could reintroduce a *GetFeatureInfo* equivalent — if users and implementers demand this capability.

NOTE 2: The second most commonly expected function, querying or filtering by the attributes of the features shown in the map, was never introduced in WMS. The same *OGC API — Maps* future "part" could provide the ability to filter by attributes using a CQL2 expression.

D

ANNEX D (INFORMATIVE) BACKGROUND COLOR CODES

ANNEX D (INFORMATIVE) BACKGROUND COLOR CODES

This is the list of W3C web color names which can be specified instead of a hexadecimal value and it is reproduced in the following table for convenience:

Table D.1 — Named web color enumeration

NAME	HEXADECIMAL	VALUE (R, G, B)
black	0x000000	0, 0, 0
dimGray	0x696969	105, 105, 105
dimGrey	0x696969	105, 105, 105
gray	0x808080	128, 128, 128
grey	0x808080	128, 128, 128
darkGray	0xa9a9a9	169, 169, 165
darkGrey	0xa9a9a9	169, 169, 165
silver	0xc0c0c0	192, 192, 192
lightGray	0xd3d3d3	211, 211, 211
lightGrey	0xd3d3d3	211, 211, 211
gainsboro	0xdcdcdc	220, 220, 220
whiteSmoke	0xf5f5f5	245, 245, 245
white	0xffffff	255, 255, 255
rosyBrown	0xbc8f8f	188, 143, 143
indianRed	0xcd5c5c	205, 92, 92

NAME	HEXADECIMAL	VALUE (R, G, B)
brown	`0xa52a2a`	165, 42, 42
fireBrick	`0xb22222`	178, 34, 34
lightCoral	`0xf08080`	240, 128, 128
maroon	`0x800000`	128, 0, 0
darkRed	`0x8b0000`	139, 0, 0
red	`0xff0000`	255, 0, 0
snow	`0xfffafa`	255, 250, 250
mistyRose	`0xffe4e1`	255, 228, 225
salmon	`0xfa8072`	250, 128, 114
tomato	`0xff6347`	255, 99, 71
darkSalmon	`0xe9967a`	233, 150, 122
coral	`0xff7f50`	255, 127, 80
orangeRed	`0xff4500`	255, 69, 0
lightSalmon	`0xffa07a`	255, 160, 122
sienna	`0xa0522d`	160, 82, 45
seaShell	`0xfff5ee`	255, 245, 238
chocolate	`0xd2691e`	210, 105, 30
saddleBrown	`0x8b4513`	139, 69, 19
sandyBrown	`0xf4a460`	244, 164, 96
peachPuff	`0xffdab9`	255, 218, 185
peru	`0xcd853f`	205, 133, 63

NAME	HEXADECIMAL	VALUE (R, G, B)
linen	`0xfaf0e6`	250, 240, 230
bisque	`0xffe4c4`	255, 228, 196
darkOrange	`0xff8c00`	255, 140, 0
burlyWood	`0xdeb887`	222, 184, 135
tan	`0xd2b48c`	210, 180, 140
antiqueWhite	`0xfaebd7`	250, 235, 215
navajoWhite	`0xffdead`	255, 222, 173
blanchedAlmond	`0xffebcd`	255, 235, 205
papayaWhip	`0xffefd5`	255, 239, 213
moccasin	`0xffe4b5`	255, 228, 181
orange	`0xffa500`	255, 165, 0
wheat	`0xf5deb3`	245, 222, 179
oldLace	`0xfdf5e6`	253, 245, 230
floralWhite	`0xfffaf0`	255, 250, 240
darkGoldenrod	`0xb8860b`	184, 134, 11
goldenrod	`0xdaa520`	218, 165, 32
cornsilk	`0xfff8dc`	255, 248, 220
gold	`0xffd700`	255, 215, 0
khaki	`0xf0e68c`	240, 230, 140
lemonChiffon	`0xfffacd`	255, 250, 205
paleGoldenrod	`0xeee8aa`	238, 232, 170

NAME	HEXADECIMAL	VALUE (R, G, B)
darkKhaki	0xbdb76b	189, 183, 107
beige	0xf5f5dc	245, 245, 220
lightGoldenRodYellow	0xfafad2	250, 250, 210
olive	0x808000	128, 128, 0
yellow	0xffff00	255, 255, 0
lightYellow	0xffffe0	255, 255, 224
ivory	0xfffff0	255, 255, 240
oliveDrab	0x6b8e23	107, 142, 35
yellowGreen	0x9acd32	154, 205, 50
darkOliveGreen	0x556b2f	85, 107, 47
greenYellow	0xadff2f	173, 255, 47
chartreuse	0x7fff00	127, 255, 0
lawnGreen	0x7cfc00	124, 252, 0
darkSeaGreen	0x8fbc8f	143, 188, 139
forestGreen	0x228b22	34, 139, 34
limeGreen	0x32cd32	50, 205, 50
lightGreen	0x90ee90	144, 238, 144
paleGreen	0x98fb98	152, 251, 152
darkGreen	0x006400	0, 100, 0
green	0x008000	0, 128, 0
lime	0x00ff00	0, 255, 0

NAME	HEXADECIMAL	VALUE (R, G, B)
honeyDew	`0xf0fff0`	240, 255, 240
seaGreen	`0x2e8b57`	46, 139, 87
mediumSeaGreen	`0x3cb371`	60, 179, 113
springGreen	`0x00ff7f`	0, 255, 127
mintCream	`0xf5fffa`	245, 255, 250
mediumSpringGreen	`0x00fa9a`	0, 250, 154
mediumAquaMarine	`0x66cdaa`	102, 205, 170
aquamarine	`0x7fffd4`	127, 255, 212
turquoise	`0x40e0d0`	64, 224, 208
lightSeaGreen	`0x20b2aa`	32, 178, 170
mediumTurquoise	`0x48d1cc`	72, 209, 204
darkSlateGray	`0x2f4f4f`	47, 79, 79
darkSlateGrey	`0x2f4f4f`	47, 79, 79
paleTurquoise	`0xafeeee`	175, 238, 238
teal	`0x008080`	0, 128, 128
darkCyan	`0x008b8b`	0, 139, 139
aqua	`0x00ffff`	0, 255, 255
cyan	`0x00ffff`	0, 255, 255
lightCyan	`0xe0ffff`	224, 255, 255
azure	`0xf0ffff`	240, 255, 255
darkTurquoise	`0x00ced1`	0, 206, 209

NAME	HEXADECIMAL	VALUE (R, G, B)
cadetBlue	`0x5f9ea0`	95, 158, 160
powderBlue	`0xb0e0e6`	176, 224, 230
lightBlue	`0xadd8e6`	173, 216, 230
deepSkyBlue	`0x00bfff`	0, 191, 255
skyBlue	`0x87ceeb`	135, 206, 235
lightSkyBlue	`0x87cefa`	135, 206, 250
steelBlue	`0x4682b4`	70, 130, 180
aliceBlue	`0xf0f8ff`	240, 248, 255
dodgerBlue	`0x1e90ff`	30, 144, 255
slateGray	`0x708090`	112, 128, 144
slateGrey	`0x708090`	112, 128, 144
lightSlateGray	`0x778899`	119, 136, 153
lightSlateGrey	`0x778899`	119, 136, 153
lightSteelBlue	`0xb0c4de`	176, 196, 222
cornflowerBlue	`0x6495ed`	100, 149, 237
royalBlue	`0x4169e1`	65, 105, 225
midnightBlue	`0x191970`	25, 25, 112
lavender	`0xe6e6fa`	230, 230, 250
navy	`0x000080`	0, 0, 128
darkBlue	`0x00008b`	0, 0, 139
mediumBlue	`0x0000cd`	0, 0, 205

NAME	HEXADECIMAL	VALUE (R, G, B)
blue	`0x0000ff`	0, 0, 255
ghostWhite	`0xf8f8ff`	248, 248, 255
slateBlue	`0x6a5acd`	106, 90, 205
darkSlateBlue	`0x483d8b`	72, 61, 139
mediumSlateBlue	`0x7b68ee`	123, 104, 238
mediumPurple	`0x9370db`	147, 112, 219
blueViolet	`0x8a2be2`	138, 43, 226
indigo	`0x4b0082`	75, 0, 130
darkOrchid	`0x9932cc`	153, 50, 204
darkViolet	`0x9400d3`	148, 0, 211
mediumOrchid	`0xba55d3`	186, 85, 211
thistle	`0xd8bfd8`	216, 191, 216
plum	`0xdda0dd`	221, 160, 221
violet	`0x40e0d0`	238, 130, 238
purple	`0x800080`	128, 0, 128
darkMagenta	`0x8b008b`	139, 0, 139
magenta	`0xff00ff`	255, 0, 255
fuschia	`0xff00ff`	255, 0, 255
orchid	`0xda70d6`	218, 112, 214
mediumVioletRed	`0xc71585`	199, 21, 133
deepPink	`0xff1493`	255, 20, 147

NAME	HEXADECIMAL	VALUE (R, G, B)
hotPink	`0xff69b4`	255, 155, 180
lavenderBlush	`0xfff0f5`	255, 240, 245
paleVioletRed	`0xdb7093`	219, 112, 147
crimson	`0xdc143c`	220, 20, 60
pink	`0xffc0cb`	255, 192, 203
lightPink	`0xffb6c1`	255, 182, 193

E

ANNEX E (INFORMATIVE) REVISION HISTORY

ANNEX E (INFORMATIVE) REVISION HISTORY

Table E.1

DATE	RELEASE	CONTRIBUTORS	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2019-03-21	0.0	C. Heazel	all	initial template
2019-06-28	T15-ER-0.00	J. Masó	all	initial version as work for the OGC API Hackathon in London
2019-09-09	T15-ER-0.01	J. Masó	all	Last version in the old document name with D014 and D16 together
2019-10-01	T15-ER-0.1	J. Masó	all	first version with the correct ToC as D014
2019-10-14	T15-ER-0.8	J. Masó	all	ready for OGC review
2019-10-21	T15-ER-0.9	J. Masó	all	results of the OGC review
2019-10-25	T15-ER-0.98	J. Masó	all	document ready for the 3 week rule in the Toulouse TC.
2019-12-09	T15-ER-0.99	C. Reed	various	Minor edits to Abstract, Exec Summary, and various other clauses prior to publication as a Public ER
2020-01-02	T15-ER-1.0	J. Masó	various	Final staff review and edits of Testbed-15 engineering report (OGC 19-069).
2023-04-20	0.8	J. Masó, J. St-Louis, B. Hards	all	Re-organized into several new requirements classes providing additional capability and flexibility, editorial fixes.
2023-05-02	1.0.0rc1	J. Masó, J. St-Louis	various	First version of OGC API — Maps submitted to the OAB for review.
2023-12-22	1.0.0rc2	J. Masó, J. St-Louis, A. Youmans, N. Julià, C. Reed	various	Implemented feedback from the OAB. Applied Carl Reed's suggestions. Added examples annex. Improved conformance and overview sections. Complete transition to https URIs. Added security considerations section. Clarified scaling and subsetting requirements classes.

DATE	RELEASE	CONTRIBUTORS	PRIMARY CLAUSES MODIFIED	DESCRIPTION
				Added JPEG XL requirements class. Several other improvements. Second version of OGC API — Maps submitted to the OAB for review.

BIBLIOGRAPHY

BIBLIOGRAPHY

[1] Jeff de La Beaujardiere, Open Geospatial Consortium: OGC 06-042, *OpenGIS Web Map Service (WMS) Implementation Specification*. Open Geospatial Consortium (2006). https://docs.ogc.org/is/06-042/06-042/pdf.

[2] Joan Maso Pau, Open Geospatial Consortium: OGC 19-069, *OGC Testbed-15: Maps and Tiles API Engineering Report*. Open Geospatial Consortium (2020). http://www.opengis.net/doc/PER/t15-D014.

[3] W3C: Data Catalog Vocabulary, W3C Recommendation 16 January 2014, https://www.w3.org/TR/vocab-dcat/

[4] W3C: Data on the Web Best Practices, W3C Recommendation 31 January 2017, https://www.w3.org/TR/dwbp/

[5] W3C/OGC: Spatial Data on the Web Best Practices, W3C Working Group Note 28 September 2017, https://www.w3.org/TR/sdw-bp/

[6] Internet Assigned Numbers Authority (IANA). Link Relation Types [online, viewed 2020-07-09], https://www.iana.org/assignments/link-relations/link-relations.xml